MINISTRY OF EDUCATION AND TRAINING
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING
FACULTY OF ADVANCED EDUCATION



COURSE REPORT

SEMESTER 1, 2026-2027

PROJECT: DESIGN AND VERIFICATION OF A RICS-V BASED CNN AI ACCELERATOR WITH LINUX GUI

Advisor: Huynh Hoang Ha, M.Eng.

Implementation Group: 7

1. Nguyen Phu Huu – 24161090
2. Nguyen Trung Nguyen – 24161115
3. Trinh Ngoc Tin – 24161140

Ho Chi Minh City, October 2026

PROGRESS REPORT

Name	ID	Progress
Nguyen Phu Huu	24161090	100%
Nguyen Trung Nguyen	24161115	100%
Trinh Ngoc Tin	24161140	100%

Instructor's comment:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Signature

Huynh Hoang Ha, M.Eng

Contents

LIST OF FIGURES i

LIST OF TABLES iv

LIST OF ABBREVIATIONS v

CHAPTER 1: INTRODUCTION 1

1.1. Background and Problem Statement 1

1.2. Problem Objectives 1

1.3. Scope and Design Approach 2

1.4. Report Organization 2

CHAPTER 2: THEORETICAL BACKGROUND 4

2.1. Embedded Linux system engineering 4

2.1.1. Embedded Linux Architecture 4

2.1.2. Concurrency and IPC Mechanisms 8

2.1.3. Hardware-Software Co-Design 13

2.1.4. Microarchitectural Optimization 16

2.1.5. Distributed Edge AI Deployment On LINUX 20

2.2. Theoretical background of RISC-V architecture 25

2.2.1. Introduction to RISC-V 25

2.2.2. RV32I Base Instruction Set 31

2.2.3. RISC-V Processor Architecture 37

2.3. Convolutional Neural Networks and Quantization 42

2.3.1. Overview of Convolutional Neural Networks 42

2.3.2. Two-Dimensional Convolution 43

2.3.3. Convolution Parameters and Tensor Dimensions 45

2.3.4. CNN Processing Layers 49

2.3.5. CNN Computational Requirements 53

2.3.6. Quantization for Hardware Implementation 56

2.4. CNN Hardware Acceleration 62

2.4.1. Motivation for CNN Hardware Acceleration 62

2.4.2 Multiply-Accumulate Unit 63

2.4.3. Processing Element 64

2.4.4. PE Array and Parallel Processing 65
2.4.5. Memory Hierarchy and On-Chip Buffers 66
2.4.6. Data Reuse and Dataflow 67
2.4.7. Accelerator Datapath 69
2.4.8. Accelerator Control Architecture 70
2.4.9. Hardware Accelerator Interface 71
2.4.10. Performance Metrics and Design Trade-offs 73
CHAPTER 3: SYSTEM ANALYSIS, DESIGN, AND IMPLEMENTATION 75
3.1. System Requirements and Design Objectives 75
3.1.1. Problem Definition 75
3.1.2. Functional Requirements 75
3.1.3. Constraints and Design Objectives 76
3.2. Overall System Architecture 76
3.2.1. System-Level Organization 77
3.2.2. Hardware and Software Partitioning 77
3.2.3. Interconnection and Address Map 78
3.2.4. End-to-End Operation 78
3.3. CNN Model Design and INT8 Deployment 79
3.3.1. Deployed Network 79
3.3.2. Training, Quantization, and Export 80
3.3.3. Input Samples and SD Image 81
3.4. CNN Hardware Accelerator Design 83
3.4.1 Datapath Overview 83
3.4.2. MAC Unit and Processing Element 83
3.4.3. Four-by-Four PE Array and Tiling 84
3.4.4. Local Buffers and Data Feeding 85
3.4.5. Address Generation, Sequencing, and Output 85
3.4.6. Controller and MMIO Protocol 86
3.5. RISC-V SoC and Memory Subsystem 87
3.5.1. RV32I Processor Architecture 87

3.5.2. Instruction and Data Memory Organization 89
3.5.3. SoC Bus and MMIO Integration 89
3.6. Peripheral and I/O Interface Design 90
3.6.1. UART Bootloader 90
3.6.2. SD-SPI and Raw RCN1 Storage 92
3.6.3. HDMI and User Interaction 94
3.7. Firmware Design and Hardware–Software Integration 95
3.7.1. Bare-Metal Software Structure 95
3.7.2. Loading and Integer Inference 95
3.7.3. Driver Sequence and Error Handling 96
3.7.4. One Complete User Transaction 96
3.8. FPGA Implementation 100
3.8.1. Source Organization and Tools 100
3.8.2. Constraints, Build, and Deployment 101
CHAPTER 4: EXPERIMENTAL RESULTS AND EVALUATION 103
4.1. Experimental Setup 103
4.1.1. Hardware Configuration 103
4.1.2. Evidence and Evaluation Scope 104
4.2. FPGA Implementation Results 105
4.2.1. FPGA Resource Utilization 105
4.2.2. Timing Analysis 106
4.3. Hardware Deployment Results 107
4.3.1. FPGA Configuration and Firmware Loading 107
4.3.2. Peripheral and User Interface Operation 108
4.4. CNN Inference Results 109
4.4.1. Single-Sample Inference Demonstration 109
4.4.2. Multiple-Sample Recognition Results 110
4.4.3. Classification Accuracy Evaluation 112
4.5. Results Discussion and Limitations 113
4.5.1. Interpretation of the Evidence 114
4.5.2. Current Limits and Next Measurements 114

4.6. RISC-V Instruction-Group Validation 115
4.6.1. Standalone Hardware Setup and UART Loading 115
4.6.2. Instruction Coverage and Results 116
CHAPTER 5: CONCLUSION AND FUTURE WORK 118
5.1. Project Conclusion 118
5.2. System Limitations 118
5.3. Future Development 119
REFERENCES 120

LIST OF FIGURES

Figure 2.1: Hardware rings and User/Kernel isolation boundary 5
Figure 2.2: System Call context switch sequence via SCI 7
Figure 2.3: Event-driven synchronous via Mutex and Condition Variables 11
Figure 2.4: Distributed topology bridging Linux Host and ESP32 13
Figure 2.5: Process Virtual Address Space Layout 17
Figure 2.6: Processor Affinity and Cache Optimization 18
Figure 2.7: Distributed Edge AI System Architecture.....20
Figure 2.8: Source Code Excerpt : Initializing the core process and capturing IPC.....21
Figure 2.9: Source Code Excerpt : Wake-up mechanism via Condition Variable.....22
Figure 2.10: Source Code Excerpt : Memory space preprocessing and Inference. 23
Figure 2.11: Edge AI Accelerator Interface.....24
Figure 2.12. RISC-V Development from Berkeley to an Open Standard.....25
Figure 2.13. The ISA as the Interface Between Software and Hardware.....26
Figure 2.14. RISC-V Base ISA and Standard Extensions 27
Figure 2.15. RISC-V Ecosystem: Software and Hardware Development Paths.....28
Figure 2.16. RV32I Single-Cycle Processor Datapath 29
Figure 2.17. RISC-V SoC with ML Accelerators, Memory, and Peripherals 30
Figure 2.18. RV32I instruction formats.....31
Figure 2.19. R-type bit distribution. 32
Figure 2.20. R-type instructions. 32
Figure 2.21. I-type bit distribution. 32
Figure 2.22. I-type instructions. 33
Figure 2.23. S-type bit distribution.....34
Figure 2.24. S-type instructions.....34
Figure 2.25. B-type bit distribution. 34
Figure 2.26. B-type instructions. 35
Figure 2.27. U-type bit distribution. 35
Figure 2.28. U-type instructions. 35
Figure 2.29. J-type bit distribution. 36
Figure 2.30. J-type instructions. 36

Figure 2.31. Program Counter block. Illustrated based on [9], [10]. 38
Figure 2.32. Instruction Memory block. Illustrated based on [7], [11]. 38
Figure 2.33. Register File block. Illustrated based on [9], [11].....39
Figure 2.34. Immediate Generator block. Illustrated based on [9], [13]. 39
Figure 2.35. Control Unit block. Illustrated based on [12]. 40
Figure 2.36. Arithmetic Logic Unit block. Illustrated based on [14]. 41
Figure 2.37. Data Memory block. Illustrated based on [15]. 41
Figure 2.38. General architecture of a Convolutional Neural Network. 43
Figure 2.39. Illustration of a convolution kernel operating on a local region of an input feature map 44
Figure 2.40. Effect of padding and stride on convolution output dimensions, adapted from [21].....47
Figure 2.41. Common CNN processing operations; dropout is applied only during training.....51
Figure 2.42. Signed-symmetric INT8 convolution datapath considered in this work. .60
Figure 2.43. Basic architecture of a multiply-accumulate unit. Illustrated by the authors based on [24]. 64
Figure 2.44. General architecture of a processing element for CNN acceleration. Illustrated by the authors based on [24], [25].....65
Figure 2.45. Generic processing-element array for parallel CNN computation. Illustrated by the authors based on [24], [25].....66
Figure 2.46. Memory hierarchy and on-chip data storage in a CNN accelerator. Illustrated by the authors based on [24], [25].....67
Figure 2.47. Comparison of weight-stationary, output-stationary, and row-stationary dataflows. Illustrated by the authors based on [24], [25]. 68
Figure 2.48. General datapath architecture of a CNN hardware accelerator. Illustrated by the authors based on [24], [25]. 70
Figure 2.49. Example control-state sequence of a CNN accelerator. Illustrated by the authors. 71
Figure 3.1. Implemented RISC-V CNN SoC organization and external interfaces. 77
Figure 3.2. End-to-end operation sequence from boot to result display.....79

Figure 3.3. Hardware and software execution flow of the deployed CNN. 80
Figure 3.4. RCN1 Image Structure and Sample Record Organization 82
Figure 3.5. Functional blocks and data movement inside the convolution accelerator. 83
Figure 3.6. Signed MAC and accumulator update in one processing element. 84
Figure 3.7. Spatial and output-channel mapping of one 4 x 4 PE tile. 85
Figure 3.8. Accelerator Controller and Sequencer State Transitions 86
Figure 3.9. Detailed datapath and bus interface of the implemented RV32I processor. 89
Figure 3.10. MMIO interconnection of the implemented RISC-V CNN SoC. 90
Figure 3.11. Firmware inference stages and error exits. 96
Figure 3.12. External-image dataset tab before file selection 98
Figure 3.13. External handwritten digit 8 and processed 28 x 28 input 98
Figure 3.14. UART log confirming custom-image replacement, CRC32 verification, and AI Runtime 99
Figure 4.1. Experimental hardware setup of the RISC-V CNN system. 104
Figure 4.2. Gowin resource screenshots showing the values transcribed in Table 4.1. 106
Figure 4.3. Setup and Hold Path Slack Values 107
Figure 4.4. FPGA configuration and UART loading evidence 108
Figure 4.5. HDMI display for IMG 001 110
Figure 4.6. HDMI observations for IMG 065, IMG 074, and IMG 094 112
Figure 4.7. HDMI result for the externally supplied handwritten digit 8 111
Figure 4.8. Tang Nano 20K setup used for standalone UART-based RV32I instruction testing 116
Figure 4.9. UART test output confirming that all six RV32I instruction groups passed 117

LIST OF TABLES

Table 2.1. Comparison of Common Dataflow Strategies in CNN Accelerators 69
Table 2.2. MMIO Register Map of the CNN Hardware Accelerator 72
Table 2.3. Performance Metrics for CNN Hardware Accelerators.....74
Table 3.1. Functional and hardware requirements of the implemented system 76
Table 3.2. Processing responsibility by layer or service 77
Table 3.3. CPU-visible memory map used by the C firmware.....78
Table 3.4. Layer dimensions and implementation.....80
Table 3.5. Numeric formats and exported data used by integer inference 81
Table 3.6. Accelerator-local storage in the RTL.....85
Table 3.7. Host-visible accelerator control and status.....87
Table 3.8. UART loader packet format and response bytes 91
Table 3.9. Principal board-level interfaces 95
Table 3.10. End-to-end transaction from selection to display.....99
Table 3.11. Project sources and build roles 101
Table 4.1. FPGA resource utilization reported in the supplied Gowin screenshots. ...106
Table 4.2. Frequency and timing values shown in Gowin 107
Table 4.3. FPGA display results and ground-truth labels.....112
Table 4.4. Real-image evidence plan for the external-image workflow 113
Table 4.5. RV32I instruction-group validation coverage and results 117

LIST OF ABBREVIATIONS

Abbreviation	Full term
AI	Artificial Intelligence
ALU	Arithmetic Logic Unit
ASIC	Application-Specific Integrated Circuit
BRAM	Block RAM
BSRAM	Block Static RAM
CFS	Completely Fair Scheduler
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CRC32	32-bit Cyclic Redundancy Check
DMEM	Data Memory
DSP	Digital Signal Processing
EDA	Electronic Design Automation
FC	Fully Connected
FPGA	Field-Programmable Gate Array
GUI	Graphical User Interface
HDMI	High-Definition Multimedia Interface
IMEM	Instruction Memory
INT8	Signed 8-bit Integer
INT32	Signed 32-bit Integer
IPC	Inter-Process Communication
ISA	Instruction Set Architecture
MAC	Multiply-Accumulate
MMIO	Memory-Mapped Input/Output
MMU	Memory Management Unit
MNIST	Modified National Institute of Standards and Technology database
ONNX	Open Neural Network Exchange
OS	Operating System
PC	Program Counter
PE	Processing Element
POSIX	Portable Operating System Interface

Abbreviation	Full term
RAM	Random-Access Memory
ReLU	Rectified Linear Unit
RISC-V	RISC-V instruction set architecture
RTL	Register-Transfer Level
RV32I	32-bit RISC-V Base Integer Instruction Set
SCI	System Call Interface
SD	Secure Digital
SoC	System-on-Chip
SPI	Serial Peripheral Interface
SRAM	Static Random-Access Memory
TLB	Translation Lookaside Buffer
TMDS	Transition-Minimized Differential Signaling
UART	Universal Asynchronous Receiver/Transmitter
UI	User Interface
USB	Universal Serial Bus
VAS	Virtual Address Space
VFS	Virtual File System

CHAPTER 1: INTRODUCTION

This chapter introduces the need for an FPGA-based handwritten-digit inference system and defines the project scope. The design is treated as a complete embedded AI system: a fixed FPGA hardware layer supplies the processor and peripherals, reloadable RV32I firmware controls inference, and a microSD image supplies the model and the 100-sample dataset. Detailed register maps, protocols, and RTL state machines are reserved for Chapter 3.

1.1. Background and Problem Statement

Handwritten-digit recognition is a compact edge-AI workload that still requires image storage, learned convolution, intermediate-tensor management, classification, and a visible result. CNNs are well suited to spatial image data, but their repeated multiply-accumulate operations and memory traffic are costly for a small embedded processor [17]-[19].

An FPGA allows the control-oriented and compute-oriented parts of the application to be separated. The RV32I processor manages boot, storage, peripheral protocols, layer sequencing, and exceptional conditions. A dedicated INT8 convolution engine exploits the regular parallelism of Conv1 and Conv2. The Tang Nano 20K is therefore used as a complete AI SoC prototype rather than as a standalone accelerator block.

The project addresses three linked constraints. Rebuilding a bitstream for every firmware change slows development; the on-chip memories cannot hold the complete model and a reusable image collection; and an accelerator result is not useful unless the system can be loaded, controlled, and observed. The implemented organization separates a Flash-resident FPGA image, UART-loaded firmware in IMEM and DMEM, and an RCN1 v3 model plus dataset on microSD.

1.2. Problem Objectives

The principal objective is to implement a ten-class CNN inference system controlled by a team-developed RV32I processor on the Tang Nano 20K. The fixed FPGA image must integrate the processor, two 8 KiB Harvard memories, MMIO interconnect, onboard BL616 UART path, SD-SPI controller, CNN accelerator, result handoff, user controls, and HDMI output.

The firmware objective is to support independent loading of IMEM and DMEM at 115200-8-N-1 with CRC32-checked packets, then execute storage management, pooling, requantization, FC1, FC2, Argmax, and result publication. The AI-data objective is to store exported model sections and 100 updateable image records on microSD without requiring the model to be retrained when an image is replaced.

The demonstration objective is to provide one PC application for COM discovery, firmware transfer, FPGA Flash programming, microSD initialization, image preprocessing, dataset updates, progress reporting, and logs. S1 initiates inference, S2 selects images without repetition inside a 100-sample cycle, and a separate pin-72 button held for 0.5 s returns the SoC to the UART loader without rewriting the .fs hardware image.

1.3. Scope and Design Approach

The scope is inference for one-channel 28 × 28 handwritten-digit images and classes 0 through 9. Conv1 and Conv2 with ReLU execute in RTL. Max pooling, requantization, FC1, FC2, and Argmax remain bare-metal RV32I workloads. The design does not claim to accelerate the complete neural network.

The project follows a three-layer update model. The FPGA bitstream contains hardware and is stored in external Flash. Firmware is transferred through the onboard BL616 bridge into separate IMEM and DMEM after configuration. Model parameters, quantization metadata, and 100 image records are stored in the raw RCN1 v3 microSD image. This separation lets firmware change without place-and-route and lets an existing class 0-9 image slot change without retraining.

The technical contribution is the integration of a reloadable-firmware FPGA SoC, a modular 4 × 4 signed-INT8 accelerator, verified raw-sector storage updates, a nonrepeating selector, a 720 × 480p HDMI presentation path, and a host control application. Evidence is reported at its actual level: source implementation, simulation, Gowin synthesis and place-and-route, or direct board observation.

1.4. Report Organization

Chapter 2 reviews embedded systems, RISC-V, CNN quantization, and acceleration principles. Chapter 3 presents the implemented architecture from system level to each major module. Chapter 4 reports model, simulation, FPGA

implementation, and available board evidence without treating one evidence type as another. Chapter 5 summarizes the design, limitations, and future work.

CHAPTER 2: THEORETICAL BACKGROUND

2.1. Embedded Linux system engineering

2.1.1. Embedded Linux Architecture

Privilege Domains and Memory Isolation

The fundamental stability, determinism, and fault-resilience of the Linux operating system are rooted in the hardware-enforced separation of execution privilege domains. Modern microprocessors, equipped with a Memory Management Unit (MMU), establish discrete operational modes mapped as hierarchical protection rings. Within the POSIX and Linux architectural paradigm, this hardware support is strictly bifurcated into two functional boundaries: Supervisor Mode (commonly designated as Ring 0 or Kernel Space) and Non-Privileged Mode (designated as Ring 3 or User Space).

Kernel Space encapsulates the core runtime responsible for the unmediated governance of the underlying physical platform. Executing at the absolute highest hardware privilege level, the kernel possesses unrestricted access to all machine instructions, system control registers, interrupt management vector tables, and physical memory pages. It orchestrates the device driver subsystem, oversees peripheral interactions via memory-mapped input-output (MMIO), and executes core routines without spatial boundaries. Because fault containment is non-existent within this execution ring, an unhandled exception or illegal memory dereference originating within Kernel Space precipitates a fatal system state, invariably manifesting as a kernel panic.

Conversely, User Space is engineered as a highly constrained runtime container, designed to isolate arbitrary application code from direct hardware manipulation. Each user-space process operates within a discrete Virtual Address Space (VAS), instantiated and rigidly policed by the kernel's virtual memory subsystem. Through multi-level page translation tables maintained by the kernel and hardware-accelerated by the CPU's Translation Lookaside Buffer (TLB), virtual addresses referenced by user applications are dynamically resolved to physical memory frames. This hardware-backed abstraction guarantees strict spatial isolation: an errant process cannot access, inspect, or corrupt the address space allocated to peer processes or the kernel itself. Any unauthorized attempt to breach these boundaries triggers a hardware page fault exception, compelling the

kernel to intercept the violation and instantaneously terminate the offending process with a segmentation violation signal (SIGSEGV), thereby preserving global system integrity.

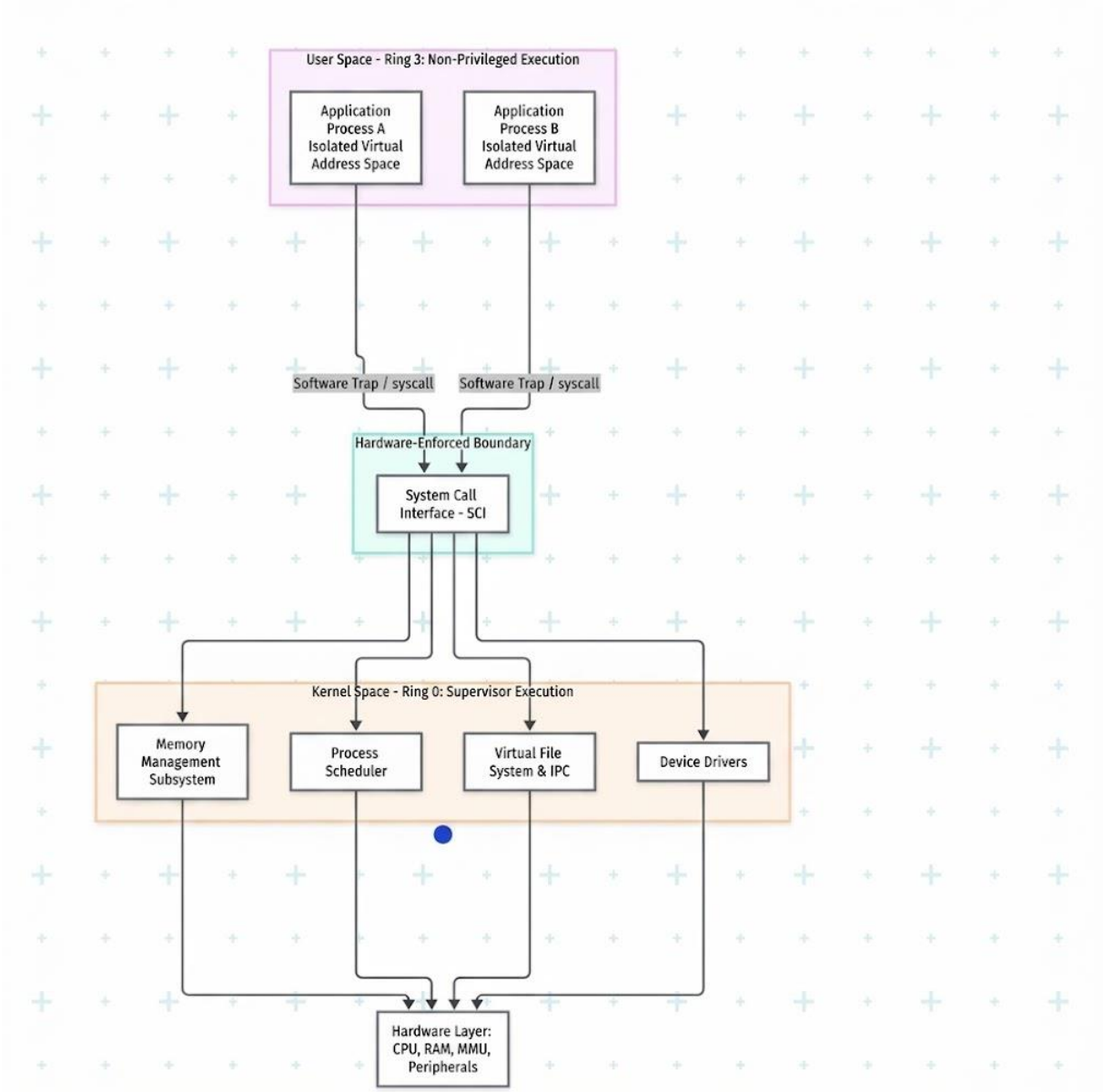


Figure 2.1: Hardware rings and User/Kernel isolation boundary

The System Call Interface (SCI) as an Architectural Gateway

Because user-space software is strictly stripped of raw hardware I/O instructions, any legitimate request for underlying system resources-such as secondary storage access, peripheral manipulation, execution scheduling alterations, or network communications-must transition across the privilege boundary. This transition is brokered exclusively by the System Call Interface (SCI). The SCI functions not as a

standard synchronous function call, but as a heavily regulated architectural gatekeeper designed to reconcile execution privilege disparities.

When an application invokes a system call (e.g., **fork()**, **read()**, or **mmap()**), the execution flow does not branch directly into kernel code. Instead, standard C library routines (such as **glibc** or **musl**) wrap the invocation by loading a standardized system call numeric identifier into a specific CPU register, alongside the operational parameters. Subsequently, the library executes an architecture-defined software trap instruction (e.g., **syscall** on x86_64 architectures or **svc** on ARM platforms).

This instruction induces an immediate hardware-level context switch. The processor suspends user-space execution, elevates its operational mode from Ring 3 to Ring 0, saves the application's current register context onto the per-process kernel stack, and transfers instruction control to the kernel's master vector dispatch handler. Within this highly privileged context, the kernel rigorously validates the parameters against security policies and memory permission flags to preempt arbitrary memory tampering or buffer overflow exploits.

Upon successful validation, the kernel indexes the master system call table (**sys_call_table**) using the registered numerical identifier and dispatches the execution flow to the corresponding internal kernel service routine. Once the requested hardware manipulation or logical operation concludes, the kernel places the execution return status (or negative error code) into the designated return register, restores the preserved user process context, lowers the hardware execution mode back to non-privileged space, and returns control to the user-space instruction immediately following the initial trap. This rigorous round-trip mechanism maintains complete physical authority within the OS core while projecting a uniform, POSIX-compliant programming interface to user-space applications.

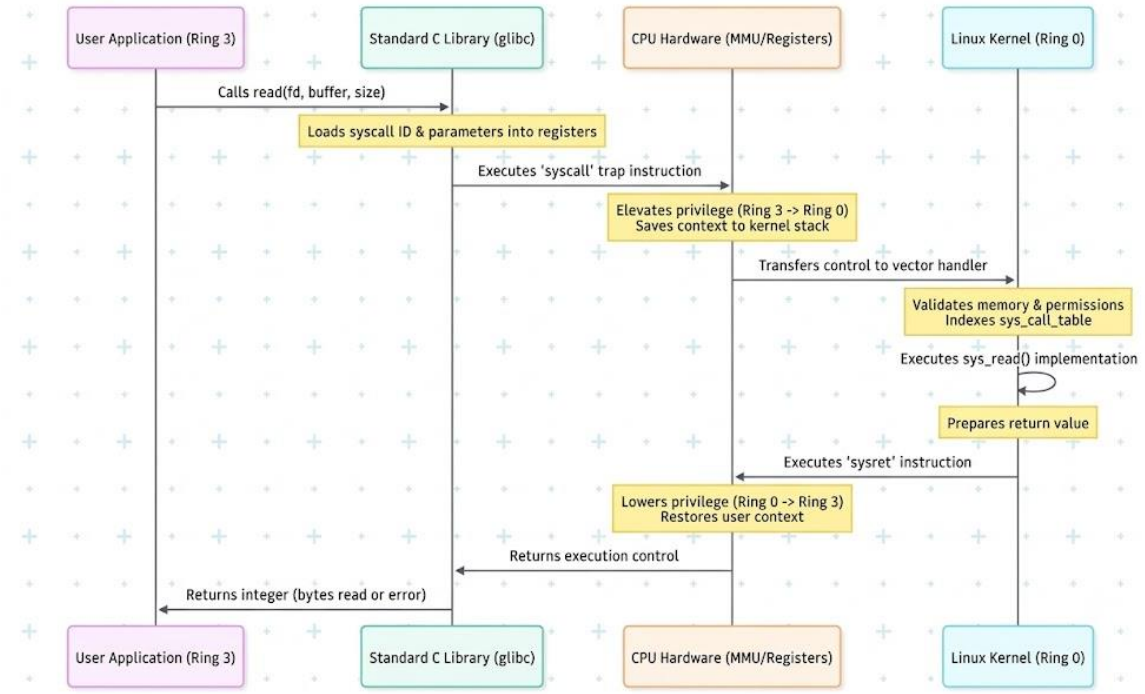


Figure 2.2: System Call context switch sequence via SCI

Essential Kernel Subsystems

When deploying the Linux kernel within resource-constrained embedded systems, the architecture must remain exceptionally lean while preserving structural determinism. Unlike general-purpose server environments burdened with redundant background daemons, expansive utility suites, and graphical subsystems, an embedded Linux system minimizes user space to bare operational essentials. This is frequently achieved through compact, multi-call executable toolkits such as BusyBox. Beneath this lean configuration, three primary kernel subsystems constitute the functional core of the environment.

The Memory Management Subsystem: This component orchestrates virtual memory allocation, demand paging, and dynamic memory recycling. Under the severe physical RAM limitations typical of microcomputing platforms, this subsystem enforces rigorous page replacement strategies and balances active and inactive page caches. It allocates kernel memory pools utilizing highly optimized allocators (**slab**, **slub**, or **slob**) designed specifically for small memory footprints to minimize internal fragmentation. Furthermore, when aggregate memory demand exceeds physical capacity-especially in the absence of swap partition-the kernel activates the Out-Of-Memory (OOM) killer.

This heuristic-driven mechanism selectively terminates rogue or non-essential user tasks based on computed "badness" scores, thereby averting total hardware lockup and preserving mission-critical processes.

The Process Scheduler: This subsystem dictates the temporal allocation of physical execution units (CPU cores) across concurrent threads and processes. Relying primarily on the Completely Fair Scheduler (CFS), the kernel employs red-black tree data structures to ensure proportional CPU processing time based on dynamic task weighting and virtual runtime (**vruntime**) tracking. However, in embedded configurations demanding stringent real-time guarantees (e.g., motor control or high-frequency sensor sampling), the scheduler incorporates deterministic POSIX real-time policie - namely **SCHED_FIFO** (First-In, First-Out) and **SCHED_RR** (Round-Robin). These policies bypass CFS logic, instantaneously preempting standard fair-share execution whenever high-priority hardware-servicing tasks transition into the ready state.

The Virtual File System (VFS): The VFS provides a standardized abstraction layer that unifies access to vastly disparate physical media and ephemeral kernel structures, adhering to the foundational UNIX design doctrine that treats all I/O interfaces as continuous streams of bytes. Through uniform function pointer structures defined within file operations tables, the VFS enables identical API invocation - such as **open(), read(), write(),** and **close() -** to interact seamlessly with flash-based file systems (e.g., UBIFS, SquashFS), network filesystems, and character devices. Crucially for embedded programming, it manages pseudo-filesystems such as **/proc** and **/sys**. These pseudo-filesystems act as direct windows into active kernel data structures, allowing user-space software to configure hardware clocks, manage general-purpose input/output (GPIO) pins, and extract runtime metrics directly through standard file operations, mitigating the need to author custom kernel-space device drivers.

2.1.2. Concurrency and IPC Mechanisms

This section establishes the formal academic foundation of system-level concurrency architectures, inter-process communication (IPC) mechanisms, and the foundational principles of resource synchronization alongside thread execution coordination within POSIX-compliant Linux operating system environments.

The POSIX Process Mode

The POSIX process model defines an autonomous execution unit allocated an independent virtual address space, an isolated file descriptor table, and hardware execution contexts managed through the **task_struct** kernel representation. The temporal progression of a process is modeled mathematically as a finite-state machine:

$$S = \{S_{new}, S_{ready}, S_{running}, S_{blocked}, S_{zombie}, S_{terminated}\}$$

CPU time-slice distribution is regulated by the Completely Fair Scheduler (CFS) via virtual runtime metrics, calculated as:

$$vruntime_i += \Delta exec_time \times \left(\frac{W_0}{W_i}\right)$$

(where Wi denotes the static scheduling weight corresponding to the process's nice level).

Process lifecycle orchestration is governed by standardized system call interfaces. The **fork()** system call duplicates the parent image into an independent child process utilizing Copy-On-Write (COW) semantics. This optimization ensures that physical page frames remain shared in a read-only state until a write operation triggers a page fault, prompting the kernel to allocate an independent physical frame. The **execve()** system call supplants the active memory segments and address space with a new executable binary while preserving the existing Process Identifier (PID).

Upon termination (via **exit()**), a child process enters the S_{zombie} state, preserving its termination status within the kernel process table. The parent process coordinates synchronization and resource reclamation using **waitpid()**, which suspends the caller until the child state transitions occur. This operation allows the kernel to completely reclaim the associated structural control blocks, preventing resource exhaustion.

POSIX Threads (pthreads)

POSIX threads serve as the fundamental schedulable entities in multithreaded systems, enabling concurrent execution within a single unified virtual address space. Unlike autonomous processes requiring isolated page translation tables enforced by the Memory Management Unit (MMU), peer threads within a host process share the common heap allocation pool, static global memory segments **(.data, .bss**), and open file descriptor handles. They maintain private execution contexts strictly through

dedicated per-thread stacks, stack pointers, and CPU register sets. This shared-memory architectural model significantly reduces context-switching latency by eliminating Translation Lookaside Buffer (TLB) flushes and avoiding page table reloading during scheduling transitions.

Standardized POSIX execution lifecycle control is exposed via the native **libpthread** interface. The **pthread_create()** primitive initiates thread instantiations, which are mapped internally to the Linux **clone()** system call configured with aggressive sharing flags (e.g., **CLONE_VM, CLONE_FS, CLONE_FILES, and CLONE_SIGHAND**). Coordinating execution paths synchronize their completion using **pthread_join(),** which acts as a barrier suspending the caller until target thread termination occurs, subsequently reclaiming the target's mapped stack allocations. Conversely, decoupled routines configured through **pthread_detach()** allow automatic resource reclamation upon thread exit without requiring synchronization handshakes with sibling threads.

Event-Driven Synchronization

Data races manifest when concurrent threads perform uncoordinated read and write operations on shared memory locations, resulting in non-deterministic system states and silent memory corruption. A mutual exclusion primitive (Mutex) resolves resource contention within critical sections by enforcing strict serialization. Mathematically, it guarantees the invariant that for any temporal coordinate t, access to the critical section (CS) by threads T_i and T_j is strictly non-overlapping:

$$\forall i \neq j, (T_i \in CS) \cap \left(T_j \in CS\right) = \emptyset$$

On Linux platforms**, pthread_mutex_t** leverages the Fast Userspace Mutex (Futex) architecture. It executes entirely within user space via hardware atomic primitives (e.g., **CMPXCHG**) in uncontended scenarios, transitioning execution to kernel space via the **sys_futex()** system call only when contention arises, efficiently suspending the contending thread into a kernel wait-queue.

However, a Mutex alone cannot efficiently coordinate execution states between data-dependent threads. In pipelined execution paradigms (e.g., a network ingestion thread feeding a downstream AI inference thread), a consumer thread attempting repeated non-blocking lock acquisitions incurs severe CPU cycle wastage due to busy-

waiting. To eliminate this inefficiency, POSIX couples Condition Variables (**pthread_cond_t**) with mutex locks to implement true event-driven synchronization.

The **pthread_cond_wait()** primitive atomically releases the associated mutex and transitions the calling thread into a deep sleep state within the kernel wait-queue, releasing all CPU resources. Once the producer thread enqueues fresh data into the shared buffer, it invokes **pthread_cond_signal()** to wake up the waiting consumer, which subsequently reacquires the mutex lock before proceeding. To guard against spurious wakeups triggered by OS anomalies, state verification is wrapped inside a conditional **while** loop, guaranteeing system state validity prior to critical section entry.

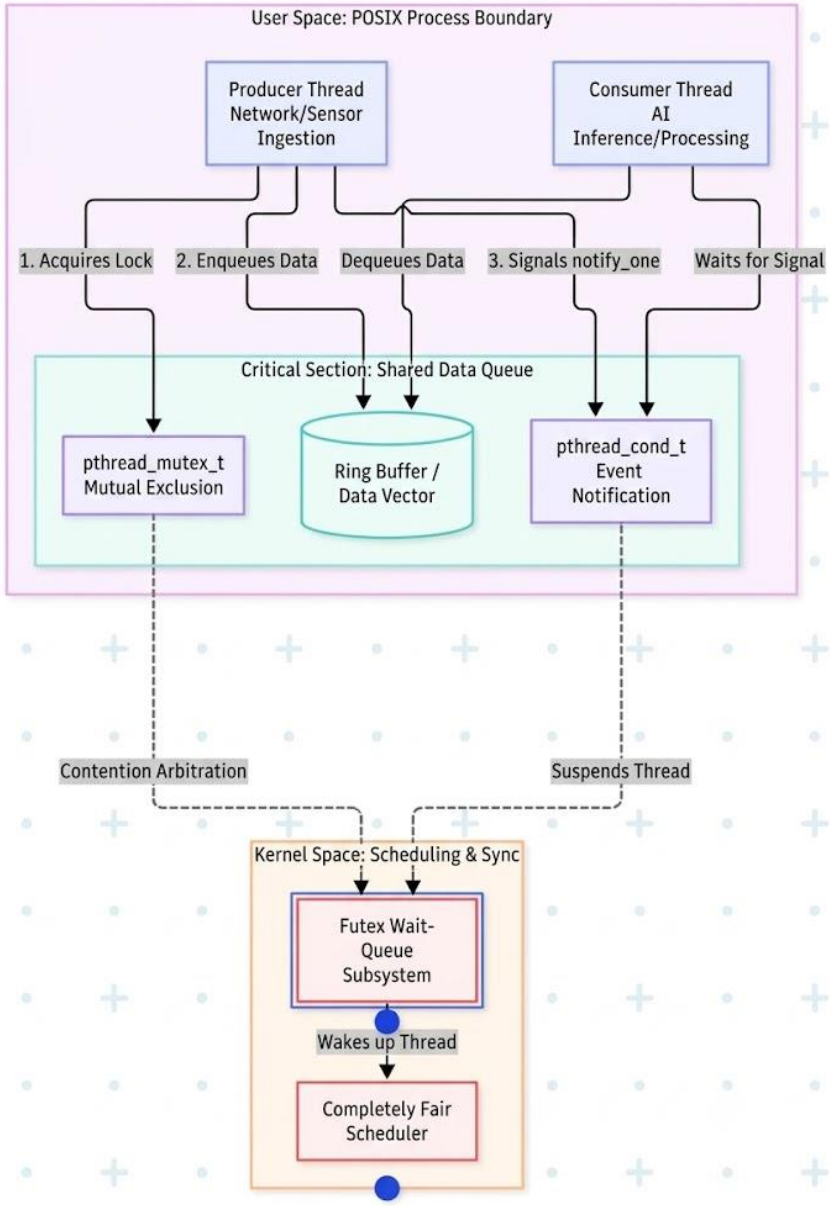


Figure 2.3: Event-driven synchronous via Mutex and Condition Variables

Inter-Process Communication (IPC)

Inter-process communication mechanisms establish structured data exchange channels across isolated virtual memory boundaries. An anonymous unidirectional pipe consists of a kernel-space circular ring buffer abstracted within the Virtual File System (VFS) layer. Transmission payloads satisfying size constraints less than **PIPE_BUF** (standardized to 4096 bytes under Linux POSIX implementations) guarantee atomicity during write operations, preventing data interleaving under concurrent multi-process writes. At stream boundaries, closing all write descriptors produces an End-Of-File (EOF) condition, resulting in read invocations returning zero bytes, whereas writing to a pipe whose read descriptors have been closed raises a **SIGPIPE** signal to indicate a broken channel.

Shared memory delivers optimal IPC throughput and minimal communication latency by directly mapping identical physical memory frames into the local virtual address spaces of disparate processes. By eliminating intermediate kernel-space buffer copies inherent in stream-oriented I/O paradigms, this architecture enables memory access speeds equivalent to local heap accesses. POSIX shared memory regions are provisioned and mapped into user space via **shm_open()** and **mmap()** interfaces. However, it mandates the implementation of robust user-space synchronization primitives (like named semaphores or cross-process mutexes) due to the absence of kernel-enforced mutual exclusion over the shared memory pages.

System Verification and Diagnostics

Validation of concurrency models, process tree hierarchies, and memory consistency relies on a synthesis of kernel virtual file system interfaces and dynamic binary instrumentation utilities. The **/proc** virtual file system exposes real-time kernel data structures to user space; specifically, the /**proc/[PID]/fd/** entry enumerates active file descriptors, anonymous pipes, and socket handles allocated to a target process, facilitating the systematic identification of descriptor leaks. Structural parent-child relationships are mapped via **pstree -p** by traversing pointers within corresponding **task_struct** instances, while dynamic process inspection tools such as **htop** track the distribution of Lightweight Processes (LWPs) across distinct physical execution cores.

To verify synchronization correctness, dynamic instrumentation with ThreadSanitizer (TSan) is incorporated via the **-fsanitize=thread** compiler flag. TSan instruments low-level memory load and store instructions alongside shadow memory tracking to record spatial and temporal memory access metadata. By applying happens-before relationship algorithms, ThreadSanitizer detects unsynchronized data races and prospective deadlock conditions at runtime with high fidelity, empirically verifying the formal correctness of mutual exclusion implementations across the target software architecture.

2.1.3. Hardware-Software Co-Design

Hardware-Software Co-Design is a foundational methodology in constructing modern embedded systems, where computational workloads are strategically partitioned between a centralized host and peripheral edge nodes. Integrating a distributed architecture between a POSIX-compliant Linux environment and a microcontroller device requires a highly deterministic and resilient communication pipeline. This integration represents the convergence of multiple architectural layers, encompassing network transport protocols, hardware-level serial interfaces, and event-driven graphical subsystems, ensuring the system satisfies strict real-time constraints while maintaining computational throughput.

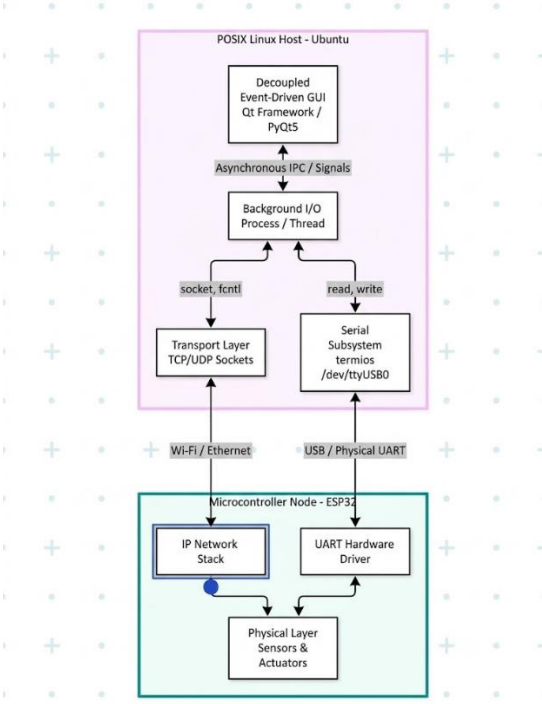


Figure 2.4: Distributed topology bridging Linux Host and ESP32

Distributed Edge Architecture

The Distributed Edge Architecture defines a topological workload distribution across two complementary entities: the POSIX Host and the Microcontroller Node. At the coordination layer, the central server (POSIX Host) acts as the primary computational brain, leveraging the expansive virtual memory space, multi-threading capabilities, and advanced file system of the Linux kernel to execute complex AI inferences, aggregate data, and render the graphical interface. Conversely, at the execution layer, the edge microcontroller node (ESP32) acts as the immediate interface to the physical environment.

Unlike the time-sharing POSIX environment governed by the Completely Fair Scheduler, the ESP32 operates utilizing a Real-Time Operating System (RTOS) or a bare-metal execution loop. This allows the edge node to process hardware sensor interrupts, debounce physical inputs, and actuate peripherals with microsecond latency. This architectural separation strictly adheres to the offloading principle: pushing hard real-time, highly synchronous tasks down to the deterministic microcontroller, while elevating heavy, asynchronous logical processing up to the Linux host server.

Non-blocking POSIX Sockets (TCP/UDP)

To maintain a continuous and reliable data stream between the Linux Host and the ESP32 over an IP network, the system implements a transport layer utilizing standard POSIX Sockets. A critical requirement for this implementation is the enforcement of non-blocking I/O semantics. By default, socket read and write operations in Linux are blocking, meaning the kernel will suspend the calling thread indefinitely if the network buffer is empty, leading to catastrophic system freezes in event-driven applications.

To circumvent this, the endpoint is initialized via the socket() function, and the non-blocking flag is explicitly applied using the file descriptor control function fcntl(sockfd, F_SETFL, O_NONBLOCK). Protocol selection depends strictly on data requirements: TCP is utilized for control command streams that demand absolute data integrity through a three-way handshake and packet retransmission, while UDP is employed for continuous telemetry data streams where minimal latency is prioritized over packet guarantee. Furthermore, to optimize I/O multiplexing without resorting to resource-intensive busy-waiting loops, stream monitoring system calls such as select(),

poll(), or epoll() are integrated. These mechanisms allow the background process to yield CPU cycles efficiently, remaining dormant until the kernel signals that network packets are physically present in the buffer.

Serial Hardware Interfacing

Beyond wireless networking, physical Serial Subsystems serve as an ultra-low latency, highly secure fallback communication channel utilizing the UART/USB protocol. Within the Linux UNIX-like architecture, this physical hardware interface is abstracted as a character device file (e.g., /dev/ttyUSB0) and is explicitly controlled via the POSIX termios library. The termios structure enables developers to manipulate physical bitstream parameters, such as defining the baud rate via cfsetospeed() and cfsetispeed(), and committing the configuration changes to the hardware driver using the tcsetattr() function.

A critical optimization at this layer involves the reconfiguration of the kernel's Line Disciplines. The Line Discipline is a software intermediate layer residing between the hardware UART driver and user space. By default, Linux configures terminal devices in Canonical mode, which buffers incoming data line-by-line until a newline or carriage return character is detected. While suitable for human-readable text, Canonical mode actively corrupts and delays binary telemetry payloads. Consequently, the termios configuration must disable parsing flags (such as ICANON and ECHO) to force the interface into Raw mode. This modification bypasses kernel-level character filtering, enabling the Linux host application to securely read high-speed binary payloads byte-by-byte precisely as they are transmitted by the ESP32 microcontroller.

Event-Driven GUI Integration

At the highest supervisory tier, the Graphical User Interface (GUI) is engineered utilizing an event-driven paradigm (such as the Qt/PyQt framework) and is structurally decoupled from all underlying I/O processing operations. A fundamental anti-pattern in UI design is executing blocking I/O tasks - such as polling a serial port or awaiting socket data - directly on the main GUI thread, which inevitably results in unresponsive application states (UI Freezing).

To resolve this bottleneck, all hardware communication and network polling logic are strictly confined to independent background worker threads or child processes. Safe

data exchange between these asynchronous background tasks and the main GUI event loop is achieved through robust Inter-Process Communication (IPC) or thread-safe signaling mechanisms. When the background worker successfully parses a complete telemetry frame from the ESP32, it bypasses direct memory manipulation of UI elements. Instead, it emits an asynchronous signal or pipes the formatted data to the standard output stream (stdout). The main GUI event loop intercepts these IPC signals and delegates them to designated handler functions (Slots), ensuring that visual components, timing tables, and AI prediction labels are updated sequentially and safely. This decoupled queuing architecture guarantees maximum GUI responsiveness, regardless of the interrupt frequency or data volume generated by the distributed edge network.

2.1.4. Microarchitectural Optimization

To empirically validate the system-level theories discussed in the preceding sections, a Convolutional Neural Network (LeNet-5) inference pipeline is deployed at the edge node. In high-performance computing (HPC) and embedded AI systems, optimizing algorithmic complexity at the application layer is insufficient. To achieve deterministic execution and ultra-low latency, the system must be meticulously tuned at the microarchitectural level. This section delves into the theoretical interactions between the Linux operating system kernel and the underlying hardware, specifically tailored for optimizing the LeNet-5 workload.

Memory Allocation Dynamics

The memory allocation strategy fundamentally dictates the efficiency of machine learning model loading and tensor propagation. A Linux process's virtual address space is strictly partitioned, with the stack and heap serving divergent roles during deep learning inference. The stack operates on a Last-In, First-Out (LIFO) mechanism and grows downwards from high memory. While the stack offers extremely fast access times due to L1 cache proximity, its capacity is strictly capped by the operating system (typically 8 MB). In the context of LeNet-5, the stack must be exclusively reserved for execution frames, function pointers, and small scalar variables (e.g., loop indices for sliding convolutional kernels).

Attempting to allocate LeNet-5's trainable parameters or its intermediate feature maps on the stack would inevitably trigger a catastrophic stack overflow. Consequently, the bulk of the network's memory footprint demands dynamic allocation as large-sized tensors on the heap, which grows upwards. Standard C/C++ libraries interface with the Linux kernel's memory management primarily through two system calls: brk and mmap. The program break can be dynamically adjusted using brk to allocate small-to-medium memory blocks on the heap for tensor activations. Conversely, for static, pre-trained network weights, the memory mapping segment is utilized via mmap. This system call directly maps the .onnx or .pth model files from secondary storage into the virtual address space, enabling a zero-copy mechanism where the Memory Management Unit utilizes demand paging to fetch weights directly into physical RAM, entirely bypassing redundant user-space buffer allocations.

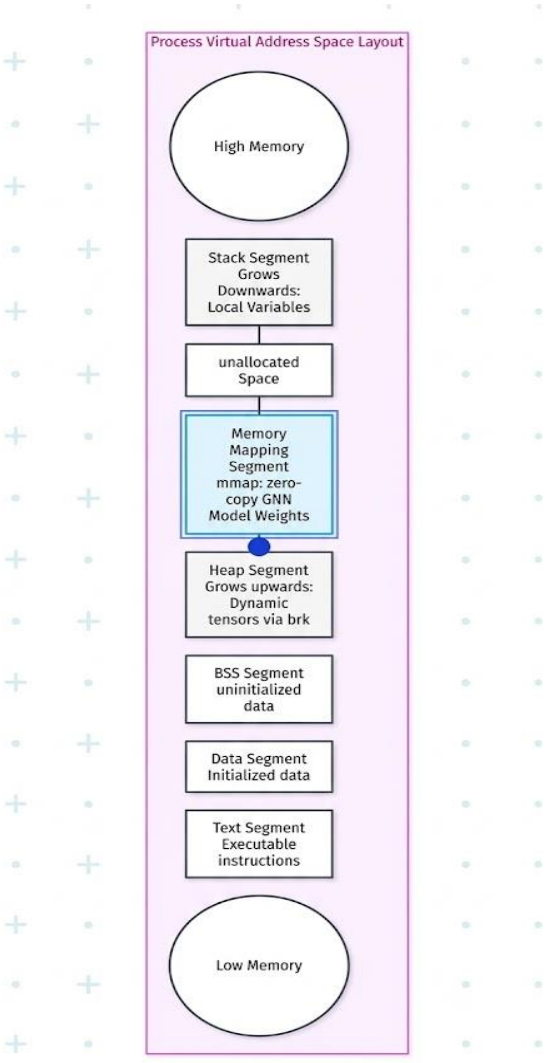


Figure 2.5: Process Virtual Address Space Layout

Processor Affinity and Core Pinning

Beyond memory management, the execution environment must be stabilized against the operating system's default load-balancing behaviors. The Completely Fair Scheduler (CFS) continuously balances system load by migrating active threads across available physical CPU cores. While optimal for general-purpose computing, this policy introduces severe cache thrashing for AI workloads. When the LeNet-5 inference thread executes on a specific core, the filter weights and localized input image pixels are actively cached in that core's private L1 and L2 caches. If the scheduler migrates this thread to a different core, a massive cache miss occurs, forcing the new core to expend thousands of clock cycles fetching the parameters from slower main memory.

To enforce microarchitectural determinism and prevent this severe performance degradation, Processor Affinity (Core Pinning) is explicitly applied using POSIX interfaces such as pthread_setaffinity_np() or sched_setaffinity(). By strictly binding the LeNet-5 ONNX Runtime inference thread to a dedicated execution unit (e.g., Core 0), the L1/L2 cache remains uniformly preserved for the neural network's computations. Simultaneously, background OS tasks and I/O networking processes are isolated and offloaded to the remaining cores, guaranteeing that the model parameters remain in a "hot cache" state and drastically reducing memory access latency.

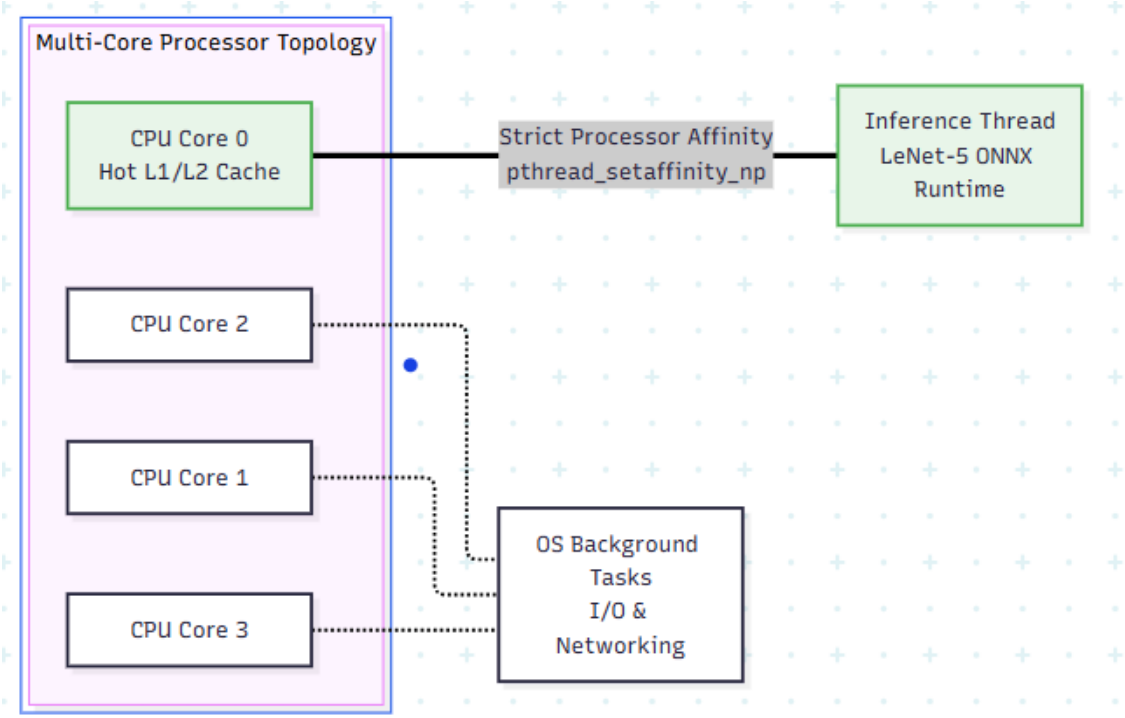


Figure 2.6: Processor Affinity and Cache Optimization

Real-Time Scheduling Disciplines

The operating system scheduling discipline further defines the temporal execution priority of the LeNet-5 inference thread against background system processes. The default CFS allocates CPU time based on a virtual runtime metric to ensure absolute fairness among all running tasks. However, in edge AI scenarios requiring strict responsiveness, this fairness introduces unpredictable latency jitter, as the OS can arbitrarily preempt the LeNet-5 forward pass to service background daemons.

To achieve hard real-time constraints, the inference thread's scheduling policy must be elevated using POSIX real-time standards. The SCHED_FIFO (First-In, First-Out) policy provides a preemptive, non-time-slicing approach where the LeNet-5 thread monopolizes the designated CPU core until completion, explicitly blocking standard user processes. Alternatively, the SCHED_RR (Round-Robin) policy enforces a fixed time quantum among multiple real-time threads of equal priority, ensuring that concurrent AI pipelines share CPU cycles predictably without ever yielding to lower-priority background noise.

Low-Level Runtime Profiling

Rigorous low-level profiling is mandatory to validate the efficacy of these microarchitectural optimizations. The primary metric is context switch quantification. Context switching forces the CPU to save and restore execution registers, contributing zero computational value and disrupting the continuous matrix arithmetic required in LeNet-5’s fully connected layers. This overhead is continuously monitored using system statistics utilities (e.g., pidstat) to ensure minimal thread preemption.

Furthermore, hardware execution efficiency relies heavily on spatial memory locality. During pooling and convolutional layers, the CPU fetches data in fixed-size blocks known as cache lines. Hardware Performance Counters (HPCs) are utilized to quantify this efficiency natively within Linux. To empirically measure the exact cache miss rate during inference, diagnostic commands such as 'perf stat -e cache-references,cache-misses' are executed. Finally, continuous inference pipelines are highly susceptible to memory leaks if dynamic heap allocations for layer activations are not explicitly managed. To prevent Out-Of-Memory (OOM) fatal errors, strict memory auditing is performed using the Valgrind Memcheck tool, which intercepts all allocation

system calls at runtime, guaranteeing the structural integrity of the AI pipeline for indefinite deployment.

2.1.5. Distributed Edge AI Deployment On LINUX

To validate the correctness of the theoretical foundations established in the preceding four sections, a Convolutional Neural Network (LeNet-5) system was empirically deployed. This application serves as a concrete manifestation of the convergence of Hardware-Software Co-Design, Inter-Process Communication (IPC), Multithreading Coordination, and OS-level Memory Optimization.

Architectural Overview of the Distributed Edge AI System

The system is architected based on a distributed topology, rigorously segregating workloads between the POSIX server and the peripheral edge node.

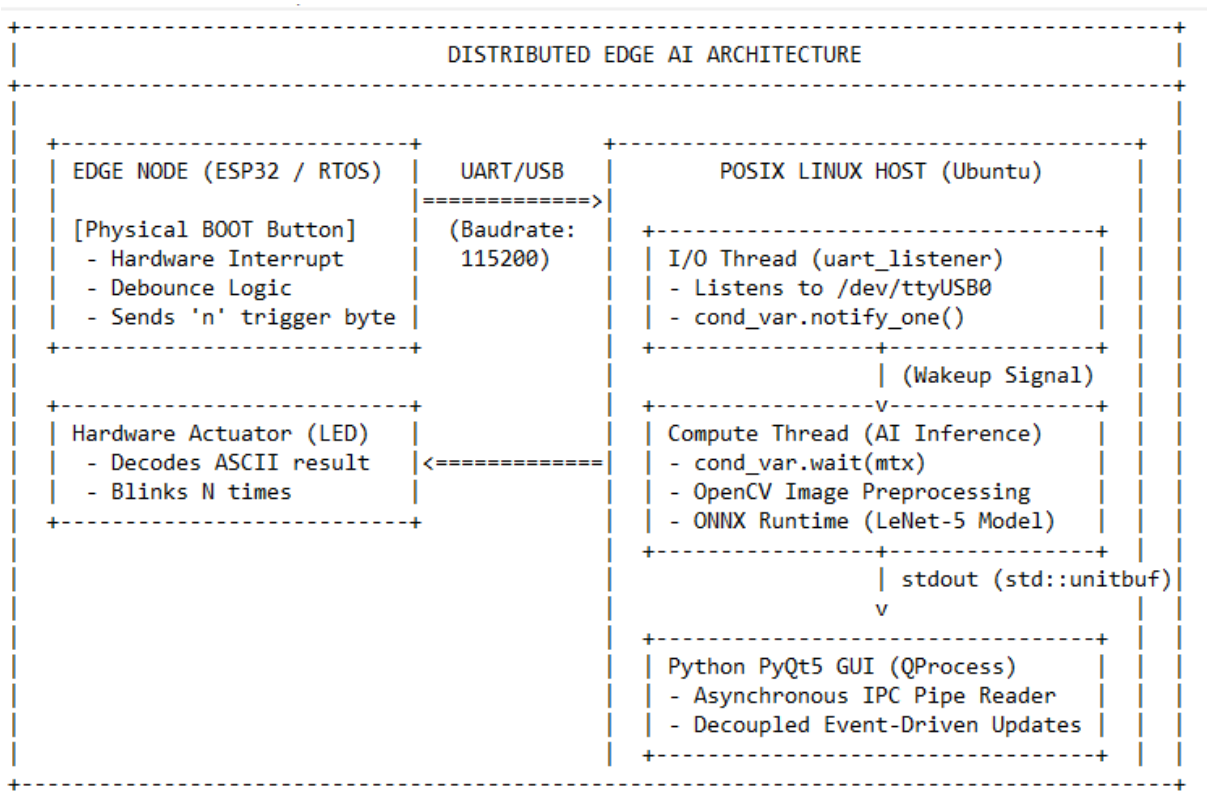


Figure 2.7: Distributed Edge AI System Architecture

This architecture adheres to the principle of Offloading: Tasks necessitating hard real-time responses, such as polling button interrupts and controlling LEDs, are relegated to the ESP32 microcontroller. Conversely, the Linux server assumes responsibility for the computationally intensive matrix operations of the LeNet-5 model, dynamic memory management, and graphics rendering.

Asynchronous GUI Architecture and the Inter-Process Communication (IPC Pipeline)

At the supervisory tier, the Graphical User Interface (GUI) is developed in Python (PyQt5) following a Decoupled Event-Driven paradigm. To circumvent the classic anti-pattern of UI Freezing caused by blocking on hardware I/O, the system establishes an IPC Pipeline (Anonymous Pipe) connecting directly to the core C++ process (AI Core).

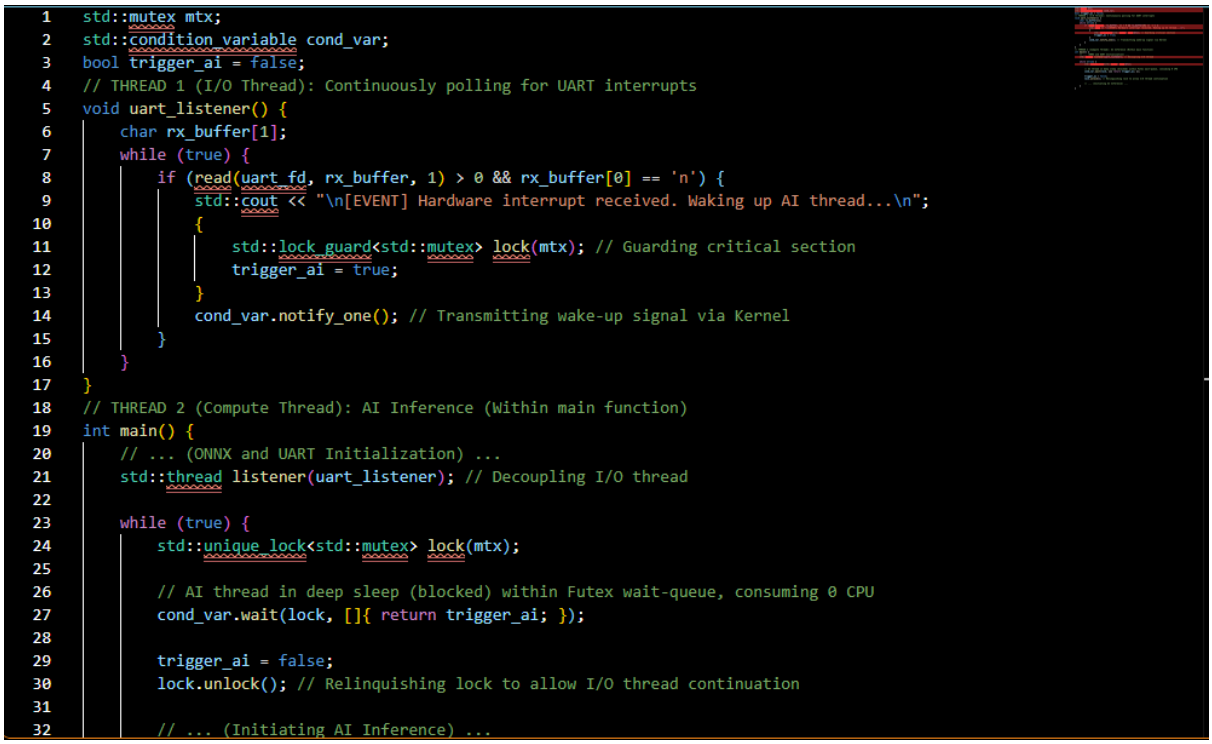
```
1   def start_cpp_core(self):
2           self.console.append("[GUI] Initializing Core...")
3           env = self.process.processEnvironment()
4           env.insert("LD_LIBRARY_PATH", "./onnxruntime/lib:" + env.value("LD_LIBRARY_PATH", ""))
5           self.process.setProcessEnvironment(env)
6
7           # Spawning an independent child process, circumventing GUI freezing
8           self.process.start("./edge_ai_core")
9
10      def handle_stdout(self):
11          # Asynchronously extracting data from the C++ process's stdout stream
12          data = self.process.readAllStandardOutput().data().decode().strip()
13          for line in data.split('\n'):
14              if line:
15                  self.console.append(line)
16                  if "[RESULT]" in line:
17                      num = line.split("so: ")[1].strip()
18                      self.lbl_result.setText(f"{num}") # Updating AI classification label
19                  elif "[TIME]" in line:
20                      time_val = line.replace("[TIME]", "").strip()
21                      self.lbl_time.setText(time_val)   # Updating latency measurement
```

Figure 2.8: Source Code Excerpt : Initializing the core process and capturing IPC.

System Analysis: The provided snippet leverages the QProcess object to spawn a child process executing the ./edge_ai_core binary. This transport layer operates as a non-blocking I/O channel. Upon completion of computation, the C++ process refrains from direct memory access into Python; instead, it flushes data to standard output (stdout). The PyQt5 event loop instantaneously intercepts these text strings, parsing the [RESULT] and [TIME] tags, and subsequently updates the interface in a thread-safe manner.

Hardware Interrupt Handling and Event-Driven Synchronization

Beneath the GUI layer, the C++ process functions as the "heart" of the system, orchestrating hardware communication and neural network computation. To mitigate the risk of CPU cycle wastage (busy-waiting) while polling the UART port, the C++ kernel implements an Event-driven synchronization paradigm utilizing Mutual Exclusion Locks (std::mutex) and Condition Variables (std::condition_variable).



```cpp
std::mutex mtx;
std::condition_variable cond_var;
bool trigger_ai = false;
// THREAD 1 (I/O Thread): Continuously polling for UART interrupts
void uart_listener() {
    char rx_buffer[1];
    while (true) {
        if (read(uart_fd, rx_buffer, 1) > 0 && rx_buffer[0] == 'n') {
            std::cout << "\n[EVENT] Hardware interrupt received. Waking up AI thread...\n";
            {
                std::lock_guard<std::mutex> lock(mtx); // Guarding critical section
                trigger_ai = true;
            }
            cond_var.notify_one(); // Transmitting wake-up signal via Kernel
        }
    }
}
// THREAD 2 (Compute Thread): AI Inference (Within main function)
int main() {
    // ... (ONNX and UART Initialization) ...
    std::thread listener(uart_listener); // Decoupling I/O thread

    while (true) {
        std::unique_lock<std::mutex> lock(mtx);

        // AI thread in deep sleep (blocked) within Futex wait-queue, consuming 0 CPU
        cond_var.wait(lock, []{ return trigger_ai; });

        trigger_ai = false;
        lock.unlock(); // Relinquishing lock to allow I/O thread continuation

        // ... (Initiating AI Inference) ...
```

Figure 2.9: Source Code Excerpt : Wake-up mechanism via Condition Variable.

System Analysis: This serves as an empirical validation of the thread optimization theory delineated previously. Thread 2 (Compute Thread) relinquishes CPU control via the cond_var.wait() directive. At this juncture, the Linux Kernel suspends this thread and places it in the Wait-Queue, completely liberating CPU resources for background OS tasks. Only when the ESP32 microcontroller transmits the 'n' character via the /dev/ttyUSB0 port does Thread 1 (I/O Thread) capture the signal, acquire the Mutex lock, assert the trigger_ai flag, and invoke notify_one(). The operating system instantaneously executes a Context Switch, awakening the AI Thread to commence the inference cycle, yielding a highly deterministic execution system and eradicating computational resource wastage entirely.

2.1.5.4. Memory Allocation Dynamics and LeNet-5 Model Inference

The Convolutional Neural Network (LeNet-5) model comprises tens of thousands of parameters that cannot be allocated on the User Space Stack due to the substantial risk of Stack Overflow. Consequently, the system utilizes Dynamic Allocation mechanisms on the Heap memory space for input Tensors.

```cpp
auto start_time = std::chrono::high_resolution_clock::now(); // Initiating chronometer

    // 1. Reading and preprocessing static image via OpenCV in RAM
    cv::Mat img = cv::imread("images/sample.png", cv::IMREAD_GRAYSCALE);
    cv::resize(img, img, cv::Size(28, 28));
    img.convertTo(img, CV_32F, 1.0 / 255.0);

    // 2. Contiguous Memory Allocation of Tensor block on Heap
    std::vector<float> input_tensor_values(img.ptr<float>(), img.ptr<float>() + 28 * 28);
    std::vector<int64_t> input_dims = {1, 1, 28, 28};

    auto memory_info = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault);
    auto input_tensor = Ort::Value::CreateTensor<float>(
        memory_info, input_tensor_values.data(), input_tensor_values.size(),
        input_dims.data(), input_dims.size());

    // 3. Executing Inference (Forward-pass Inference) through LeNet-5 network
    const char* input_names[] = {"Input3"};
    const char* output_names[] = {"Plus214_Output_0"};
    auto output_tensors = session.Run(Ort::RunOptions{nullptr},
                                      input_names, &input_tensor, 1, output_names, 1);

    // 4. Determining highest probability for final prediction
    float* floatarr = output_tensors.front().GetTensorMutableData<float>();
    int prediction = std::distance(floatarr, std::max_element(floatarr, floatarr + 10));

    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> inference_time = end_time - start_time;
```

Figure 2.10: Source Code Excerpt : Memory space preprocessing and Inference.

System Analysis: The Ort::Value::CreateTensor invocation instantiates a memory container on the Heap via foundational system calls (brk / mmap). The utilization of OpenCV's cv::Mat in conjunction with std::vector ensures the 28x28 image data is flattened into a Contiguous Memory Layout. This arrangement optimizes Spatial Locality within the CPU's L1 Cache, enabling the ONNX Runtime network to execute consecutive matrix multiplications with minimal latency. The std::chrono function is wrapped around this cycle to accurately profile execution velocity at the microsecond echelon. Ultimately, the inference result (ASCII) is pushed through the IPC Pipe for visualization and simultaneously transmitted down the UART (write(uart_fd, &result, 1)) to provide feedback to the microcontroller hardware.

Empirical System Verification

The culmination of the entire Hardware-Software Integration process is visually represented via the Dashboard interface during real-world system operation.

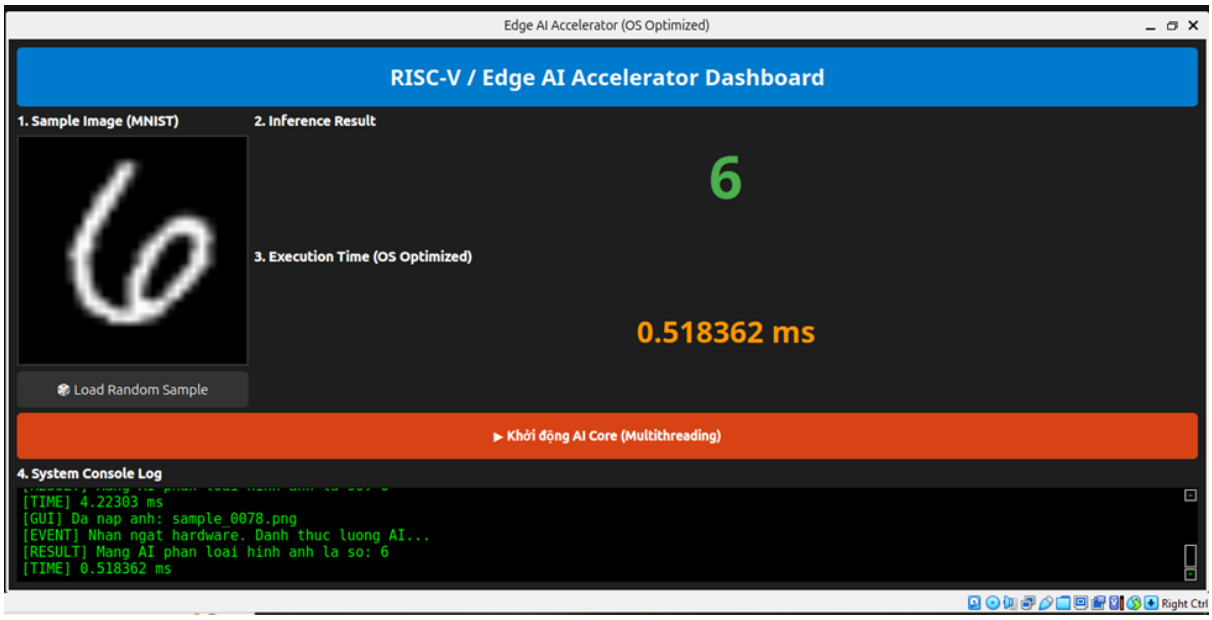


Figure 2.11: Edge AI Accelerator Interface

Artifact Analysis: Predicated on the aforementioned empirical interface, the system demonstrates success across the three core objectives established:

- **Thread Synchronization Validation**: Within the System Console Log window (Pane 4), the manifestation of the green message line [EVENT] Hardware interrupt received. Waking up AI thread... serves as absolute physical proof that the AI computation thread was genuinely awakened from a deep sleep (blocked) state within the Wait-Queue by an interrupt signal originating from the microcontroller transmitted via the UART standard. This substantiates that the Condition Variable mechanism operates flawlessly.
- **Inter-Process Communication (IPC) Reliability**: Notwithstanding the C++ thread being persistently awakened in the background and executing high-intensity computations, the Python GUI remains un-frozen. The classification labels (Inference Result) accurately display the integer 8, and the latency label perfectly synchronizes with data pushed through the Stdout Pipe, demonstrating the absolute stability of the Decoupled GUI architecture.
- **Microarchitectural Performance Optimization**: The Execution Time (OS Optimized) metric records that the process of reading the sample_0853.png image (containing the digit 8), loading it into RAM, feeding the tensor into the LeNet-5 network, and unpacking the result vector consumed a mere 4.22303 ms (milliseconds). This velocity represents an exceptionally impressive Benchmark

on a Linux kernel operating system, verifying that optimizations regarding dynamic Heap memory allocation, Context Switch minimization, and L1/L2 Cache exploitation have achieved maximal efficacy.

2.2. Theoretical background of RISC-V architecture

2.2.1. Introduction to RISC-V

RISC-V is an open-standard Instruction Set Architecture (ISA) based on the principles of Reduced Instruction Set Computing (RISC). The architecture originated at the University of California, Berkeley, where its development began in 2010 under the leadership of researchers including Krste Asanović, David Patterson, Andrew Waterman, and Yunsup Lee. RISC-V was initially developed to provide a practical architecture for computer architecture research and education, particularly because existing commercial ISAs were often proprietary and difficult to modify or extend for experimental hardware designs. Since its introduction, RISC-V has evolved beyond its academic origins into an architecture used for both research and industrial processor development. The official specifications are currently maintained and ratified by RISC-V International, which provides the ISA specifications publicly for hardware designers, software developers, researchers, and educational institutions [1], [2].

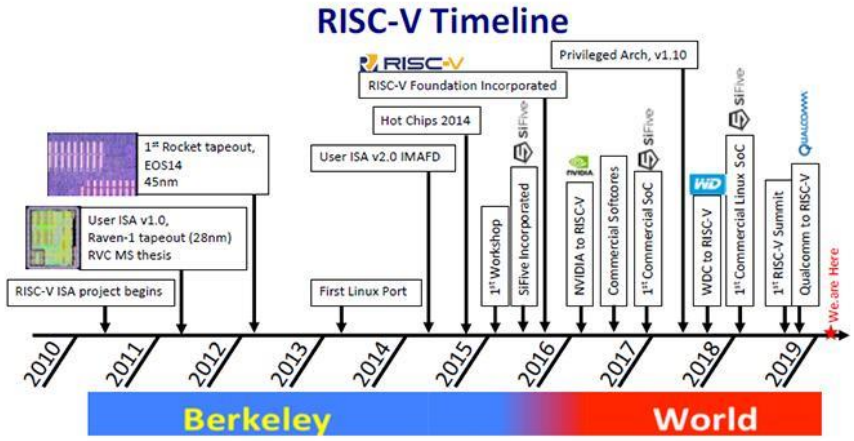


Figure 2.12. RISC-V Development from Berkeley to an Open Standard

An Instruction Set Architecture defines the interface between software and processor hardware. It specifies the operations that a processor can execute, the programmer-visible registers, instruction encodings, memory-access behavior, and other architectural characteristics required for software to execute correctly. RISC-V

follows this principle by defining a standardized instruction set while leaving processor designers considerable freedom in how the underlying microarchitecture is implemented. Consequently, processors implementing the same RISC-V ISA may use different datapath organizations, control units, cache structures, pipeline depths, or execution strategies while remaining compatible with software compiled for the supported RISC-V ISA. This separation between ISA and microarchitecture is particularly important in hardware development because it allows architectural compatibility to be maintained while different processor implementations are explored and optimized [3].

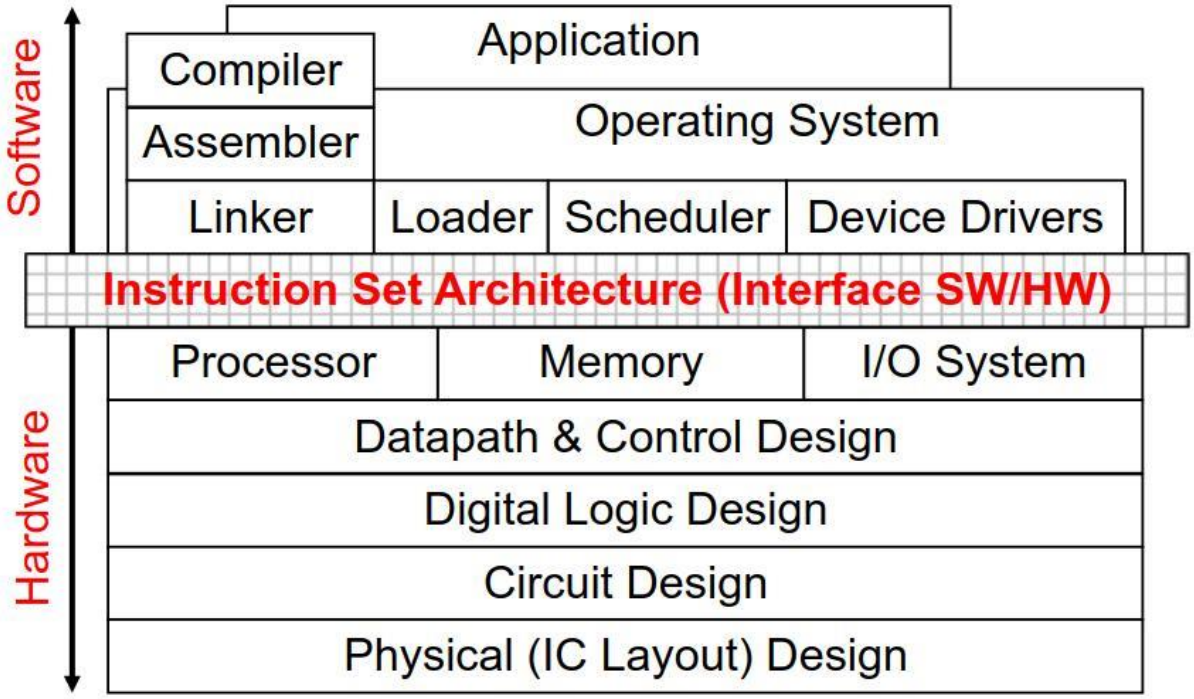


Figure 2.13. The ISA as the Interface Between Software and Hardware

RISC-V is not a single fixed instruction set but rather a family of related ISAs constructed from a base integer instruction set together with optional extensions. Two important base variants are RV32I and RV64I, which provide 32-bit and 64-bit integer register and address-space architectures, respectively. The base integer ISA contains a deliberately compact set of instructions sufficient to support compilers, assemblers, linkers, operating-system components, and general program execution when combined with the necessary privileged facilities. Additional capabilities can then be introduced through standardized extensions. This modular organization enables a processor to

implement only the architectural features required by its intended application instead of requiring every possible instruction and hardware feature [3], [4].

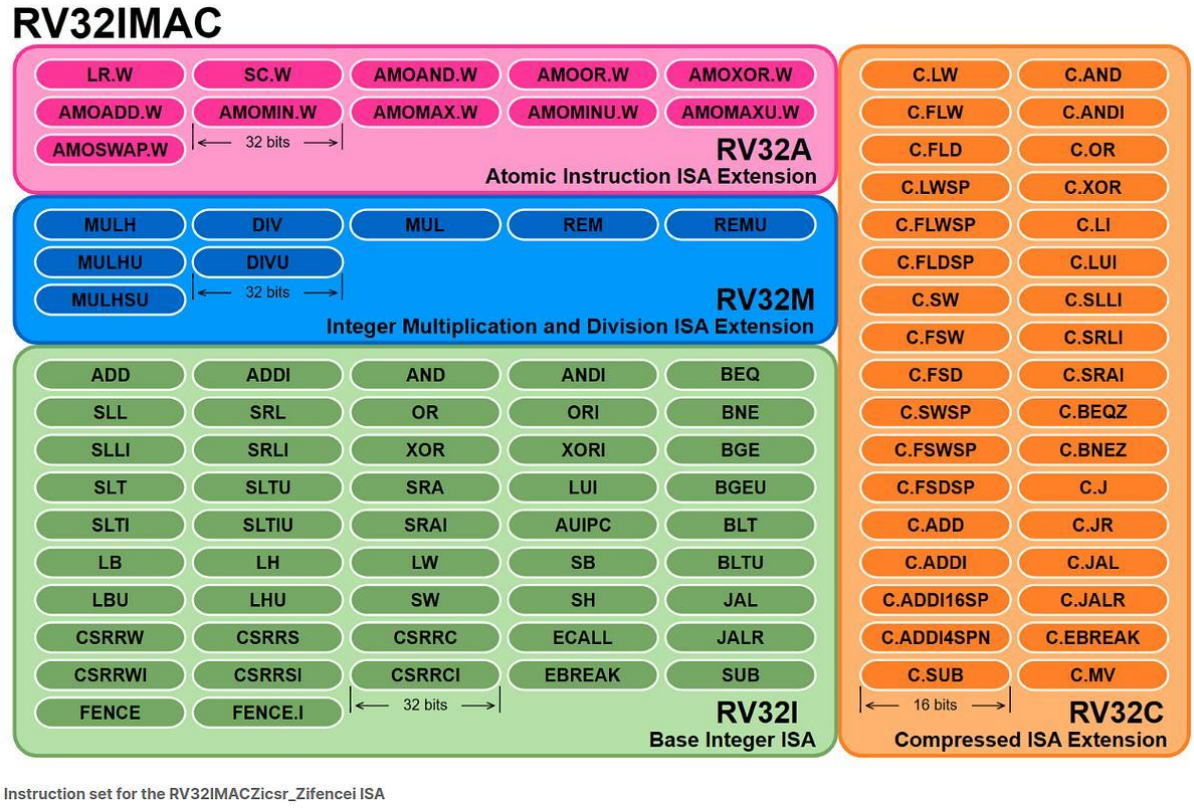


Figure 2.14. RISC-V Base ISA and Standard Extensions

For example, a processor based on RV32I provides the fundamental 32-bit integer instructions, while additional extensions can introduce capabilities required for more advanced applications. Standard extensions include integer multiplication and division, floating-point computation, compressed instruction encoding, vector processing, and other specialized operations. This modular structure is one of the defining characteristics of RISC-V because it enables the architecture to scale from relatively simple embedded processors to more capable systems. Furthermore, RISC-V permits the development of custom extensions, allowing designers to introduce application-specific instructions while retaining the standard base architecture. Such extensibility is especially valuable in domains involving specialized computation, including signal processing and hardware acceleration, where customized instructions or accelerators can improve the efficiency of frequently executed operations [3], [5].

Another fundamental characteristic of RISC-V is its status as an open-standard ISA. Unlike proprietary processor ISAs whose use and implementation may be subject

to restrictions imposed by their owners, the RISC-V specifications are publicly available and collaboratively developed and maintained through RISC-V International. The organization describes RISC-V as an open-standard ISA and makes its ratified specifications freely accessible. Importantly, the openness of the ISA should be distinguished from the licensing of individual processor implementations. RISC-V standardizes the architectural interface, but a specific RISC-V processor core may be distributed under either an open-source or proprietary license depending on its developer. Therefore, the principal benefit of the open standard is that organizations, researchers, and hardware designers can develop independent processor implementations that conform to the same standardized ISA [2], [6].

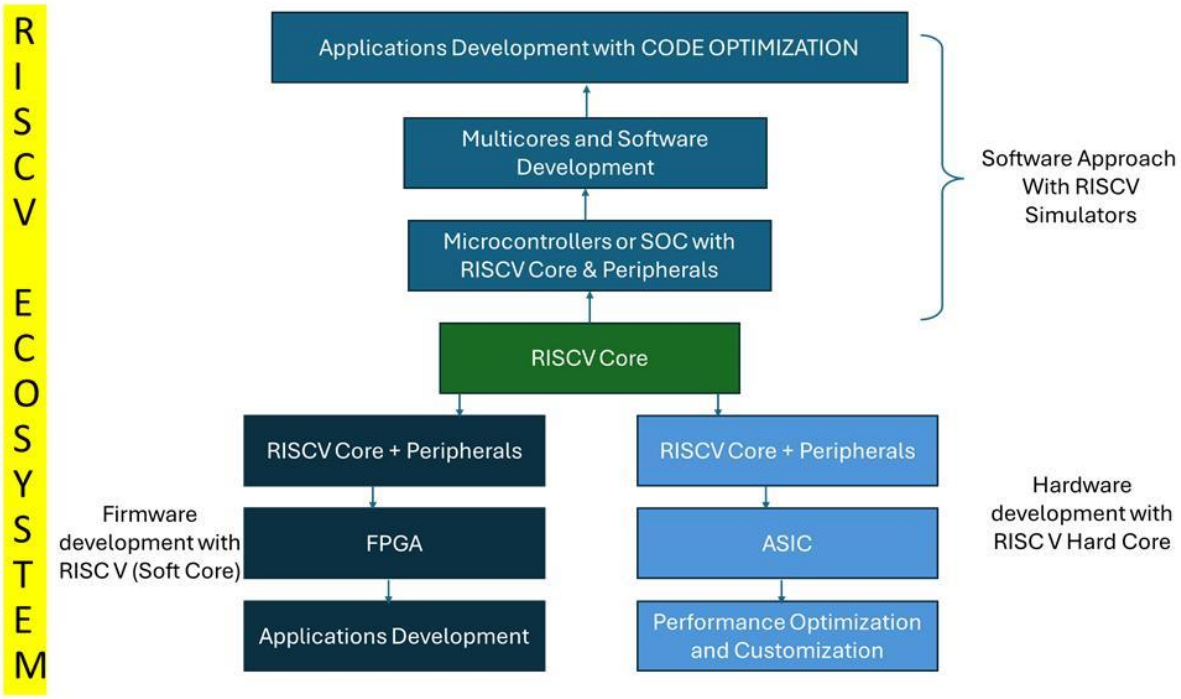


Figure 2.15. RISC-V Ecosystem: Software and Hardware Development Paths

This open and implementation-independent approach provides substantial advantages for research and education. According to the historical rationale published with the RISC-V specification, the architecture was specifically developed to support research involving real hardware implementations and to provide processor designs that students could explore in computer architecture courses. Proprietary architectures can limit this type of experimentation because researchers may be unable to freely modify, redistribute, or implement the ISA in register-transfer-level hardware. In contrast, RISC-V allows researchers and students to investigate processor design at multiple abstraction

levels, ranging from instruction execution and datapath organization to pipelining, memory systems, and specialized accelerators. RISC-V implementations have consequently been used in undergraduate and graduate courses as well as FPGA and ASIC research projects [1].

The architecture is also particularly suitable for hardware development because the ISA does not prescribe a specific microarchitecture or implementation technology. A RISC-V processor can therefore be implemented as a simple single-cycle CPU for educational purposes, extended into a pipelined architecture for higher instruction throughput, or developed into a more sophisticated processor containing caches, branch prediction, parallel execution resources, or specialized accelerators. Similarly, the architecture is not restricted to a particular physical implementation platform and can be implemented using FPGA, ASIC, or other technologies. This flexibility allows designers to preserve ISA compatibility while investigating different trade-offs among performance, hardware complexity, power consumption, area, and design cost [3].

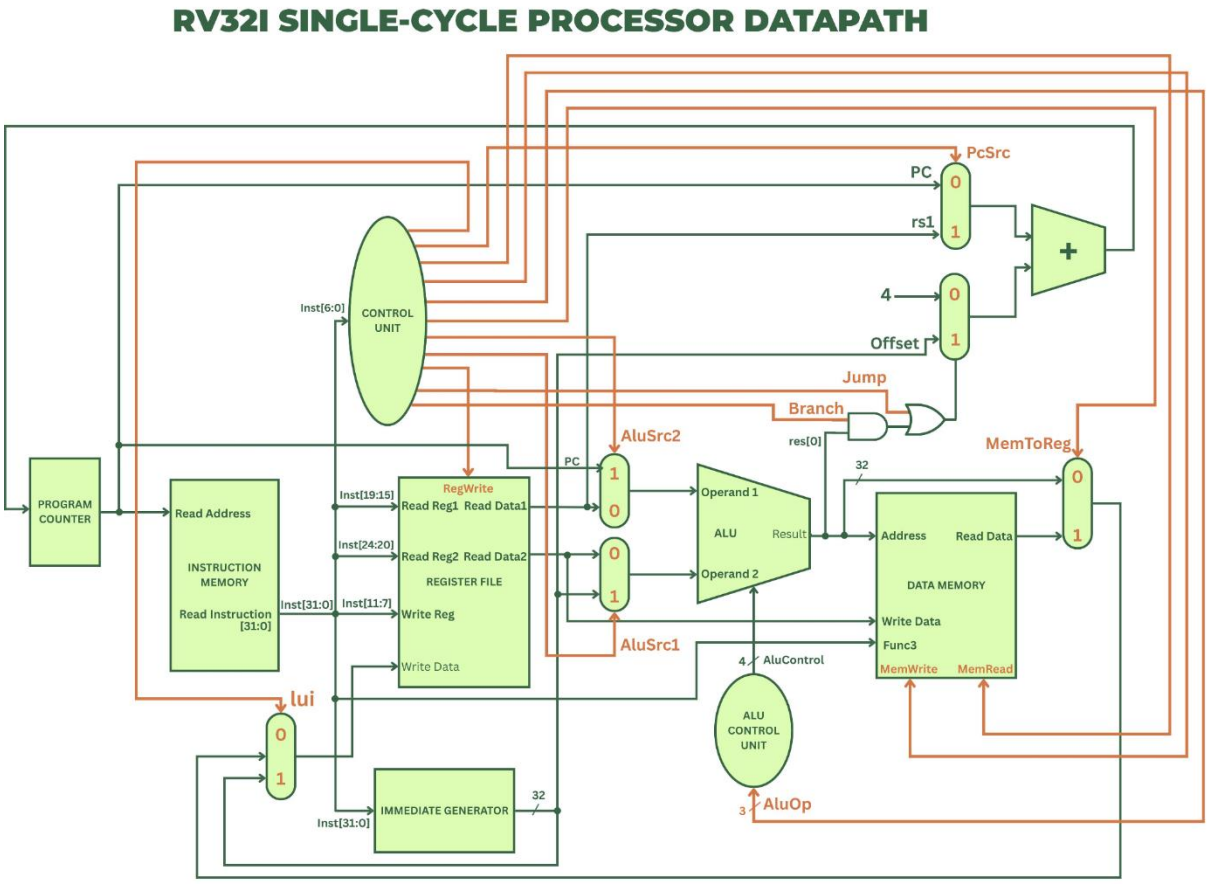


Figure 2.16. RV32I Single-Cycle Processor Datapath

The extensibility of RISC-V is especially relevant to systems that combine a general-purpose processor with dedicated hardware accelerators. The official ISA design rationale identifies specialized and heterogeneous accelerators as an important motivation for providing a flexible and extensible base architecture. Instead of replacing the entire processor architecture for a specialized application, designers can retain a standard RISC-V processor for control and general-purpose computation while integrating custom hardware for computationally intensive tasks. For example, in an artificial-intelligence system, the RISC-V processor can manage program execution, memory access, data transfer, and communication with external software, while a dedicated accelerator performs computationally expensive operations such as matrix multiplication or convolution. This organization provides a practical foundation for hardware–software co-design because the programmable processor and application-specific hardware can be developed as complementary components within the same system [1].

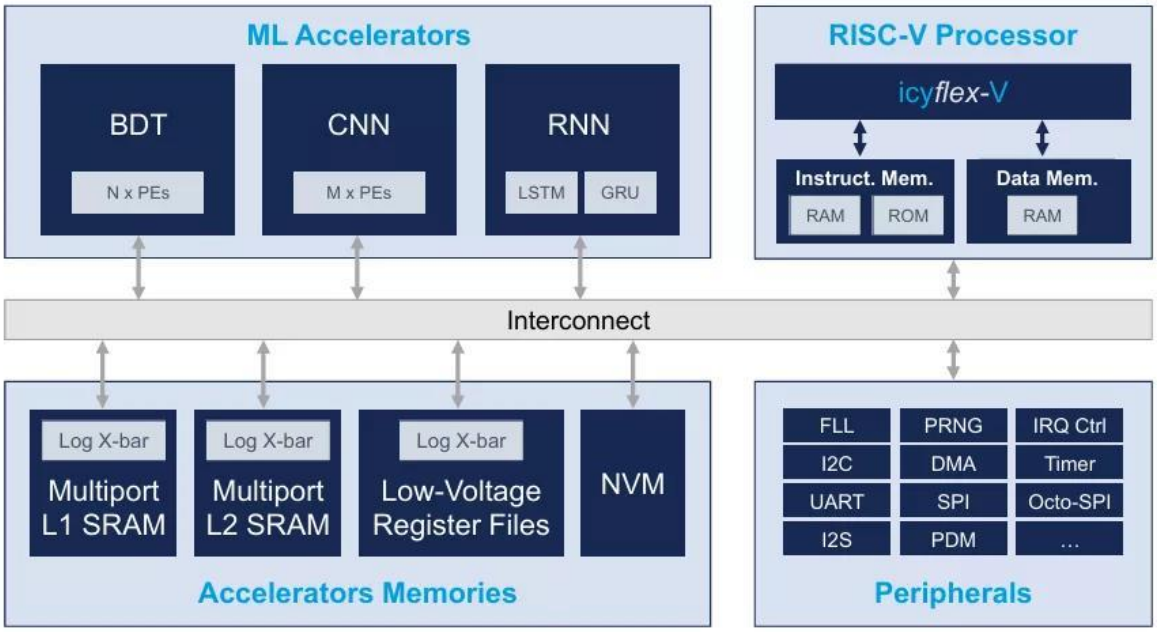


Figure 2.17. RISC-V SoC with ML Accelerators, Memory, and Peripherals

Overall, RISC-V combines a standardized ISA with an open, modular, and extensible architectural philosophy. Its small base integer instruction sets provide a practical foundation for processor implementation, while optional standard and custom extensions enable the architecture to be adapted to different computational requirements. The availability of public specifications, independence from a prescribed

microarchitecture, and support for hardware customization make RISC-V particularly appropriate for academic research, FPGA-based processor development, and experimental computer architectures. These characteristics also make it a suitable processor architecture for systems that integrate dedicated hardware accelerators, where RISC-V can provide general-purpose control while specialized modules perform computation-intensive workloads.

2.2.2. RV32I Base Instruction Set

The RV32I base integer instruction set is the fundamental 32-bit integer ISA of the RISC-V architecture. It provides the essential instructions required for integer computation, memory access, and control-flow operations. In RV32I, the integer register width is 32 bits (XLEN = 32), and the architecture provides 32 integer registers, denoted as x0 to x31. Register x0 is permanently hardwired to zero, whereas registers x1–x31 are general-purpose registers. In addition, the program counter (PC) stores the address of the current instruction.

RV32I instructions have a fixed length of 32 bits. The ISA defines four core instruction formats—R, I, S, and U—together with the B and J immediate variants. These formats maintain consistent positions for important register fields such as rs1, rs2, and rd, which simplifies instruction decoding in hardware.

Core Instruction Formats

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

Figure 2.18. RV32I instruction formats

The six instruction formats serve different purposes. R-type instructions mainly perform register-to-register arithmetic and logical operations; I-type instructions support immediate arithmetic, loads, and indirect jumps; S-type instructions store data into memory; B-type instructions implement conditional branches; U-type instructions manipulate upper immediate values; and J-type instructions provide PC-relative unconditional jumps.

R-type Instructions

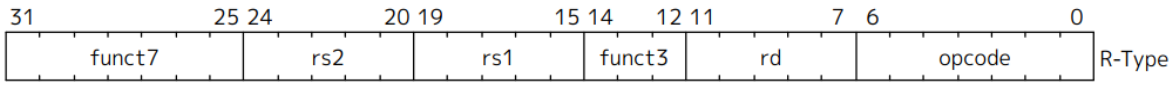


Figure 2.19. R-type bit distribution.

R-type instructions are primarily used for register-to-register arithmetic, logical, comparison, and shift operations. Two source operands are read from registers rs1 and rs2, and the computed result is written to the destination register rd. The funct3 and funct7 fields, together with the opcode, determine the specific operation to be performed.

For example, the instruction:

add x3, x1, x2

performs

x3 = x1 + x2

where x1 and x2 are the source registers and x3 is the destination register. Similarly, SUB performs subtraction, while AND, OR, and XOR perform bitwise logical operations. RV32I also provides comparison instructions such as SLT and SLTU, as well as shift instructions including SLL, SRL, and SRA.

Therefore, the major R-type instructions can be summarized as:

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends

Figure 2.20. R-type instructions.

I-type Instructions

I-type instructions use one source register and an immediate operand. The 12-bit immediate field occupies bits [31:20] and is generally sign-extended before being used by the datapath.

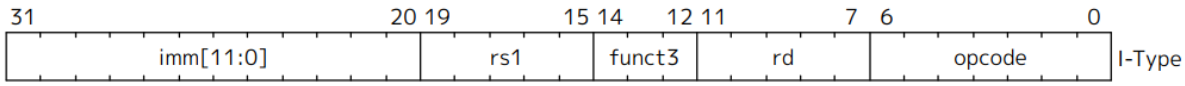


Figure 2.21. I-type bit distribution.

One common example is:

addi x3, x1, 10

which performs

x3 = x1 + 10

Unlike an R-type instruction, the second ALU operand is obtained from the immediate field rather than rs2. Consequently, in a processor datapath, an ALUSrc multiplexer is typically required to select between the second register operand and the immediate value.

Integer register-immediate instructions include ADDI, SLTI, SLTIU, XORI, ORI, and ANDI. Shift-immediate operations are represented by SLLI, SRLI, and SRAI.

I-type encoding is also used by other instruction categories, particularly load instructions (LB, LH, LW, LBU, and LHU) and the indirect jump instruction JALR. For load operations, the effective memory address is calculated from the base register and a sign-extended offset:

Address = x[rs1] + SignExt(imm)

For example,

lw x5, 4(x1)

calculates the memory address as x1 + 4 and loads a 32-bit word from that address into x5.

Therefore, the major I-type instructions can be summarized as:

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 | imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	

Figure 2.22. I-type instructions.

S-type Instructions

S-type instructions are used for store operations, which transfer data from a register to memory. Unlike R- and I-type instructions, S-type instructions do not contain an rd field because no value is written back to the register file. Instead, the immediate is divided into two fields, imm[11:5] and imm[4:0].

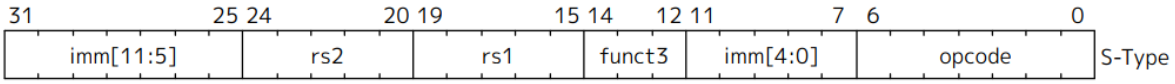


Figure 2.23. S-type bit distribution.

The effective address is calculated as

$$\text{Address} = x[rs1] + \text{SignExt}(imm)$$

while rs2 contains the data to be written into memory.

For example,

sw x5, 4(x1)

performs conceptually:

Memory[x1+4]=x5

RV32I defines SB, SH, and SW for storing 8-bit bytes, 16-bit halfwords, and 32-bit words, respectively.

Therefore, the major S-type instructions can be summarized as:

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	

Figure 2.24. S-type instructions.

B-type Instructions

B-type instructions implement conditional control-flow operations. They compare two registers and modify the program counter when the specified condition is satisfied. B-type encoding is derived from the S-type format, but the immediate bits are rearranged to represent a PC-relative branch offset.

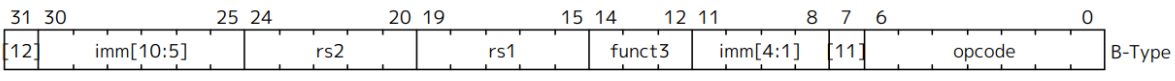


Figure 2.25. B-type bit distribution.

When a branch is taken, the target address is determined by adding the sign-extended branch offset to the address of the branch instruction:

$$PC_{target} = PC + SignExt(B\text{-}immediate)$$

The encoded branch offset represents multiples of two bytes, and its least significant bit is therefore implicitly zero. This organization reduces hardware complexity by avoiding a conventional dynamic left-shift operation for the branch immediate.

RV32I provides six fundamental conditional branch instructions:

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 >= rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm	zero-extends

Figure 2.26. B-type instructions.

For example,

beq x1, x2, target

transfers execution to target when x1 and x2 contain equal values. Otherwise, execution continues with the next sequential instruction.

U-type Instructions

U-type instructions provide a 20-bit upper immediate and are primarily intended for constructing large constants and PC-relative addresses. Their encoding is relatively simple because only an immediate field and destination register are required.



Figure 2.27. U-type bit distribution.

RV32I defines two U-type instructions: LUI and AUIPC.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	

Figure 2.28. U-type instructions.

LUI (Load Upper Immediate) places the U-immediate in the upper 20 bits of the destination register while setting the lower 12 bits to zero:

$$x[rd] = imm[31:12] \ll 12$$

For example,

lui x5, 0x12345

produces

$$x5 = 0x12345000$$

AUIPC (Add Upper Immediate to PC) first shifts the immediate by 12 bits and then adds it to the current program counter:

$$x[rd] = PC + (imm[31:12] \ll 12)$$

This operation is particularly useful for constructing PC-relative addresses and supporting position-independent code.

J-type Instructions

J-type encoding is used by the JAL (Jump and Link) instruction to perform an unconditional PC-relative jump while simultaneously storing a return address. Its immediate field is distributed across several portions of the 32-bit instruction.

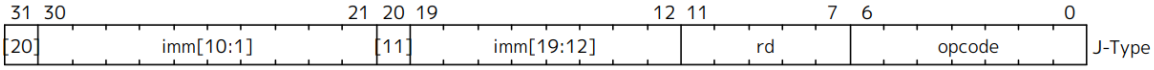


Figure 2.29. J-type bit distribution.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	

Figure 2.30. J-type instructions.

The JAL instruction performs two operations:

$$x[rd]=PC+4$$

and

$$PC=PC+SignExt(J\text{-}immediate)$$

For example,

```
jal x1, function
```

jumps to the address represented by function while storing the address of the following instruction (PC + 4) in x1. Therefore, JAL can be used for function calls, where x1 conventionally serves as the return-address register.

A plain unconditional jump can also be implemented using JAL with x0 as the destination:

```
jal x0, target
```

Since writes to x0 are discarded, only the jump operation remains. The related JALR instruction performs an indirect jump but uses the I-type rather than the J-type

encoding. For JALR, the target is calculated from rs1 and a sign-extended 12-bit immediate, with the least-significant target-address bit cleared.

2.2.3. RISC-V Processor Architecture

A single-cycle RISC-V processor executes each instruction within one clock period. Instruction fetch, decoding, operand selection, execution, and any required memory access occur between two active clock edges. At the ending edge, the processor updates the program counter and the state selected by the instruction. These are functional steps within one cycle, rather than separate pipeline stages [7].

The architecture considered here is a basic RV32I datapath containing a Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, Arithmetic Logic Unit, and Data Memory. Multiplexers, a PC incrementer, and branch-comparison logic connect these blocks. The model assumes combinational instruction and data reads and clocked state updates; the memory organization must satisfy these timing assumptions for execution to remain single-cycle [7], [8].

Program Counter

The Program Counter (PC) is a 32-bit register that stores the address of the current instruction. Its output addresses Instruction Memory and supplies the base for PC-relative calculations. At reset, the register receives an implementation-defined reset vector. During normal operation, it captures NextPC at the active clock edge. Figure 2.31 shows the register interface [9], [10].

A separate adder produces PC + 4 for sequential execution of base RV32I instructions. A next-PC multiplexer selects this value or a branch/jump target. Conditional branches use a comparison of the source registers; a taken branch selects PC plus its immediate offset. JAL also uses a PC-relative target, whereas JALR selects the register-plus-immediate result with bit zero cleared. Target alignment must still meet the base ISA requirements [9], [10].

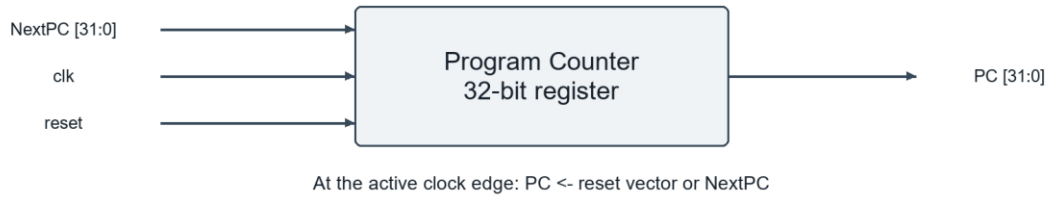


Figure 2.31. Program Counter block. Illustrated based on [9], [10].

Instruction Memory

Instruction Memory stores the encoded program and returns the instruction addressed by the PC. In this single-cycle organization, its read path is combinational: after the address changes, the 32-bit instruction becomes valid following the memory access delay. The instruction fields then feed the Control Unit, Register File, and Immediate Generator [7], [11].

The PC is a byte address. In a word-organized implementation, the aligned address is translated into a word index relative to the memory base. Separate instruction and data memory ports allow instruction fetching and a load or store to occur during the same cycle. This separation is a datapath design choice and does not require RISC-V software to use distinct architectural address spaces [7], [11].

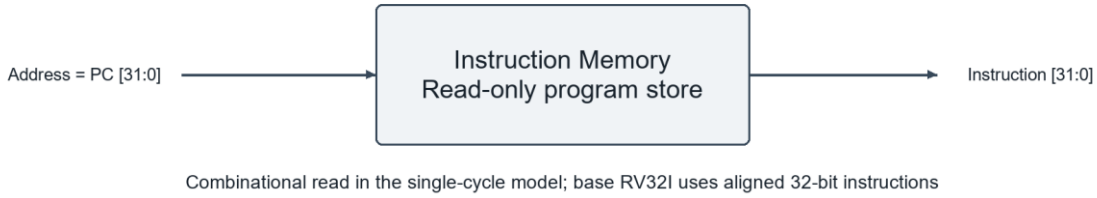


Figure 2.32. Instruction Memory block. Illustrated based on [7], [11].

Register File

The Register File holds the integer operands and instruction results. RV32I defines 32 registers, x0 through x31, each 32 bits wide. Register x0 always returns zero, and writes to it are discarded. A conventional single-cycle register file has two combinational read ports and one clocked write port, as shown in Figure 2.33 [9], [11].

The five-bit rs1 and rs2 indices select the two read values. The destination index rd, WriteData, and RegWrite control an update at the active clock edge. A write-back multiplexer chooses the ALU result, loaded data, or PC + 4. For example, add x3, x1,

x2 reads x1 and x2 during the cycle and writes their sum to x3 at its end. Stores and conditional branches leave the register file unchanged [11], [12].



Figure 2.33. Register File block. Illustrated based on [9], [11].

Immediate Generator

The Immediate Generator produces the 32-bit constant required by immediate arithmetic, memory addressing, and control-flow instructions. It receives the instruction and an ImmSel signal from the Control Unit. Combinational wiring selects the relevant fields, rearranges them according to the format, and forms ImmExt without storing additional state [13].

I-type immediates use instruction bits [31:20], while S-type immediates combine [31:25] and [11:7]. Both are sign-extended from 12 bits. B-type and J-type offsets rearrange the encoded fields and append a zero least-significant bit before sign extension. U-type formation places bits [31:12] above twelve zeros. For example, an I-type immediate of 0xFFC becomes 0xFFFFFFFC, representing -4 in 32-bit arithmetic [9], [13].

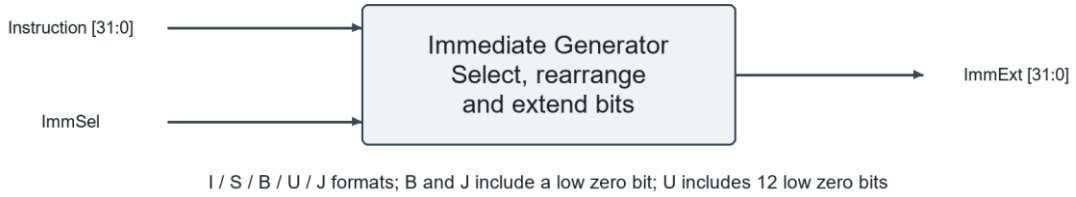


Figure 2.34. Immediate Generator block. Illustrated based on [9], [13].

Control Unit

The Control Unit determines which datapath operations an instruction performs. A combinational decoder interprets opcode, funct3, and relevant funct7 bits, while branch-condition inputs participate in next-PC selection. Control outputs must settle

before the state elements capture their inputs at the ending clock edge. A single-cycle controller does not require a multi-cycle sequencing state machine for the ordinary datapath operations described here [12].

RegWrite enables a destination-register update, and MemWrite enables a store. ImmSel selects the immediate format; ASel and BSel select ALU operands; ALUControl chooses the operation. ResultSrc selects the write-back value, and PCSel selects sequential or redirected execution. For lw, the controller selects register-plus-immediate addition, disables memory writing, and writes memory data to rd. For sw, it enables the memory write and disables register write-back. The signal names in Figure 2.35 are illustrative [12].

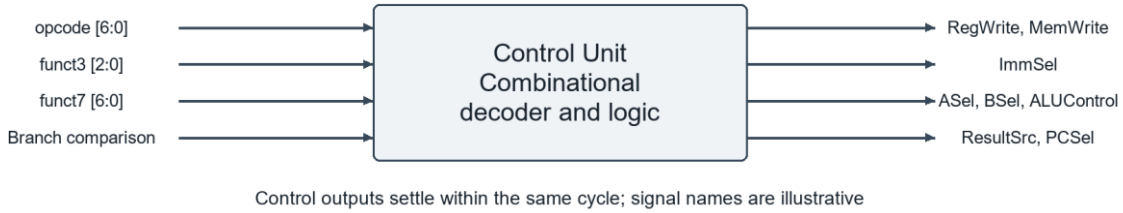


Figure 2.35. Control Unit block. Illustrated based on [12].

Arithmetic Logic Unit

The Arithmetic Logic Unit (ALU) is a combinational circuit that computes a result from two 32-bit operands. ALUControl selects arithmetic, logical, shift, or comparison operations. Operand multiplexers choose register data, the PC, or an immediate, allowing the same ALU to perform register arithmetic, effective-address generation, and PC-relative calculations [14].

For lw and sw, the ALU adds the base register to the sign-extended offset. For AUIPC, it adds the PC to the U-type immediate; a pass-B operation can provide the immediate value for LUI. Signed and unsigned comparisons must be distinguished, and shifts use the lower five bits of the shift amount. In this organization, a separate branch comparator evaluates the source registers while the ALU calculates the branch target, so both results are available within one cycle [10], [14].

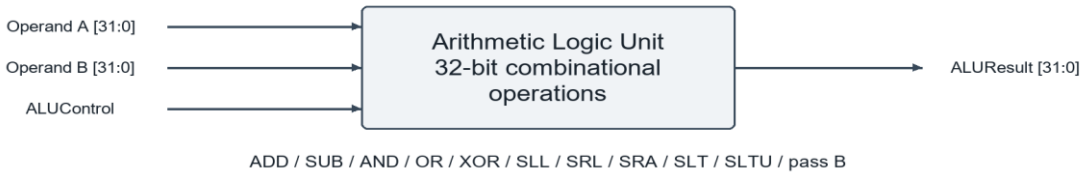


Figure 2.36. Arithmetic Logic Unit block. Illustrated based on [14].

Data Memory

Data Memory stores program variables and supplies or receives operands for load and store instructions. Its address comes from the ALU, and store data comes from the second Register File read port. Figure 2.37 shows a simplified word-oriented interface. For LW, the addressed 32-bit word returns through the write-back multiplexer; for SW, the memory stores the 32-bit register value at the active clock edge [15].

The simplified figure covers LW and SW word transfers. Supporting LB, LH, LBU, LHU, SB, or SH also requires transfer-size selection, byte enables, and load extension logic; those signals are omitted from this block diagram. The model assumes combinational reads and clocked writes. A synchronous-read memory changes the timing assumption and requires a different execution organization [15], [8].

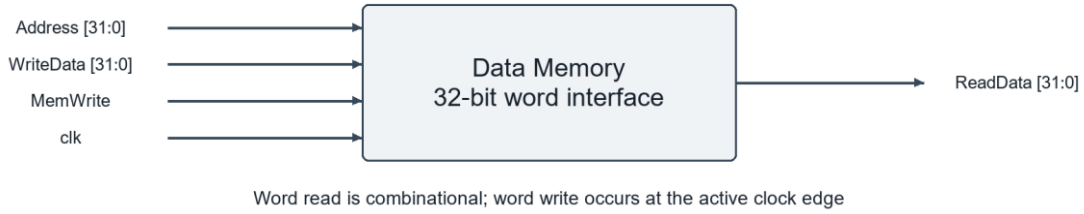


Figure 2.37. Data Memory block. Illustrated based on [15].

Single-Cycle Operation and Timing

For example, lw x5, 4(x1) passes through Instruction Memory, reads x1 from the Register File, forms the immediate 4, and uses the ALU to calculate the effective address. Data Memory then supplies the loaded word. At the ending clock edge, x5 captures the selected memory data and the PC captures PC + 4. An arithmetic instruction uses a shorter path, but still occupies the same clock period [15], [16].

The clock period must accommodate the longest active path, including clock-to-output delay, combinational propagation, and destination setup time. A load commonly determines this limit because it traverses both instruction and data memories. Thus, an ideal single-cycle core has one cycle per instruction, but its clock period can be long.

This architecture provides a direct relationship between instruction behavior and hardware control, making it useful for explaining the processor implementation in Section 2.2.4 [16].

2.3. Convolutional Neural Networks and Quantization

This section presents the fundamental concepts of convolutional neural networks that are directly related to the hardware accelerator developed in this project. The discussion focuses on convolution operations, feature-map processing, computational requirements, and integer quantization. These concepts provide the theoretical foundation for understanding the datapath, arithmetic units, and numerical representation used in the CNN accelerator.

2.3.1. Overview of Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a class of neural networks particularly well suited to processing data with a grid-like spatial structure, such as digital images. Instead of connecting every input element to every neuron as in a conventional fully connected layer, convolutional layers operate on local regions of the input using small learnable filters, commonly referred to as kernels. This local connectivity allows the network to capture spatial relationships among neighboring pixels while avoiding the large number of independent parameters that would be required by a fully connected mapping of the same image data [17], [18].

One of the fundamental properties of a convolutional layer is **parameter sharing**. A convolution kernel contains a set of learned coefficients that is repeatedly applied at different spatial positions of the input. Consequently, the same feature detector can be used throughout an image rather than learning an independent set of parameters for every position. Parameter sharing substantially reduces the number of distinct trainable parameters and enables convolutional layers to detect similar local patterns at different spatial locations [18]. For unit stride and away from padded boundaries, shifting the input produces a corresponding shift in the convolutional representation; this property is known as translation equivariance. Stride, padding, and boundary effects can make the relationship only approximate.

CNNs construct useful representations hierarchically. The first convolutional layers generally operate on small local regions and can learn relatively simple visual patterns,

whereas subsequent layers combine information from preceding feature maps to represent increasingly complex structures. Through multiple processing layers, a deep neural network can therefore transform raw input data into progressively more abstract representations that are useful for tasks such as image recognition and classification [19]. Convolutional neural networks have been particularly successful in applications involving images and other spatially structured signals.

A typical CNN consists of two major stages: **feature extraction** and **classification**. During feature extraction, convolutional layers generate feature maps by applying learned kernels to the input. Nonlinear activation functions are subsequently introduced to increase the representational capability of the network, while pooling operations may be used to reduce the spatial dimensions of the feature maps. These operations can be repeated across multiple layers to construct a hierarchical feature representation. The extracted features are then supplied to a classification stage, which commonly uses one or more fully connected layers to produce the final class scores or predictions [17], [19].

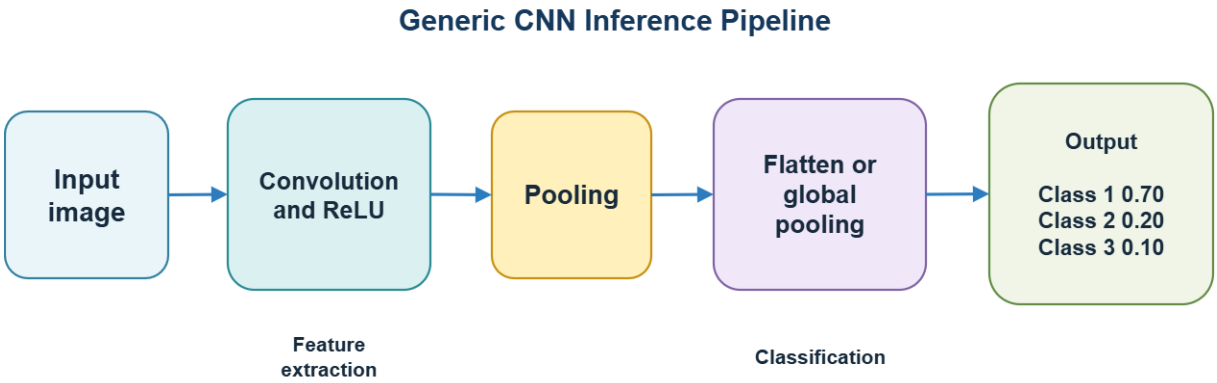


Figure 2.38. General architecture of a Convolutional Neural Network.

The convolution operation is therefore the principal computational component responsible for feature extraction in a CNN. Its regular structure consists of repeated multiplication and accumulation over input activations and learned kernel weights. A detailed description of this operation, including input and output feature maps, channels, kernel dimensions, stride, and padding, is presented in the following subsections.

2.3.2. Two-Dimensional Convolution

Two-dimensional convolution is the fundamental operation used by convolutional neural networks to extract spatial features from input data. A convolutional layer applies a small learnable filter, or **kernel**, to local regions of an input feature map. At each spatial

position, the corresponding input values are multiplied by the kernel coefficients and the resulting products are summed to produce an element of the output feature map [18].

For an input tensor X, a weight tensor W, and an output tensor Y, the operation for output channel o at spatial position (i, j) can be expressed as

$$Y[o,i,j] = b[o] + \sum_{c=0}^{C_{in}-1} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} X\,[c, iS_h + m - P_h, jS_w + n - P_w]W[o,c,m,n]$$

where C_{in} is the number of input channels, K_h and K_w are the kernel height and width, S_h and S_w denote the stride, P_h and P_w represent the padding, and $b[o]$ is an optional bias associated with output channel o. The influence of kernel size, stride, padding, and tensor dimensions on the convolution output is discussed in Section 2.3.3.

For a single-channel input, the operation can be visualized by moving a two-dimensional kernel across the input feature map. At each position, an element-wise multiplication is performed between the kernel and the corresponding input region, followed by summation of all products.

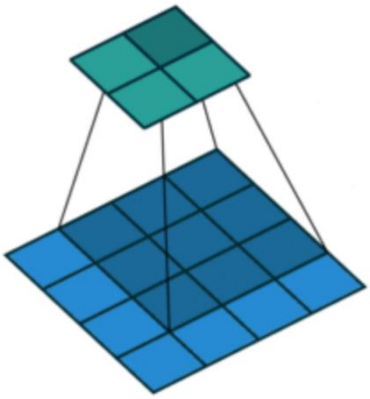


Figure 2.39. Illustration of a convolution kernel operating on a local region of an input feature map

For example, a 3×3 kernel requires nine multiplications for each input channel at one output position.

For multi-channel inputs, each output channel has a separate set of kernel weights for every input channel. The partial convolution results obtained from all input channels are accumulated to produce one output feature map. Consequently, if the input contains C_{in} channels and the kernel size is $K_h \times K_w$ the computation of one output element requires

$$C_{in} \times K_h \times K_w$$

multiplication-and-accumulation terms. A convolutional layer containing C_{out} filters therefore produces C_{out} output feature maps. The corresponding weight tensor has the general shape

$$C_{out} \times C_{in} \times K_h \times K_w$$

In most modern deep-learning frameworks, the operation commonly referred to as convolution is technically **cross-correlation**. Unlike the strict mathematical definition of convolution, cross-correlation does not reverse the kernel before applying it to the input. This distinction generally does not affect neural-network training because the kernel coefficients are learned directly from data [18]. PyTorch follows this convention in its Conv2d operator, which defines the layer output using a two-dimensional cross-correlation between the input and the learned weights [20].

From a hardware perspective, the repeated arithmetic operation involved in convolution can be represented as a sequence of multiply-accumulate operations:

$$ACC_{next} = ACC + A \times W$$

where A represents an input activation, W represents a kernel weight, and ACC stores the accumulated partial sum. Repeating this operation over the kernel elements and input channels produces one convolution output value. This regular multiply-accumulate structure makes convolution well suited to dedicated hardware architectures containing multiple parallel arithmetic units. The computational implications of these repeated operations are discussed further in Section 2.3.5.

2.3.3. Convolution Parameters and Tensor Dimensions

A two-dimensional convolutional layer is characterized by several parameters that determine how the kernel is applied to the input and how the dimensions of the output feature maps are formed. The most important parameters are the **kernel size**, **stride**, **padding**, and the numbers of **input and output channels**. Together, these parameters determine both the spatial dimensions of the output tensor and the amount of computation required by the convolutional layer [18], [21]. Dumoulin and Visin specifically describe the relationships among input shape, kernel shape, zero padding, stride, and output shape in convolutional layers.

Kernel Size

The kernel size defines the spatial extent of the local region processed at each convolution position. For a two-dimensional convolution, a kernel is commonly represented by a height K_h and width K_w. For example, a 3×3 kernel covers nine spatial positions of each input channel at a time.

Smaller kernels are widely used in CNN architectures because multiple convolutional layers can progressively increase the effective receptive field while maintaining a regular computational structure. From a hardware perspective, the kernel dimensions also directly determine the number of multiplication terms required for each output element.

For a kernel of size

$$K_h \times K_w,$$

and C_{in} input channels, one output element requires

$$C_{in} \times K_h \times K_w$$

multiplication terms before accumulation.

Stride

The **stride** specifies the number of spatial positions by which the kernel moves between consecutive convolution operations. A stride of $S = 1$ moves the kernel by one position at a time, whereas a larger stride skips intermediate positions and therefore reduces the spatial dimensions of the output feature map.

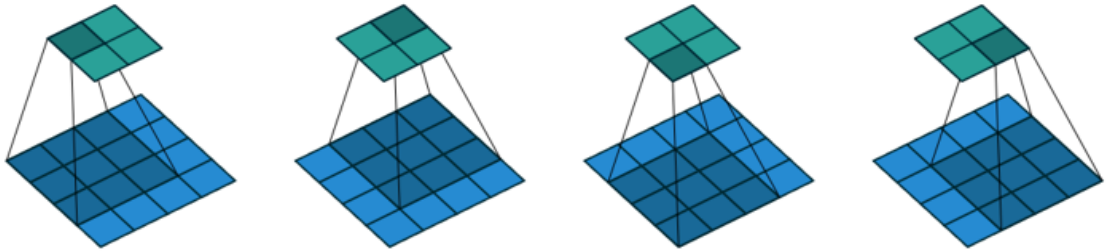
In the general two-dimensional case, separate strides can be defined for the vertical and horizontal dimensions as S_h and S_w. PyTorch similarly allows stride to be specified either as a single value or as a pair of values for the two spatial dimensions.

Padding

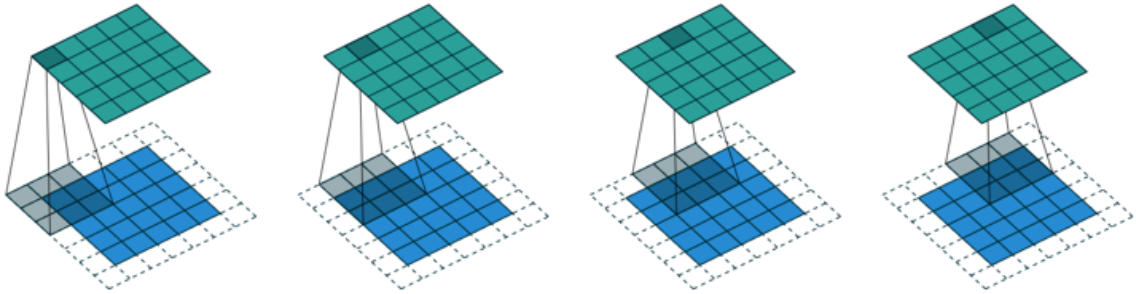
Without padding, convolution can reduce the spatial dimensions of a feature map because the kernel can only be placed where it remains completely within the input boundary. **Padding** introduces additional values around the input boundary, most commonly zeros, allowing the spatial dimensions of the convolution output to be controlled independently from the kernel size [18], [21].

For example, consider a 3×3 kernel with stride $S = 1$. If no padding is applied, $P = 0$, the output spatial dimensions are smaller than those of the input. If one zero-

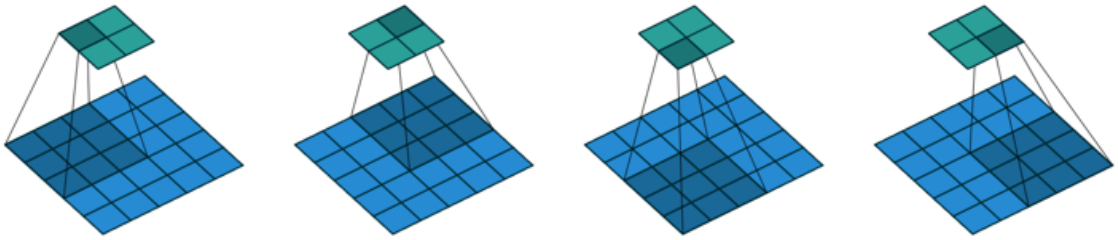
valued element is added around each boundary, $P = 1$, the convolution can preserve the original spatial height and width. This case is often described as **same spatial size** when stride is one.



(No padding, unit strides) Convolving a 3 × 3 kernel over a 4 × 4 input using unit strides (i.e., i = 4, k = 3, s = 1 and p = 0).



(Half padding, unit strides) Convolving a 3 × 3 kernel over a 5 × 5 input using half padding and unit strides (i.e., i = 5, k = 3, s = 1 and p = 1).



(No zero padding, arbitrary strides) Convolving a 3 × 3 kernel over a 5 × 5 input using 2 × 2 strides (i.e., i = 5, k = 3, s = 2 and p = 0).

Figure 2.40. Effect of padding and stride on convolution output dimensions, adapted from [21]

Input and Output Channels

A convolutional layer processes tensors containing one or more channels. Ignoring the batch dimension initially, an input feature map can be represented as

$$X \in \mathbb{R}^{C_{in} \times H_{in} \times W_{in}},$$

where:

- C_{in} is the number of input channels,
- H_{in} is the input height,
- W_{in} is the input width.

For a standard convolution with C_{out} output channels, the weight tensor has the shape

$$W \in \mathbb{R}^{C_{out} \times C_{in} \times K_h \times K_w}.$$

Each output channel corresponds to one learned filter spanning all input channels. Consequently, the resulting output tensor has the shape

$$Y \in \mathbb{R}^{C_{out} \times H_{out} \times W_{out}}.$$

When the batch dimension is included, the conventional tensor representations become

$$N \times C_{in} \times H_{in} \times W_{in}$$

for the input and

$$N \times C_{out} \times H_{out} \times W_{out}$$

for the output, where N denotes the batch size. This corresponds to the tensor convention used by PyTorch Conv2d.***Output Spatial Dimensions***

For a standard convolution with dilation equal to one, the output height and width are determined by the input dimensions, padding, kernel size, and stride:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P_h - K_h}{S_h} \right\rfloor + 1$$

and

$$W_{out} = \left\lfloor \frac{W_{in} + 2P_w - K_w}{S_w} \right\rfloor + 1.$$

These equations are the dilation-1 form of the more general convolution output-size equations used by modern deep-learning frameworks.

For example, consider an input feature map with

$$H_{in} = W_{in} = 28,$$

a 3×3 kernel,

$$K_h = K_w = 3,$$

stride

$$S_h = S_w = 1,$$

and padding

$$P_h = P_w = 1.$$

The output height becomes

$$H_{out} = \left\lfloor \frac{28 + 2(1) - 3}{1} \right\rfloor + 1 = 28,$$

and similarly,

$$W_{out} = 28.$$

Therefore, a 3×3 convolution with stride 1 and padding 1 preserves the input spatial dimensions.

Understanding these relationships is important not only for defining CNN architectures but also for hardware implementation. Kernel dimensions determine the number of arithmetic operations required for each output value, while stride and padding determine which input addresses must be accessed and how many output positions must be generated. These parameters therefore directly influence the data-access pattern, address-generation logic, and control sequence of a convolution accelerator.

2.3.4. CNN Processing Layers

In addition to convolutional layers, convolutional neural networks commonly employ several processing operations to transform extracted feature maps into representations suitable for classification. These operations include nonlinear activation functions, pooling, flattening, and fully connected layers. During training, regularization techniques such as dropout may also be introduced to reduce overfitting [18]. Modern CNNs may replace flattening and fully connected layers with global pooling; the sequence described here represents a conventional classifier architecture.

Rectified Linear Unit

After a convolutional operation, a nonlinear activation function is generally applied to the resulting feature map. One of the most widely used activation functions in convolutional neural networks is the **Rectified Linear Unit (ReLU)**. For an input value x, ReLU is defined as

$$\mathrm{ReLU}(x) = \max(0, x)$$

Therefore,

$$\mathrm{ReLU}(x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases}$$

ReLU suppresses negative values while preserving positive values. The introduction of nonlinear activation functions is important because a sequence of purely linear transformations would still represent only a linear transformation, regardless of the number of layers. ReLU provides a computationally simple nonlinear operation and has become widely used in deep neural-network architectures [22]. Nair and Hinton demonstrated the effectiveness of rectified linear units as an alternative to conventional activation units in neural-network models.

From a hardware perspective, ReLU is particularly attractive because its operation only requires determining whether the input value is negative and selecting either zero or the original value. Consequently, it can be implemented with relatively simple comparison and selection logic.

Pooling

Pooling layers are commonly used to reduce the spatial dimensions of feature maps by summarizing local responses, although some spatial detail is discarded. A pooling operation applies a fixed function to a local region of the input rather than performing a weighted multiplication as in convolution [18], [21].

Two common forms are **max pooling** and **average pooling**. Max pooling selects the maximum value within each pooling window:

$$y = \max_{x \in R} x,$$

where R denotes the local pooling region. Average pooling instead computes the mean value of the elements within the region:

$$y = \frac{1}{|R|} \sum_{x \in R} x.$$

For example, applying 2×2 max pooling to

$$\begin{bmatrix} 1 & 7 \\ 3 & 4 \end{bmatrix}$$

produces

7.

By moving the pooling window across the feature map, the spatial height and width can be reduced. The resulting output dimensions depend on the pooling-window size and stride in a manner similar to convolution. Dumoulin and Visin describe pooling as

a local aggregation operation in which a window is moved across the input and summarized using a function such as the maximum or average [21].

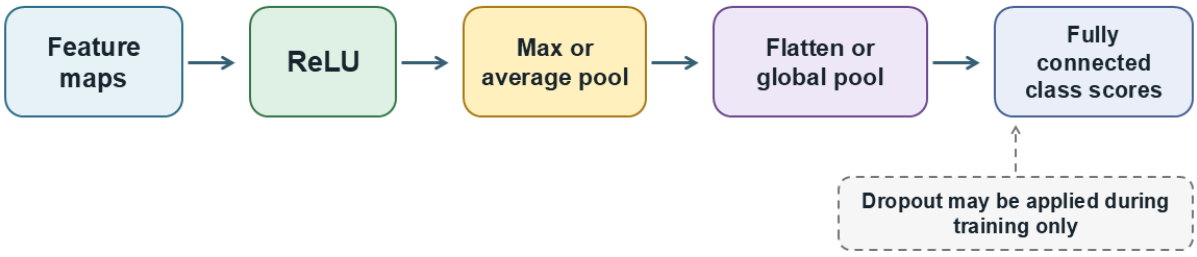


Figure 2.41. Common CNN processing operations; dropout is applied only during training.

Flattening

After feature extraction, the resulting feature maps may need to be converted into a representation suitable for a conventional fully connected classifier. The **flattening operation** rearranges a multidimensional feature tensor into a one-dimensional vector without changing the underlying values.

For a feature tensor with dimensions

$$C \times H \times W,$$

flattening produces a vector containing

$$C \times H \times W$$

elements:

$$X \in \mathbb{R}^{C \times H \times W} \quad \rightarrow \quad x \in \mathbb{R}^{CHW}.$$

Flattening does not itself perform feature extraction or introduce trainable parameters. Instead, it changes the organization of the data so that the extracted features can be processed by subsequent fully connected layers.

Fully Connected Layer

A **fully connected layer**, also referred to as a dense layer, connects each output unit to all elements of the input vector. Its basic operation can be expressed as

$$y = Wx + b,$$

where x is the input feature vector, W is the learned weight matrix, b is the bias vector, and y is the output vector.

In CNN-based classification systems, fully connected layers are typically placed after the feature-extraction stage. Their role is to combine the learned features and map them into a representation suitable for producing class scores. Whereas convolutional layers exploit local spatial connectivity and parameter sharing, fully connected layers perform a global combination of the extracted features [18].

The final classification layer commonly produces one score (logit) for each candidate class. In digit recognition systems, such as the 10-class MNIST classifier (classes 0 to 9), the final layer produces a 10-dimensional output vector $z = [z_0, z_1, \dots, z_9]$. The predicted digit class is the index of the maximum score: $y_{\text{pred}} = \underset{c \in \{0,\dots,9\}}{\text{argmax}}\, z_c$. For integer logits, Argmax avoids the exponentiation and normalization required by Softmax. Because Softmax preserves the ordering of the logits, $\text{argmax}(\text{Softmax}(z)) = \text{argmax}(z)$.

Dropout

Deep neural networks containing many trainable parameters may be susceptible to overfitting. **Dropout** is a regularization technique designed to reduce this problem by randomly omitting a subset of units and their connections during training [23]. Srivastava *et al.* proposed dropout as a method for reducing excessive co-adaptation among neural-network units and demonstrated its effectiveness across several supervised-learning tasks.

During training, a dropout mask can be represented conceptually as

$$\tilde{x} = m \odot x,$$

where x is the input vector, m is sampled from a Bernoulli distribution with retention probability $1-p$, and $\odot$ denotes element-wise multiplication. Under inverted dropout, retained activations are scaled by $1/(1-p)$ during training.

Dropout is a training-time operation. During inference, units are not randomly removed; with the inverted-dropout convention used by common frameworks, the layer behaves as an identity mapping [23]. Consequently, dropout does not normally introduce an additional computational stage into the deployed CNN inference datapath.

The processing operations described above complement convolutional feature extraction. ReLU introduces nonlinearity, pooling reduces spatial dimensions, flattening converts multidimensional feature maps into vectors, and fully connected layers perform the final feature combination required for classification. These operations define the principal processing stages surrounding convolution in a conventional CNN, while the convolutional layers remain responsible for a substantial portion of the repeated arithmetic workload considered in the following section.

2.3.5. CNN Computational Requirements

Convolutional neural networks require a large number of arithmetic operations and memory accesses during inference. Among the different layers of a CNN, convolutional layers are particularly computationally intensive because the same multiply-and-accumulate operation is repeatedly performed over many spatial positions, input channels, and output channels. As CNN models become deeper and process larger feature maps, the resulting computational workload and data-transfer requirements can become substantial [24].

Multiply-Accumulate Operations

As described in Section 2.3.2, the fundamental arithmetic operation of convolution is the multiply-accumulate operation:

$$ACC_{next} = ACC + A \times W,$$

where A represents an input activation, W represents a kernel weight, and ACC stores the accumulated partial sum.

For a convolutional layer with C_{in} input channels, C_{out} output channels, a kernel of size $K_h \times K_w$, and an output feature map of dimensions $H_{out} \times W_{out}$, the total number of MAC operations required for one input sample can be expressed as

$$N_{MAC} = H_{out} W_{out} C_{out} C_{in} K_h K_w.$$

For a standard convolution with groups equal to one, this count assumes one input sample and excludes bias addition, activation, and requantization. The convention used here counts one multiplication followed by one accumulation as one MAC; when individual arithmetic operations are reported, one MAC is commonly counted as two operations. The expression shows that the computational cost increases directly with the number of channels, kernel dimensions, and spatial size of the output feature maps.

Consequently, even relatively small convolution kernels can generate a large number of MAC operations when applied repeatedly across many channels and spatial locations.

The regularity of these operations is important from a hardware perspective. Each output element is produced through the same sequence of multiplications and accumulations, making convolution suitable for architectures containing multiple arithmetic units operating concurrently.

Memory Access and Data Movement

Arithmetic operations represent only one part of the computational cost of CNN inference. Convolution also requires repeated access to three major types of data:

- input activations,
- kernel weights,
- partial sums or output activations.

These data may be stored at different levels of the memory hierarchy, including registers, on-chip buffers, and external memory. Accessing data from distant memory levels generally requires more energy and introduces greater latency than accessing data locally.

Sze *et al.* emphasize that efficient DNN processing depends not only on reducing arithmetic cost but also on minimizing data movement between memory and processing elements [24]. Similarly, the Eyeriss accelerator was specifically designed to reduce expensive data transfers by maximizing local data reuse; its authors identify data movement as a major contributor to the energy cost of CNN computation [25].

Therefore, a hardware implementation should attempt to keep frequently used data close to the arithmetic units whenever possible instead of repeatedly retrieving the same values from external memory.

Data Reuse

Convolution naturally contains several forms of **data reuse**. An input activation can participate in multiple overlapping convolution windows, a kernel weight can be reused at many spatial positions, and partial sums are repeatedly updated until the corresponding output value is complete.

The three principal forms of reuse can be summarized as follows:

- **Weight reuse:** the same kernel weight is applied at multiple spatial positions.
- **Activation reuse:** an input activation may contribute to multiple neighboring output elements.
- **Partial-sum reuse:** intermediate accumulated values are repeatedly updated before the final output is produced.

Efficient hardware architectures exploit these reuse opportunities by storing data in local buffers or registers and organizing the computation so that frequently reused values remain close to the processing elements. This reduces repeated accesses to higher levels of the memory hierarchy and can substantially improve both throughput and energy efficiency [24], [25].

This observation is also the basis of many CNN accelerator dataflow strategies, which determine when activations, weights, and partial sums are moved or retained within the processing architecture.

Parallelism in Convolution

Another important characteristic of convolution is the presence of substantial computational parallelism. Many MAC operations associated with different output channels, spatial positions, or kernel elements can be executed independently or concurrently.

Parallelism can therefore be exploited along several dimensions, including: **output channels, spatial positions, and input channels**.

Instead of computing all convolution operations sequentially with a single arithmetic unit, a hardware accelerator can employ multiple processing elements to execute several MAC operations simultaneously. Spatial accelerator architectures such as Eyeriss exploit arrays of processing elements together with carefully selected dataflows to increase computational parallelism while also reducing costly data movement [25].

However, increasing the number of parallel arithmetic units alone does not guarantee efficient acceleration. The memory system must provide sufficient data to keep these units active, and the dataflow must allow activations, weights, and partial sums to be reused efficiently. Therefore, computational parallelism and memory organization must be considered together when designing a CNN accelerator.

Computational Bottlenecks

The computational requirements of CNN inference can therefore be viewed as two closely related challenges. The first is the large number of arithmetic operations required by convolution. The second is the movement and storage of the data required to support those operations.

A design that provides many MAC units but frequently waits for data may achieve low hardware utilization. Conversely, reducing memory traffic without sufficient computational resources may limit processing throughput. Efficient CNN acceleration therefore requires a balance among arithmetic parallelism, local storage, data reuse, and memory bandwidth [24].

These characteristics motivate specialized CNN hardware architectures that employ parallel multiply-accumulate units, processing-element arrays, local buffers, and dedicated dataflow strategies. The architectural techniques used to address these requirements are discussed in Section 2.4.

2.3.6. Quantization for Hardware Implementation

Neural networks are commonly trained using floating-point representations such as 32-bit floating point (Float32), which provide a wide numerical range and sufficient precision for gradient-based optimization. However, floating-point arithmetic generally requires more storage, data-transfer bandwidth, and hardware resources than low-precision integer arithmetic. For resource-constrained inference systems, quantization is therefore widely used to represent neural-network parameters and activations with a reduced numerical precision [24], [26]. Jacob et al. demonstrated an inference scheme in which both weights and activations are represented using 8-bit integers and computation can be performed using integer arithmetic, while maintaining accuracy close to floating-point inference for the evaluated networks.

Quantization Principle

Quantization maps a real-valued quantity x to a discrete integer value q. A commonly used affine quantization model represents the relationship between the real and integer domains using a positive **scale factor** s and an integer **zero point** z:

$$x \approx s(q - z).$$

The corresponding quantization operation can be expressed as

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right),$$

where $q_{\min}$ and $q_{\max}$ define the numerical limits of the selected integer representation.

The scale s determines the spacing between adjacent quantized values, whereas the zero point z specifies which integer value represents real-valued zero. The clipping operation ensures that values outside the representable integer range are restricted to its minimum or maximum value.

For an 8-bit representation, only $2^8 = 256$ discrete integer levels are available. Therefore, quantization reduces numerical precision compared with Float32 but substantially reduces the number of bits required to represent each activation or weight.

Symmetric INT8 Quantization

A hardware-friendly form of quantization is **signed symmetric quantization**, in which the zero point is fixed to

$$z = 0.$$

The real-value approximation then becomes

$$x \approx sq.$$

For a symmetric signed INT8 range of

$$-127 \leq q \leq 127,$$

the scale may be determined from the maximum absolute magnitude of the tensor:

$$s = \frac{\max|x|}{127}.$$

The quantized value is consequently obtained as

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right), -127, 127\right).$$

The approximate reconstructed real value is

$$\hat{x} = sq.$$

Using a zero-point-free signed INT8 representation simplifies integer multiplication because no zero-point correction is needed. A common scale choice is $s = \frac{\max(|x|)}{127}$, followed by $q = \text{clip}(\text{round}(\frac{x}{s}), -127, 127)$ in the signed 8-bit storage range. This choice still introduces rounding and possible clipping error. Bit-exact agreement between an integer software reference and RTL requires the same scales,

rounding, clipping, saturation, accumulation order, bias handling, and requantization rules; it does not imply bit-exact agreement with Float32 inference. The signed INT8 storage type also permits −128, but the strictly symmetric quantizer used here clips to [−127, 127], leaving that extra negative code unused.

Quantized Activations and Weights

Both the input activations and convolution weights can be quantized:

$$A_{FP32} \rightarrow A_{INT8}$$

and

$$W_{FP32} \rightarrow W_{INT8}.$$

Let s_A and s_W denote their corresponding scales. Their real-valued approximations are

$$A \approx s_A A_q$$

and

$$W \approx s_W W_q.$$

The product of the two quantities can therefore be approximated by

$$AW \approx s_A s_W (A_q W_q).$$

This relationship is important for hardware implementation because the computationally intensive multiplication can be performed directly in the integer domain while the scaling factors preserve the relationship with the original real-valued computation.

Input images and learned weights may be converted from Float32 to signed INT8 after suitable preprocessing and scale calibration. For an input activation x with scale s_{A}, a typical conversion is $q_{\mathrm{A}} = \mathrm{clip}(\mathrm{round}(\frac{x}{s_{\mathrm{A}}}), -127, 127)$. Storing both activations and weights in 8-bit rather than 32-bit format can reduce their raw storage and transfer volume by a factor of four, before accounting for metadata, padding, or intermediate results. The preprocessing constants and calibrated scales used by a particular model should be specified with the implementation.

INT8 Multiplication and INT32 Accumulation

Although activations and weights may be stored as INT8 values, the result of their multiplication requires a wider numerical representation.

For two signed INT8 operands,

$$A_q \times W_q,$$

the product can be represented using a signed 16-bit value. However, convolution does not consist of a single multiplication. Multiple products must be accumulated:

$$ACC = \sum_{k=1}^{N} A_q^{(k)} W_q^{(k)}.$$

When a convolutional layer includes a bias, the bias is represented in INT32 and added to the INT32 accumulator before requantization. Under symmetric quantization, the bias scale equals the activation scale multiplied by the corresponding weight scale. As the number of accumulated terms increases, the required numerical range becomes significantly larger than that of an individual INT8 operand. A wider accumulator is therefore necessary to prevent overflow.

A commonly used integer convolution datapath can consequently be represented as

$$INT8 \times INT8 \rightarrow INT32\ Accumulation$$

where the multiplication operands are low-precision integers while the partial and final sums are maintained with higher precision. Integer-only inference schemes similarly employ wider integer accumulators before converting the result to the required output quantization format [26].

For N accumulated products of signed INT8 operands, the magnitude of the product sum is bounded conservatively by $N \times 128^2$ before bias addition. For example, a 3 × 3 kernel spanning 16 input channels has $N = 3 \times 3 \times 16 = 144$ terms, giving a bound of $144 \times 128^2 = 2{,}359{,}296$, well within the signed INT32 range. This calculation supports using an INT32 accumulator for that product sum, but the complete arithmetic path must also account for the INT32 bias, any subsequent scaling operation, and the chosen overflow or saturation behavior. The bound for a deployed model should be checked against its actual layer configuration.

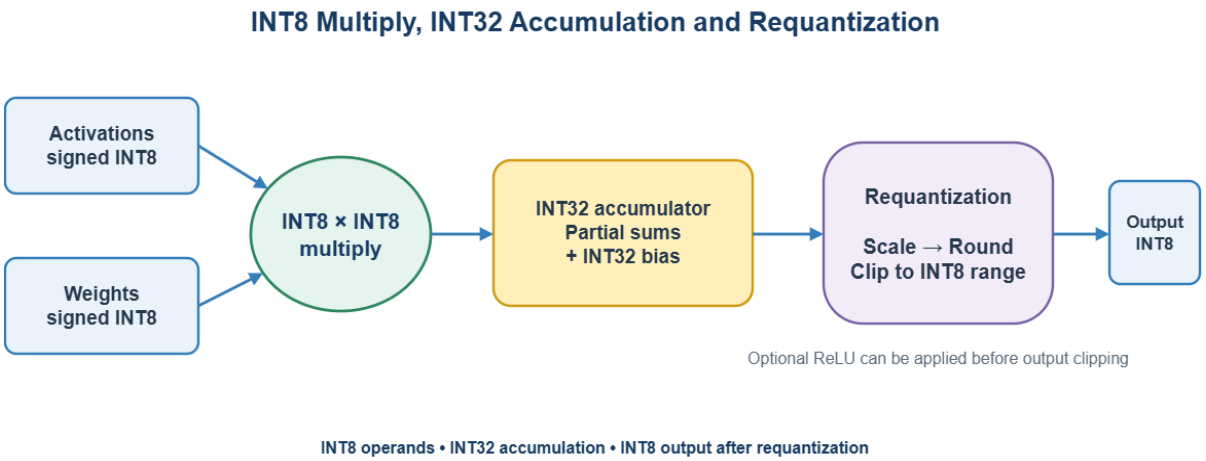


Figure 2.42. Signed-symmetric INT8 convolution datapath considered in this work.

Quantization Error

Since quantization maps many possible floating-point values onto a finite set of integer levels, the reconstructed value $\hat{x}$ is generally not identical to the original value x. The quantization error can be expressed as

$$e_q = x - \hat{x}.$$

Two principal sources of error are **rounding** and **clipping**. Rounding error occurs because an arbitrary real value may lie between two representable quantization levels. Clipping error occurs when the magnitude of a value exceeds the numerical range represented by the selected scale and bit width.

There is therefore a trade-off between numerical range and resolution. A larger scale covers a wider range of real values but increases the distance between adjacent quantization levels. A smaller scale provides finer numerical resolution but increases the possibility of clipping large-magnitude values.

Consequently, the selection or calibration of quantization parameters can influence the accuracy of the quantized neural network. Integer quantization is useful when the reduction in computational and storage cost can be achieved while maintaining an acceptable difference from the original floating-point model [26].

Per-Tensor and Per-Channel Quantization

The scale can be defined at different levels of granularity. In **per-tensor quantization**, a single scale is used for all elements of a tensor:

$$s = s_{\text{tensor}}.$$

This approach is relatively simple and requires little additional quantization metadata.

In **per-channel quantization**, different channels may use separate scales:

$$s_0, s_1, \ldots, s_{C-1}.$$

This can better adapt the numerical representation to differences in magnitude among channels, but it also introduces additional scale information and more complex data handling. For convolution weights, per-channel quantization commonly assigns one scale to each output channel, while activations often retain a per-tensor scale. In this case, the requantization multiplier also depends on the output channel. The appropriate choice depends on the desired balance among numerical accuracy, hardware complexity, storage, and control requirements.

For a hardware accelerator, quantization format and granularity must therefore be defined explicitly so that both software and hardware interpret activation and weight data consistently.

Requantization of Accumulated Results

After the convolution has produced an INT32 accumulated result, the output may need to be converted into the numerical format required by the next neural-network layer. This process is commonly referred to as **requantization**.

For the symmetric case, the real-valued convolution result is approximately

$$Y \approx s_A s_W ACC.$$

If the next layer expects an INT8 output with scale s_Y, an idealized requantization relationship can be expressed as

$$Y_q \approx \text{round}\left(\frac{s_A s_W}{s_Y} ACC\right),$$

followed by clipping to the representable INT8 range.

In practical embedded inference systems, the requantization multiplier $M = \frac{s_{\text{A}} \times s_{\text{W}}}{s_{\text{Y}}} \approx M_0 \times 2^{-n}$ can be represented with an integer multiplier and a right shift. With an explicitly defined rounding rule, the result can be written as $Y_{\text{q}} = \text{clip}(\text{round}(\frac{M_0(ACC_{\text{INT32}} + Bias_{\text{INT32}})}{2^n}), -127, 127)$. This fixed-point formulation avoids floating-point division and permits requantization, ReLU, and spatial pooling in integer hardware or bare-metal RISC-V firmware.

Advantages for Hardware Implementation

Quantization provides several advantages for hardware-oriented CNN inference. Lower-precision operands reduce storage requirements and memory traffic, while integer arithmetic can simplify arithmetic datapaths compared with full-precision floating-point computation. Reduced precision can also allow more arithmetic units or stored values to fit within a fixed hardware-resource budget. These characteristics make quantization an important algorithm–hardware co-design technique for efficient DNN processing.

The quantized convolution datapath follows this processing sequence:

Float32 data → INT8 quantization → INT8 × INT8 multiplication → INT32 accumulation and bias addition → requantization and clipping → INT8 output.

Application to the implemented network. The deployed model applies these principles with signed INT8 activations and weights, signed INT32 convolution accumulators, and firmware requantization. The RTL accelerator executes Conv1 and Conv2 with ReLU; pooling, FC1, FC2, and Argmax remain processor operations. This boundary connects the theory to the implementation without introducing RTL detail in Section 2.3.

2.4. CNN Hardware Acceleration

This section explains the architectural principles used to accelerate convolutional neural networks in digital hardware. It connects the computational requirements and quantized arithmetic introduced in Sections 2.3.5 and 2.3.6 with the structures that execute those operations efficiently. The discussion proceeds from the multiply-accumulate unit to processing elements and arrays, then considers memory hierarchy, dataflow, datapath organization, control, interfaces, and performance metrics. The architectures remain generic so that implementation choices for the proposed accelerator can be presented separately in the design chapter.

2.4.1. Motivation for CNN Hardware Acceleration

Convolutional layers contain a large number of regular multiply-accumulate operations. Although this regularity exposes substantial parallelism, the arithmetic alone does not determine performance. Activations, weights, and partial sums must also be transferred among external memory, on-chip buffers, and arithmetic units. When

reusable data are repeatedly fetched from a distant memory level, transfer latency, bandwidth demand, and energy consumption can dominate the cost of the computation [24].

A dedicated CNN accelerator addresses these constraints by combining parallel arithmetic units with local on-chip storage. In a heterogeneous embedded SoC, a general-purpose processor can manage external communication, memory transfers, layer configuration, and operations that are less suited to the accelerator, while the convolution engine executes the regular multiply-accumulate workload. For the CNN used in this project, Conv1 and Conv2 require 1,016,064 of the 1,218,048 MACs across the two convolutional and two fully connected layers for one input, or approximately 83.4%. This share motivates convolution acceleration, although the achieved speedup also depends on data movement and the remaining software workload.

2.4.2 Multiply-Accumulate Unit

The multiply-accumulate unit, commonly abbreviated as MAC, is the basic arithmetic building block of a convolution engine. For each kernel element and input channel, the unit multiplies an activation A by a weight W and adds the product to a running partial sum ACC. The state update is

$$ACC_{next} = ACC + A \times W$$

A MAC datapath therefore contains a multiplier, an adder, and an accumulator register. The accumulator feeds the current partial sum back to the adder until all products contributing to one output value have been processed. After the reduction is complete, the accumulated result can pass through bias addition, activation, and requantization before it is stored as an output activation.

The numerical widths of the multiplier output and accumulator must be selected independently from the input precision. For example, multiplying two INT8 operands produces a wider signed product, and many such products may be added for one output. A wider accumulator, commonly INT32 in quantized inference pipelines, reduces overflow risk before the result is scaled and clipped to the output range. The required width still depends on the layer dimensions, operand ranges, bias representation, and any saturation policy.

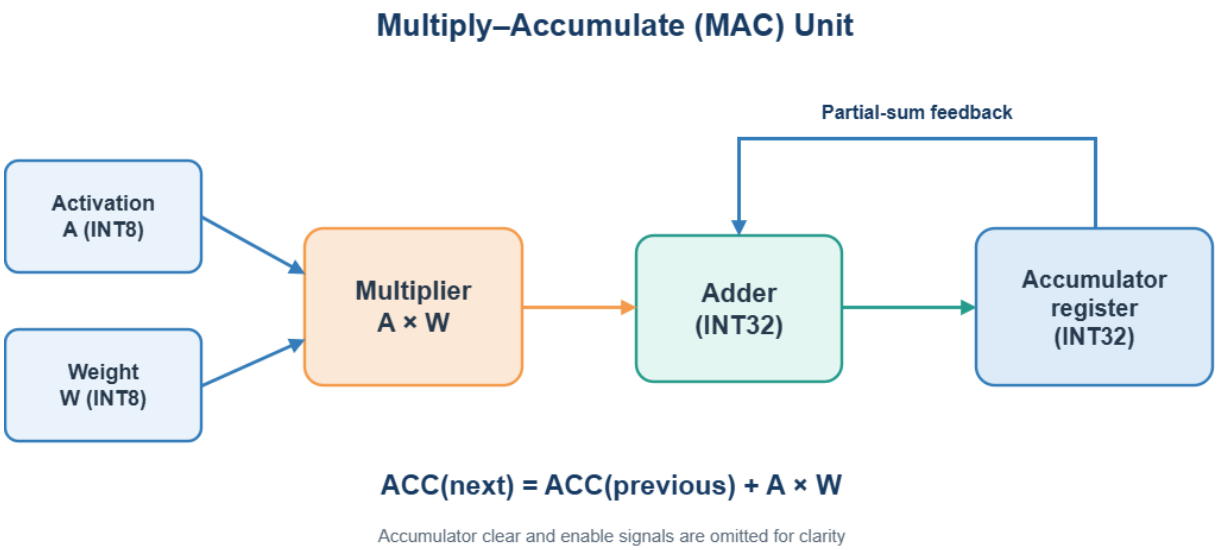


Figure 2.43. Basic architecture of a multiply-accumulate unit. Illustrated by the authors based on [24].

2.4.3. Processing Element

A processing element, or PE, combines one or more MAC units with local registers, operand-selection logic, and control signals. Typical local state includes an activation register, a weight register, and a partial-sum register. The PE receives operands from an on-chip interconnect, performs the scheduled MAC operation, and either retains the partial sum locally or forwards data to a neighboring PE or shared buffer.

Local storage is essential because it allows an operand to participate in several operations without being fetched repeatedly from a larger memory. Which operand remains in the PE depends on the selected dataflow. A weight-stationary mapping retains weights, an output-stationary mapping retains partial sums, and other mappings may retain rows or tiles of several data types. The PE architecture must therefore support the intended reuse pattern and the required communication paths.

A PE may also include gating and validity control so that inactive operations do not update state. These mechanisms are useful when layer dimensions do not exactly match the array dimensions, when padding creates invalid positions, or when a tile contains fewer channels than the hardware can process in parallel. The generic PE in Figure 2.44 illustrates the functional roles without fixing a project-specific RTL implementation.

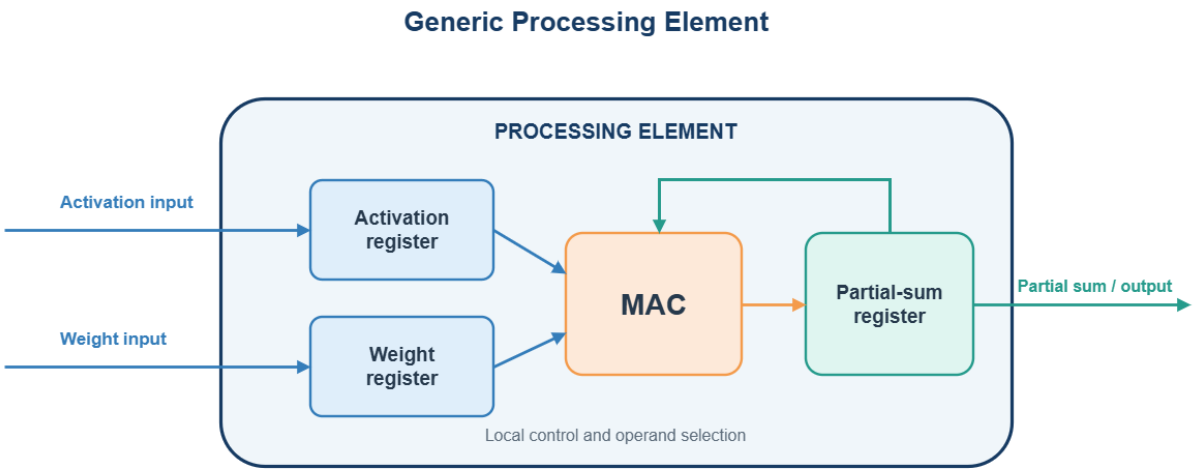


Figure 2.44. General architecture of a processing element for CNN acceleration. Illustrated by the authors based on [24], [25].

2.4.4. PE Array and Parallel Processing

Multiple processing elements can be organized into a two-dimensional array to execute many MAC operations concurrently. In one possible 4 × 4 mapping, four rows serve different output positions and four columns serve different output channels. When all 16 PEs receive valid operands and the array does not stall, it can perform at most 16 INT8 × INT8 MAC operations per clock cycle. A complete dot product requires repeated MAC updates across kernel positions and input channels. Such a mapping does not by itself establish a systolic architecture; that term requires a defined schedule of data movement between neighboring PEs.

Parallel work may be extracted from different convolution dimensions. Several PEs may compute different output pixels, different output channels, or different input-channel contributions at the same time. These mappings are often combined. For example, one array dimension can process output channels while the other processes spatial positions. The best mapping depends on tensor shapes, kernel size, stride, grouping, and available memory bandwidth [24].

Peak throughput is reached only when most PEs perform useful operations in most cycles. Utilization decreases when a layer dimension is smaller than the corresponding array dimension, when boundary conditions create invalid work, or when data delivery stalls the array. Tiling, flexible mapping, and appropriate buffering improve utilization, but they also add control and storage overhead. A generic M by N organization is therefore more suitable for theoretical discussion than a fixed array size.

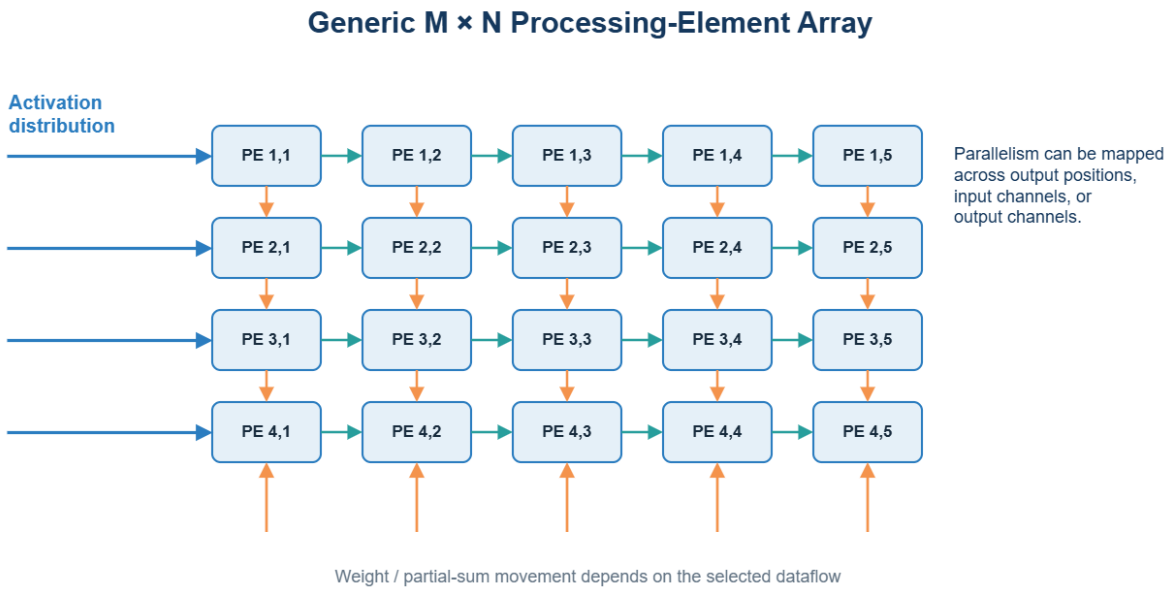


Figure 2.45. Generic processing-element array for parallel CNN computation. Illustrated by the authors based on [24], [25].

2.4.5. Memory Hierarchy and On-Chip Buffers

The memory system determines whether the PE array can sustain its intended throughput without stalling. A typical embedded FPGA SoC may use external nonvolatile storage for model data, CPU-addressable on-chip RAM for firmware and temporary data, dedicated accelerator buffers for input activations, weights, and outputs, and PE-local registers for active operands and partial sums. These storage levels differ in capacity, access cost, and data-reuse opportunity. In the current Tang Nano 20K top-level configuration, the CPU data memory contains 2,048 32-bit words, equivalent to 8 KiB; it should not be confused with the separate accelerator buffers.

The global buffer is commonly partitioned logically into an activation buffer, a weight buffer, and an output or partial-sum buffer. The activation buffer stores input tiles and, in multi-layer execution, may also receive the preceding layer's outputs. The weight buffer holds the filter tile needed by the current computation. The output buffer stores completed results or partial sums that cannot remain inside the PE array.

On-chip capacity is limited, so large feature maps and weight tensors are divided into tiles. A tile is loaded, reused across as many MAC operations as the mapping permits, and then replaced. Double buffering can overlap the transfer of the next tile with computation on the current tile, provided that memory bandwidth and buffer

capacity are sufficient. This technique improves throughput but requires two buffer regions and careful synchronization.

The buffer organization must be designed together with the dataflow. A large buffer cannot compensate for a mapping that repeatedly moves partial sums, and a large PE array cannot remain busy if operands arrive too slowly. The hierarchy in Figure 2.46 emphasizes that storage levels are selected according to both capacity and reuse distance.

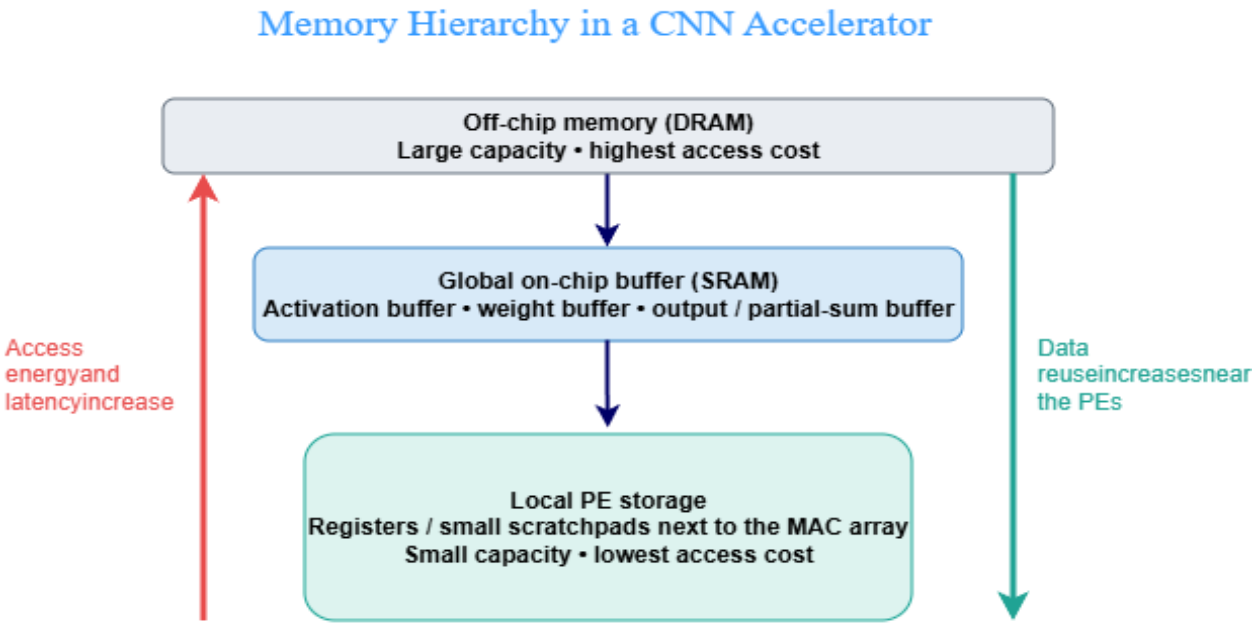


Figure 2.46. Memory hierarchy and on-chip data storage in a CNN accelerator. Illustrated by the authors based on [24], [25].

2.4.6. Data Reuse and Dataflow

Convolution exposes three important forms of reuse. Activation reuse occurs when one input value contributes to several neighboring output positions or output channels. Weight reuse occurs when one kernel coefficient is applied at many spatial positions and, depending on the layer, across a batch. Partial-sum reuse occurs when successive products are accumulated into the same output value. An accelerator dataflow specifies where these operands are stored and how they move during the computation [24].

In a weight-stationary dataflow, weights remain in local PE storage while activations and partial sums move through the array. This organization reduces repeated weight reads and can be effective when each weight is reused many times. Its efficiency

depends on whether the interconnect and reduction structure can move activations and partial sums without creating a bottleneck.

In an output-stationary dataflow, a PE keeps an output partial sum locally while successive activation-weight products update it. If the complete reduction fits within that local state, intermediate partial sums need not be written back to a larger buffer, which can reduce memory traffic. Activations and weights must still be delivered to the PEs, and the benefit depends on tiling, buffering, and available bandwidth. Once accumulation is complete, the result can be transferred for bias addition, activation, requantization, or storage as required by the implementation.

Row stationary is a hybrid dataflow that maps rows of filters and activations onto the array so that several forms of reuse can be exploited together. The Eyeriss architecture uses this approach to reduce data movement across the memory hierarchy [25]. Row stationary is an important example, but it is not universally optimal: the preferred dataflow depends on layer shapes, array organization, storage size, and interconnect cost.

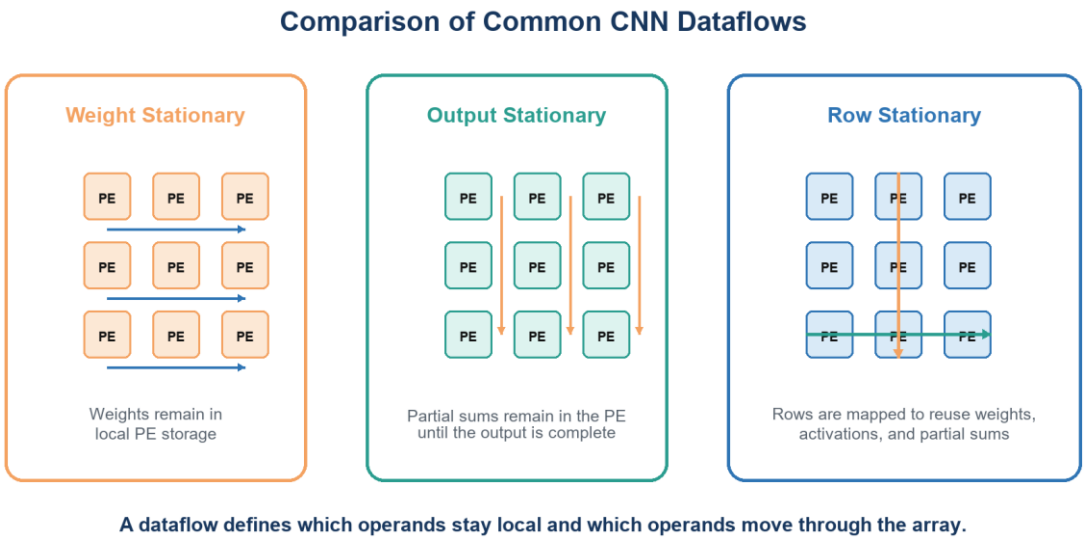


Figure 2.47. Comparison of weight-stationary, output-stationary, and row-stationary dataflows. Illustrated by the authors based on [24], [25].

Table 2.1. Comparison of Common Dataflow Strategies in CNN Accelerators

Dataflow	Data kept near the PE	Primary benefit	Typical trade-off
Weight stationary	Weights	Reduces repeated weight fetches	Activation delivery and partial-sum reduction remain significant
Output stationary	Partial sums	Reduces partial-sum traffic until an output is complete	Weights and activations must be distributed for each output tile
Row stationary	Mapped rows of several operand types	Combines multiple reuse opportunities	Requires more involved mapping, storage allocation, and communication

2.4.7. Accelerator Datapath

The accelerator datapath integrates the memory hierarchy, interconnect, PE array, accumulation path, and output-processing units. Activations and weights are transferred from external memory into their on-chip buffers. The interconnect then supplies the scheduled operands to the PEs, which generate partial sums. Completed accumulations pass through any required bias addition, activation function, and requantization before entering the output buffer.

A convolution that exceeds the array or buffer dimensions is executed as a sequence of tiles. For each tile, the controller selects source addresses, configures loop bounds, issues buffer reads, enables the appropriate PEs, and writes results to the output buffer. Partial sums may remain in the PE, move through a reduction network, or be stored temporarily, depending on the dataflow and the number of input-channel tiles.

For quantized inference, the datapath must preserve scale information across layers. INT8 activations and weights may be multiplied locally, accumulated in INT32, combined with an INT32 bias expressed at the accumulator scale, and then requantized to the next layer's INT8 scale. Saturation and rounding behavior should be defined consistently with the software reference model.

The organization in Figure 2.48 is intentionally generic. It shows the functional path used by many CNN accelerators without prescribing a fixed PE count, buffer depth, bus

protocol, or state encoding. Those parameters belong to the project-specific architecture and implementation.

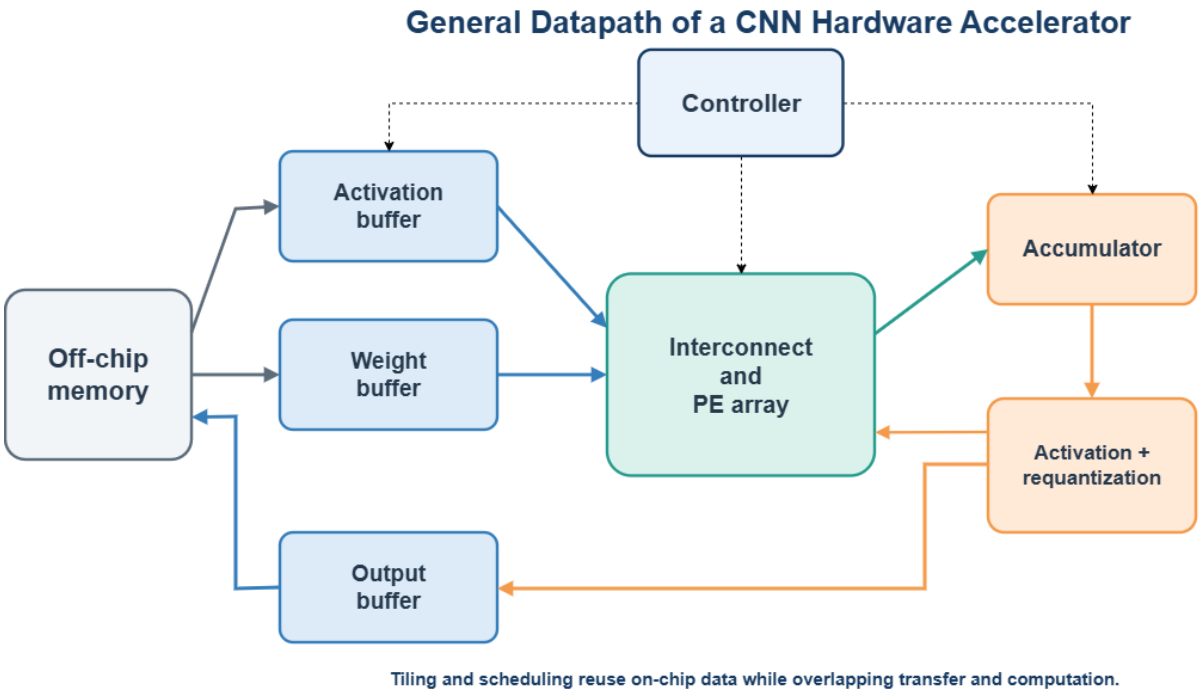


Figure 2.48. General datapath architecture of a CNN hardware accelerator. Illustrated by the authors based on [24], [25].

2.4.8. Accelerator Control Architecture

Control logic coordinates configuration, data transfer, computation, and result write-back. A simple conceptual sequence consists of IDLE, CONFIGURE, LOAD, COMPUTE, WRITE, and DONE states. Real implementations may overlap these activities, repeat LOAD and COMPUTE for multiple tiles, or use several interacting finite-state machines rather than one monolithic controller.

Nested counters represent convolution dimensions such as output channel, input channel, output row, output column, kernel row, and kernel column. Address generators convert these counters into buffer addresses while accounting for stride, padding, and tile offsets. Validity logic prevents padded or out-of-range coordinates from producing unintended memory accesses or accumulator updates.

The controller must also manage stalls and completion conditions. A computation may pause when an input buffer is empty, an output buffer is full, or a memory transaction has not completed. Status flags and error conditions make these events visible to the host system. Figure 2.49 presents a theoretical state sequence rather than the state machine of a specific implementation.

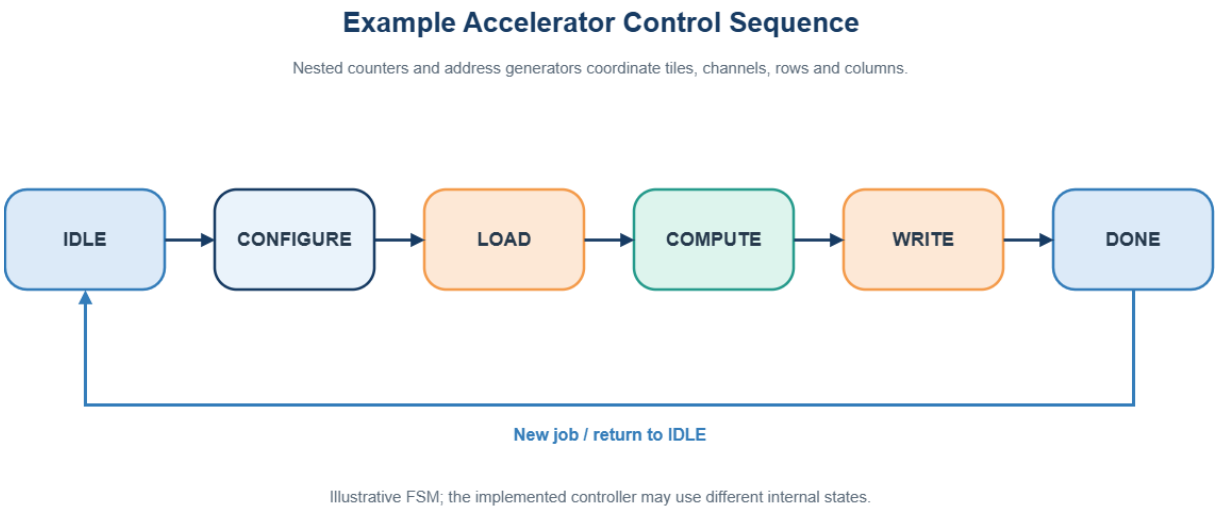


Figure 2.49. Example control-state sequence of a CNN accelerator. Illustrated by the authors.

2.4.9. Hardware Accelerator Interface

A hardware accelerator requires a well-defined interface for configuration, data transfer, and execution monitoring. In the SoC considered here, the accelerator is integrated as a memory-mapped I/O (MMIO) slave at base address 0x4000_0000. The register map is summarized in Table 2.2.

Table 2.2. MMIO Register Map of the CNN Hardware Accelerator

Register Group	**Register / Offset**	**Description**
Control	CTRL (0x00)	Generates START and CLEAR control signals.
Status	STATUS (0x04)	Reports bit 0 BUSY, bit 1 DONE, and bit 2 ERROR.
Input Dimensions	INPUT_H (0x08) INPUT_W (0x0C)	Configures input feature map height and width.
Channel Configuration	CIN (0x10) COUT (0x14)	Specifies input and output channel counts.
Convolution Configuration	KERNEL_SIZE (0x18) STRIDE (0x1C) PADDING (0x20)	Defines convolution kernel size, stride, and padding.
Activation	RELU_EN (0x24)	Enables or disables ReLU activation.
Output Dimensions	OUT_H (0x28) OUT_W (0x2C)	Specifies output feature map height and width.
Input Data	INPUT_DATA (0x30)	Transfers packed INT8 activations (4 values per 32-bit word).
Weight Data	WEIGHT_DATA (0x34)	Transfers packed INT8 weights (4 values per 32-bit word).
Output Data	OUTPUT_DATA (0x38)	Provides 32-bit convolution results (INT32).
Data Counters	INPUT_COUNT (0x3C) WEIGHT_COUNT (0x40) OUTPUT_COUNT (0x44)	Reports or manages input, weight, and output data counts.
Diagnostics	ERROR_CODE (0x48)	Reports configuration or execution errors.

Hardware validation logic checks the configuration before execution. The host processor writes START and then reads STATUS bit 0 (BUSY), bit 1 (DONE), and bit 2 (ERROR). Validation or execution failures are reported through ERROR and can be diagnosed with ERROR_CODE; CONFIG_OK is not a STATUS bit in the current RTL.

Bulk data transfers may be handled through MMIO data ports or dedicated memory interfaces, depending on the system architecture. The specific bus protocol, timing, and optional interrupt mechanism are defined during implementation.

2.4.10. Performance Metrics and Design Trade-offs

An accelerator should be evaluated with metrics that distinguish arithmetic capacity from delivered system performance. Latency measures the time required to complete one inference or layer. Throughput measures completed results or operations per second. PE utilization indicates how much of the available array performs useful work. External memory traffic is the total number of bytes transferred, whereas external memory bandwidth is the transfer rate in bytes per second; reporting both helps identify data-supply limits.

Energy efficiency is influenced by both arithmetic and data movement. Reducing precision can lower multiplier cost and increase the number of operands stored on chip, but numerical accuracy must remain acceptable. Increasing the PE count raises peak throughput, but also increases routing, buffering, and bandwidth requirements. Similarly, larger buffers reduce off-chip traffic but consume silicon area and leakage power.

For this reason, a meaningful evaluation reports layer dimensions, clock frequency, precision, array size, on-chip storage, external traffic, achieved utilization, and output accuracy together. A speedup value without the baseline hardware, software, batch size, and numerical format is not sufficient to compare accelerator designs.

Table 2.3. Performance Metrics for CNN Hardware Accelerators

Metric	What it measures	Why it matters
Latency	Time to complete a layer or inference	Determines response time for real-time and embedded use
Throughput	Work completed per second	Shows sustained processing capability
PE utilization	Fraction of available PE cycles doing valid work	Reveals mapping, padding, and stall losses
External memory traffic	Total bytes transferred to and from off-chip storage	Shows data movement per inference or layer
External memory bandwidth	Bytes transferred per second	Shows whether the data supply limits sustained compute
Energy efficiency	Useful operations or inferences per unit energy	Captures the cost of compute and data movement
Numerical accuracy	Difference from the reference model or task accuracy	Verifies that quantization and saturation remain acceptable

Project boundary. Section 2.4 describes why MAC units, processing elements, arrays, local buffers, control, and MMIO are useful in general. Chapter 3 describes the exact modules, capacities, schedules, state machines, and interfaces implemented in this project.

CHAPTER 3: SYSTEM ANALYSIS, DESIGN, AND IMPLEMENTATION

This chapter maps the final RTL, C firmware, Python tools, model export, and board constraints into one implementation description. It explains the processor, memories, UART, accelerator, microSD, HDMI, firmware, and host application at module level. The chapter also documents the external-image inference mode, in which the desktop application converts a user-selected file into the deployed INT8 format, commits it to a verified microSD slot, restores AI Runtime, and selects that slot for prediction. Every verification statement identifies whether it comes from source inspection, simulation, Gowin implementation, or physical-board evidence.

3.1. System Requirements and Design Objectives

The requirements are grouped by function, FPGA hardware, firmware and host software, AI data, and user interface. This grouping prevents a board-level requirement from being confused with an internal module feature.

3.1.1. Problem Definition

The system must classify a 28 × 28 grayscale digit into classes 0-9 on a resource-constrained GW2AR-LV18 device. A fixed bitstream alone would make each software revision require synthesis and place-and-route, while an all-software convolution would spend many RV32I cycles on repeated loads, multiplications, and additions. The selected architecture keeps control in firmware and moves the regular convolution reductions into dedicated RTL.

The model and the 100-image collection exceed the practical working-memory budget, so they reside on microSD. The board must also expose a repeatable host workflow and visible output. These requirements lead to independent hardware, firmware, and data update paths.

3.1.2. Functional Requirements

Table 3.1 records the requirement category, the implemented response, and the evidence needed to close the requirement. A source feature is not marked as board-verified unless a dated capture or reproducible log exists for the same hardware, firmware, and SD-image versions.

Table 3.1. Functional and hardware requirements of the implemented system

Category	Requirement	Implemented response	Verification criterion
Functional	Classify 28 × 28 digits in classes 0-9	LeNet-inspired INT8 pipeline	Compare with the exported integer reference
Hardware	Operate on Tang Nano 20K	RV32I SoC and 4 × 4 PE accelerator	Inspect synthesis P&R and timing reports
Software	Reload firmware without rebuilding FPGA	BL616 UART loader writes 8 KiB IMEM and DMEM	PING load CRC ACK RUN and reload sequence
AI data	Store model and 100 records externally	RCN1 v3 raw microSD image	Header section image and record CRC checks
User interface	Select stored samples or submit one external image and observe the result	S1, S2, pin-72 reload, GUI preprocessing and verified slot replacement, LEDs, and HDMI	Dated GUI, UART, microSD, and HDMI captures tied to one build set

3.1.3. Constraints and Design Objectives

IMEM and DMEM are each limited to 2,048 words, or 8 KiB. The 27 MHz board clock is divided to a 5.4 MHz SoC clock, while HDMI uses a 27 MHz pixel clock and a 135 MHz serial clock. Convolution operands are signed INT8, products are signed INT16, and accumulation remains signed INT32 until firmware requantizes the tensor.

The accelerator buffers hold at most 3,136 input bytes, 4,608 weight bytes, and 12,544 INT32 outputs. These capacities cover the deployed Conv1 and Conv2 tensors but leave little room for a larger model. The final design must also satisfy FPGA resource and timing limits; Chapter 4 reports that setup passes while hold closure is not complete in the latest inspected report.

3.2. Overall System Architecture

The architecture uses one fixed FPGA hardware image and two independently loadable firmware programs. AI Runtime performs inference. Dataset Manager accepts a replacement image, writes the affected microSD sectors, reads them back, checks CRC values, and commits the record as valid only after verification.

3.2.1. System-Level Organization

Figure 3.1 shows the principal control and data paths. Before RUN, the loader owns the serial path and the CPU is held in reset. The GUI writes IMEM and DMEM through the onboard BL616 USB-to-UART bridge. After RUN, the RV32I processor becomes bus master, reads raw sectors through SD-SPI, programs the accelerator, executes software layers, and commits image and result fields for HDMI.

The accelerator never reads microSD directly. Firmware moves each required tensor through MMIO and checks status, counts, and error codes. This boundary keeps file-format and policy decisions in software while the RTL compute path remains regular.

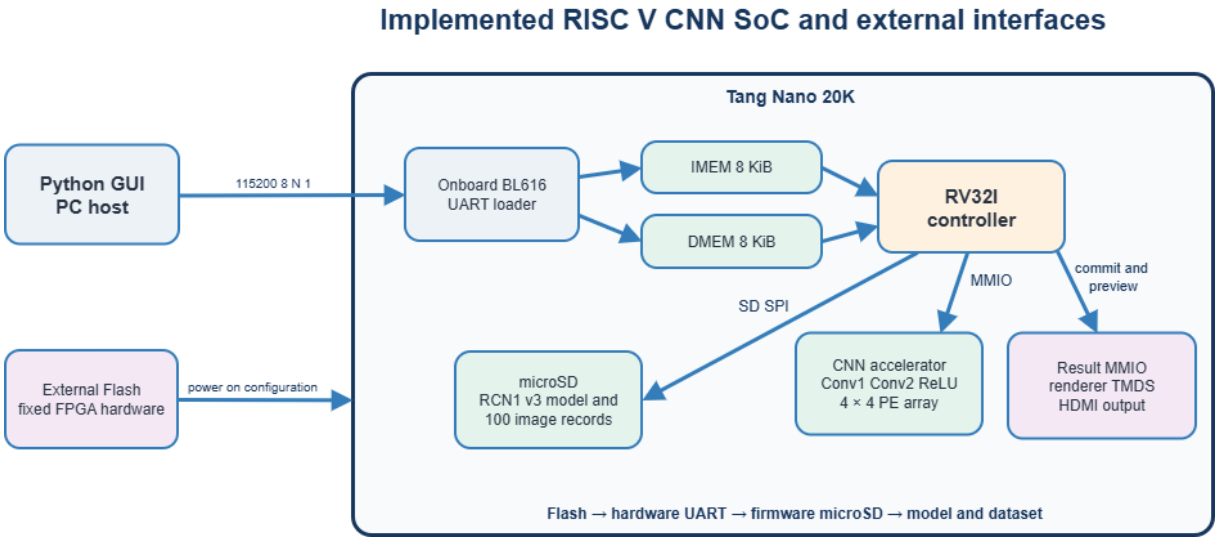


Figure 3.1. Implemented RISC-V CNN SoC organization and external interfaces.

3.2.2. Hardware and Software Partitioning

Table 3.2 separates permanent hardware from reloadable software and replaceable AI data. Conv1 and Conv2 plus ReLU execute in the accelerator. Pooling, requantization, FC1, FC2, Argmax, SD transactions, and result publication execute in firmware. The GUI prepares and deploys artifacts but does not participate in the inference arithmetic.

Table 3.2. Processing responsibility by layer or service

Layer	Contents	Update path
FPGA hardware	RV32I memories MMIO UART SD-SPI accelerator controls HDMI	Program .fs to external Flash
RV32I firmware	AI Runtime or Dataset Manager	Load IMEM and DMEM through UART

Layer	Contents	Update path
microSD data	RCN1 v3 model quantization metadata and 100 image records	Write full image or verified slot update

3.2.3. Interconnection and Address Map

The CPU uses separate instruction and data paths. IMEM is fetched from address 0x0000_0000. C firmware uses DMEM at 0x8000_0000, while a low-address alias remains for legacy assembly. High address regions decode to result handoff, runtime UART, SD-SPI, the CNN accelerator, and the UI/preview block. Invalid accesses assert the CPU data-fault input.

Synchronous DMEM and accelerator reads require wait behavior. The SoC inserts a read bubble so the CPU does not consume stale data or advance through a back-to-back load before the selected block returns a registered word.

Table 3.3. CPU-visible memory map used by the C firmware

Address range	Block	Purpose
0x0000_0000	IMEM	8 KiB instruction fetch and loader writes
0x8000_0000	DMEM	8 KiB data BSS workspace and stack
0x1000_0000-0x1000_000C	Result MMIO	Prediction sample ID confidence commit
0x2000_0000-0x2000_000C	Runtime UART	RX TX status control
0x3000_0000-0x3000_000C	SD-SPI	Control status byte data divider
0x4000_0000-0x4000_0048	CNN accelerator	Configuration data ports counts status and errors
0x4000_1000	Result stream	Write-only host result path
0x5000_0000-0x5000_0160	UI and preview	Sample controls and 25 preview words

3.2.4. End-to-End Operation

At power-on, external Flash restores the .fs hardware image. The GUI opens the BL616 COM port, sends PING, transfers CRC32-protected IMEM and DMEM chunks of at most 256 bytes, and sends RUN. AI Runtime initializes microSD, validates the RCN1 v3 header and section CRC values, loads the selected sample and model data, executes the hardware/software CNN pipeline, and commits the result to HDMI.

Holding the external pin-72 button high for 13,500,000 cycles of the 27 MHz clock, approximately 0.5 s, generates a reload request. The request is stretched for 255 system-clock cycles, synchronized to the 5.4 MHz domain, resets the CPU, and restores loader ownership. The FPGA hardware and HDMI clock path remain configured.

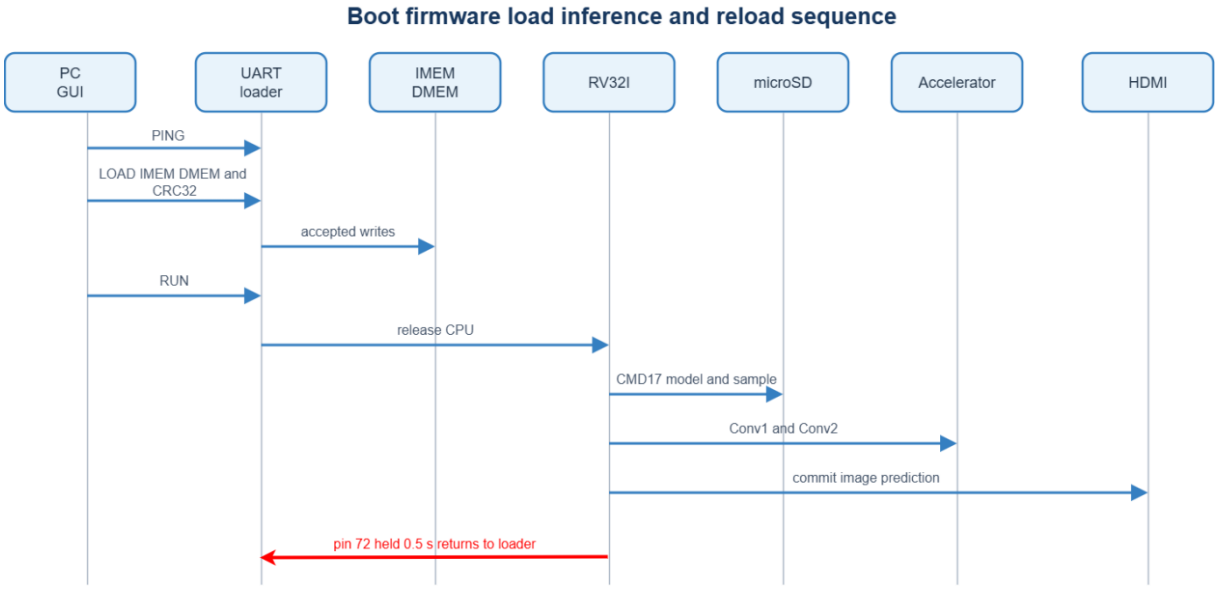


Figure 3.2. End-to-end operation sequence from boot to result display.

3.3. CNN Model Design and INT8 Deployment

The deployed model is LeNet-inspired and uses two 3 × 3 convolution layers with padding one and stride one, two 2 × 2 max-pooling stages, FC1, FC2, and Argmax. Figure 3.3 and Table 3.4 show the tensor shapes and execution owner for every stage.

3.3.1. Deployed Network

The input is one 28 × 28 tensor. Conv1 generates 16 × 28 × 28 and Pool1 reduces it to 16 × 14 × 14. Conv2 generates 32 × 14 × 14 and Pool2 reduces it to 32 × 7 × 7. Flatten produces 1,568 values, FC1 produces 128 activations, and FC2 produces ten logits for Argmax.

Conv1 requires 112,896 MACs and Conv2 requires 903,168 MACs per image. FC1 and FC2 together require 201,984 MACs. The two accelerated convolutions therefore represent approximately 83.4 percent of the learned layers' MAC count. This count motivates the partition but is not a measured speedup.

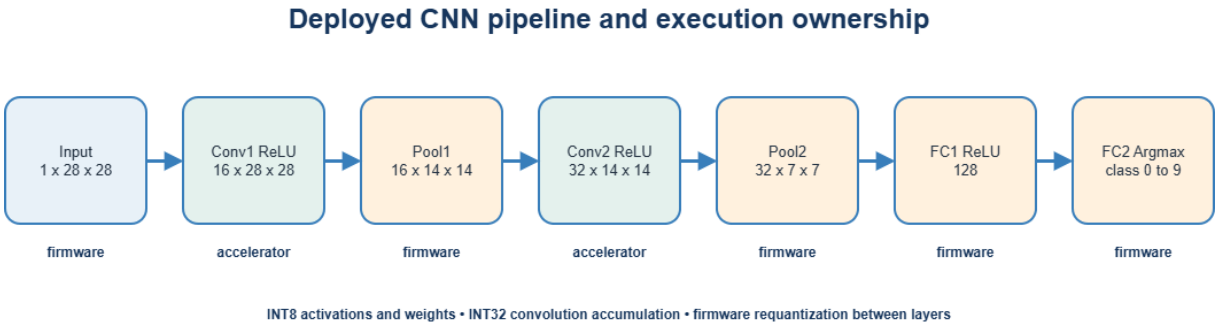


Figure 3.3. Hardware and software execution flow of the deployed CNN.

Table 3.4. Layer dimensions and implementation

Stage	Input	Output	Execution
Conv1 ReLU	1 × 28 × 28	16 × 28 × 28	Accelerator
Pool1	16 × 28 × 28	16 × 14 × 14	RV32I firmware
Conv2 ReLU	16 × 14 × 14	32 × 14 × 14	Accelerator
Pool2	32 × 14 × 14	32 × 7 × 7	RV32I firmware
FC1 ReLU	1568	128	RV32I firmware
FC2 Argmax	128	10 logits class 0-9	RV32I firmware

3.3.2. Training, Quantization, and Export

The model is trained in PyTorch [30] with MNIST data [31]. The exporter applies symmetric per-tensor INT8 quantization with zero point zero. Activations and weights are stored as signed bytes, each product is sign-extended, and convolution sums remain signed INT32. Fixed-point multipliers and shifts reproduce the next layer's scale in firmware.

The exporter, firmware, and RTL must agree on tensor order, byte packing, rounding, saturation, bias handling, and section CRC values. The running FPGA does not retain a Float32 network. Float32 and simulated-INT8 accuracy therefore describe offline model evidence, while the full firmware simulation checks the exported integer path.

Table 3.5. Numeric formats and exported data used by integer inference

Item	Representation	Use
Activation and weight	Signed INT8 zero point 0	Packed model and intermediate tensors
Product	Signed INT16	Sign-extended before accumulation
Accumulator	Signed INT32	Convolution output and hardware ReLU
Requantization	Fixed-point multiplier shift rounding clip	Map INT32 to next-layer INT8
Classifier output	Ten integer logits	Firmware Argmax selects class

3.3.3. Input Samples and SD Image

The raw RCN1 v3 image stores model sections and 100 image records. The dataset begins at sector 408. Each record is 816 bytes: sample ID, class label, valid flag, image CRC32, whole-record CRC32, a calibration maximum, ten reference-confidence values, reserved alignment bytes, and 784 signed INT8 pixels. The complete image is 290,816 bytes, or 568 sectors.

The GUI accepts PNG, JPG, JPEG, or BMP files and converts the selected file to RGB and then grayscale. It estimates the background from border pixels and inverts a light-background image, applies autocontrast and a small Gaussian blur, computes an Otsu threshold with a minimum threshold of 12, closes one-pixel gaps, crops the detected foreground, and scales the digit so that its longest dimension occupies at most 20 pixels. The result is placed on a 28 x 28 black canvas and shifted by at most two pixels in each direction to align its center of mass.

External Image Quantization and Reference Metadata

Thin strokes can disappear when a photograph is reduced to 20 pixels. The preprocessing path therefore measures foreground coverage and applies a limited blend with a one-pixel dilation only when coverage is below 0.22, followed by a contrast adjustment. This preserves narrow strokes without deliberately closing the loops in digits such as 0, 6, 8, and 9.

Each pixel is normalized with the MNIST mean 0.1307 and standard deviation 0.3081, divided by the deployed input scale 0.02221643079922894, rounded, saturated

to the signed INT8 range, and stored in two's-complement byte form. The resulting payload is exactly 784 bytes and is compatible with the same CNN input path used by the original 100 records.

Before the file is transferred, the host reference path executes the exported integer convolution and dense parameters. It derives a Conv2 calibration maximum, ten percentage confidence bytes, and a host-side predicted class. These values support preview and metadata generation; they do not replace the prediction produced later by the FPGA. The assigned label, host reference prediction, and FPGA prediction must therefore be recorded as separate fields.

Fixed Slot Strategy

The external file is not streamed directly into the accelerator as a temporary tensor. The current implementation replaces one of the 100 fixed RCN1 records, verifies the updated record on microSD, and then runs AI Runtime from that slot. This design retains the image after a reset, keeps inference input handling identical for stored and external samples, and avoids adding a second unverified data path to the accelerator. Replacing a record does not alter model weights; supporting a new class outside 0-9 still requires retraining and export.

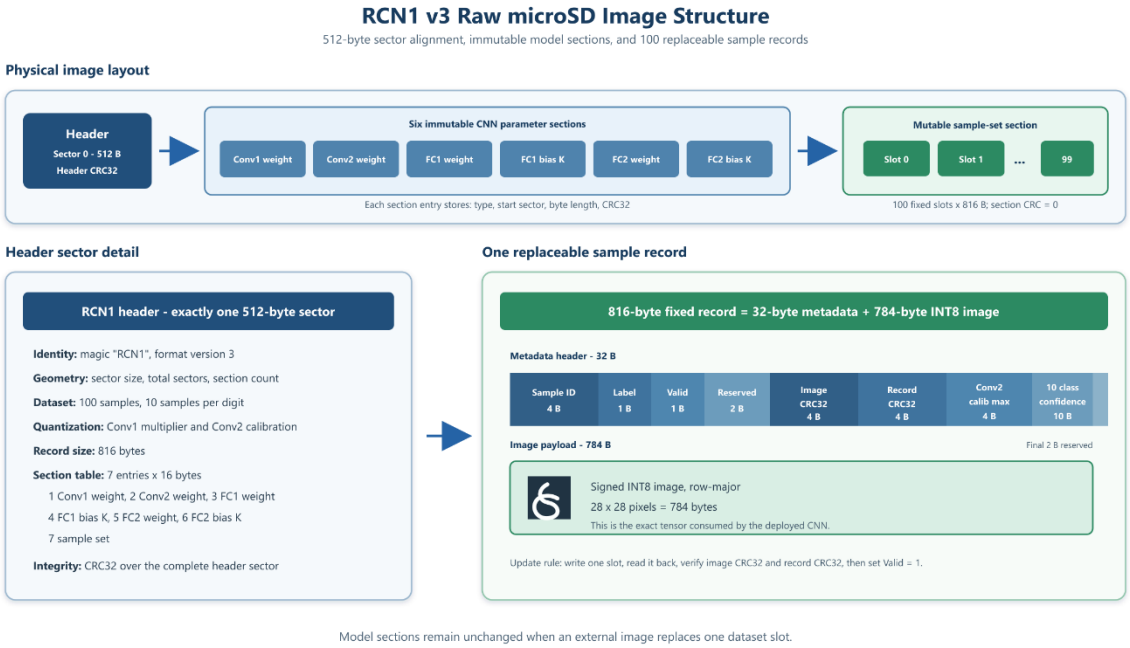


Figure 3.4. RCN1 Image Structure and Sample Record Organization

3.4. CNN Hardware Accelerator Design

The accelerator is a memory-mapped convolution peripheral. The hierarchy separates CPU-facing registers, validation, transaction control, loop sequencing, address generation, synchronous data feeding, arithmetic, and output storage. This separation lets each block be verified independently before full-system simulation.

3.4.1 Datapath Overview

Purpose and interface. accelerator_mmio accepts aligned 32-bit CPU reads and writes from 0x4000_0000 through 0x4000_0048. Four signed bytes are packed into each INPUT_DATA or WEIGHT_DATA word. Configuration registers describe the input, output, channel, kernel, stride, padding, and ReLU settings.

Internal operation. accelerator_validation checks dimensions and loaded counts against the implemented capacities. accelerator_controller uses IDLE, CHECK, RUN, DONE, and ERROR. accelerator_compute_core connects the buffers, sequencer, feeder, PE array, output tile writer, ReLU, and output buffer.

Verification. MMIO tests must cover legal and illegal accesses, packed-byte order, counter updates, START and CLEAR behavior, registered output reads, configuration errors, completion, and timeout handling.

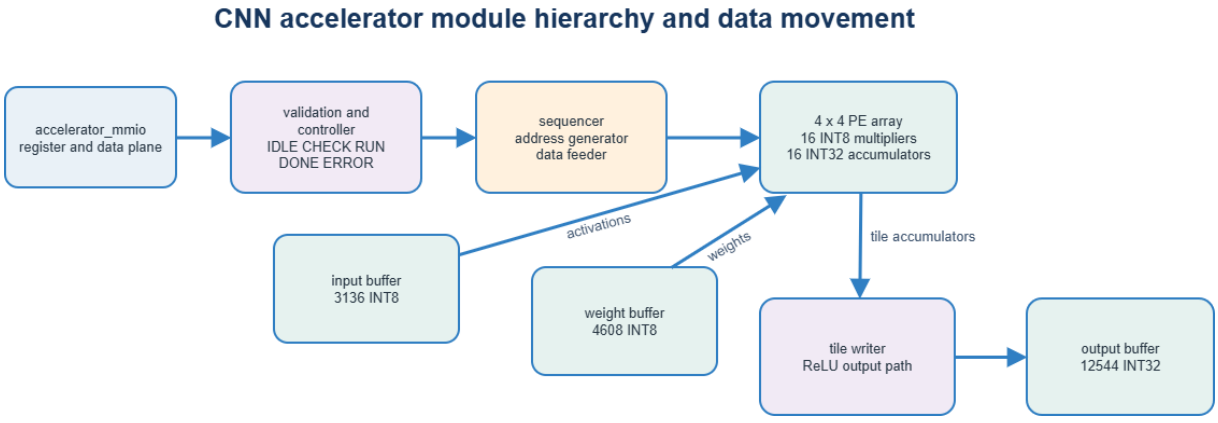


Figure 3.5. Functional blocks and data movement inside the convolution accelerator.

3.4.2. MAC Unit and Processing Element

MAC unit. mac_unit forms a signed 16-bit product from one signed INT8 activation and weight, sign-extends it to 32 bits, and adds the prior accumulator. It is combinational.

Processing element. pe adds the clocked INT32 accumulator. clear_acc resets the partial sum, and enable allows an update only when the sequencer and feeder present a valid reduction term. Without enable, the register retains its value.

Verification. Directed tests use positive, negative, zero, clear, hold, and overflow-sensitive operand patterns. The registered result is compared with a signed software reference after every valid contribution.

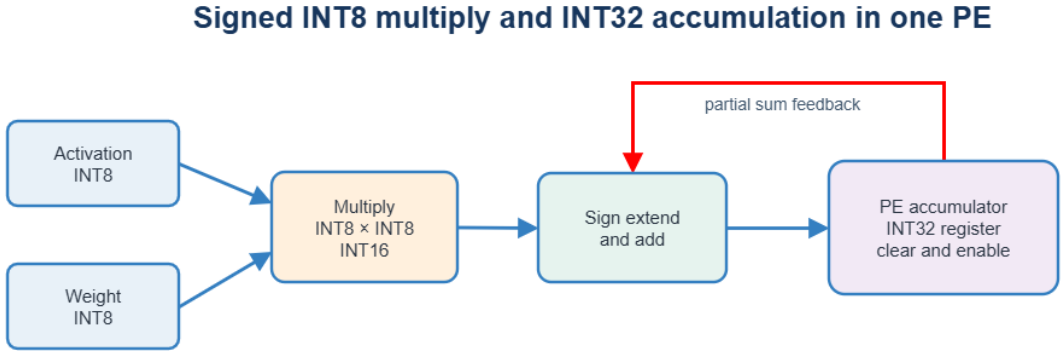


Figure 3.6. Signed MAC and accumulator update in one processing element.

3.4.3. Four-by-Four PE Array and Tiling

Purpose and mapping. pe_array_4x4 instantiates sixteen PEs. Four activation lanes broadcast by row and four weight lanes broadcast by column. One tile represents four output-width positions by four output channels when all lanes are valid.

Boundary behavior. padding and tensor tails generate validity masks. Invalid spatial positions supply zero or suppress updates, and incomplete channel tiles disable nonexistent PE columns. The current array is broadcast-based; it is not a neighbor-to-neighbor systolic dataflow.

Verification. All sixteen accumulator lanes are compared with a software golden tile for interior, padded-corner, width-tail, and channel-tail cases

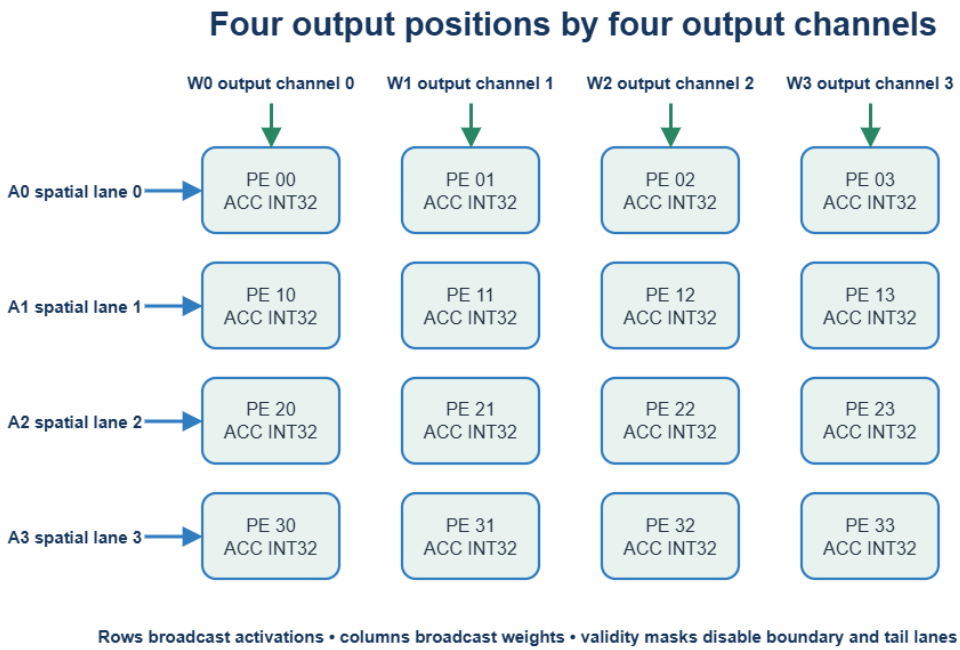


Figure 3.7. Spatial and output-channel mapping of one 4 x 4 PE tile.

3.4.4. Local Buffers and Data Feeding

Storage. input_buffer stores 3,136 signed bytes, weight_buffer stores 4,608 signed bytes, and output_buffer stores 12,544 signed INT32 values. These arrays are distinct from the CPU's DMEM.

Feeder. conv_data_feeder_4x4 converts four synchronous byte reads into 32-bit activation and weight buses. Its IDLE, ISSUE0, ISSUE1, ISSUE2, ISSUE3, and CAPTURE3 states account for read latency before data_valid is asserted.

Verification. Buffer tests cover packed writes, capacity boundaries, synchronous read latency, feeder-state alignment, and exact loaded and stored counts.

Table 3.6. Accelerator-local storage in the RTL

Storage block	Capacity	Purpose
Input buffer	3136 INT8	Conv2 input capacity and packed MMIO load
Weight buffer	4608 INT8	All Conv2 3 × 3 × 16 × 32 weights
Output buffer	12544 INT32	Completed convolution values for CPU readback
PE accumulators	16 × INT32	One 4 × 4 spatial and channel tile

3.4.5. Address Generation, Sequencing, and Output

Sequencer. conv_sequencer_4x4 enumerates output row, a four-position output-width base, a four-channel output base, input channel, kernel row, and kernel column. It

clears a tile, issues a reduction step, waits for step_done, and holds tile_ready until the output path accepts the accumulators.

Address generator. conv_address_generator_4x4 calculates four input-byte addresses and four weight-byte addresses from the loop indices. It applies stride and padding and produces lane-valid masks for borders and tails.

Output path. output_tile_writer serializes valid accumulator lanes into row-major tensor order. relu_unit replaces negative signed INT32 values with zero when enabled. output_path controls tile acceptance, stored_count, and sequential CPU readback.

Verification. Generated addresses and masks are compared with a software loop for corner and interior tiles. Waveforms must show index progression, ISSUE and CAPTURE states, data_valid, PE enable, tile_ready, tile_accept, output address, data, and count.

3.4.6. Controller and MMIO Protocol

A START request leaves IDLE and enters CHECK. A valid configuration enters RUN; an invalid configuration enters ERROR. Completion enters DONE. DONE and ERROR remain visible until CLEAR returns the block to IDLE. Firmware writes configuration and operands, checks the input and weight counts, starts the transaction, polls status, reads ERROR_CODE on failure, and streams exactly OUTPUT_COUNT words through OUTPUT_DATA.

The MMIO read path is registered. The SoC inserts the required wait cycle so consecutive CPU loads do not observe the same output word. Subword or misaligned accelerator accesses assert a CPU data fault rather than silently altering the transaction.

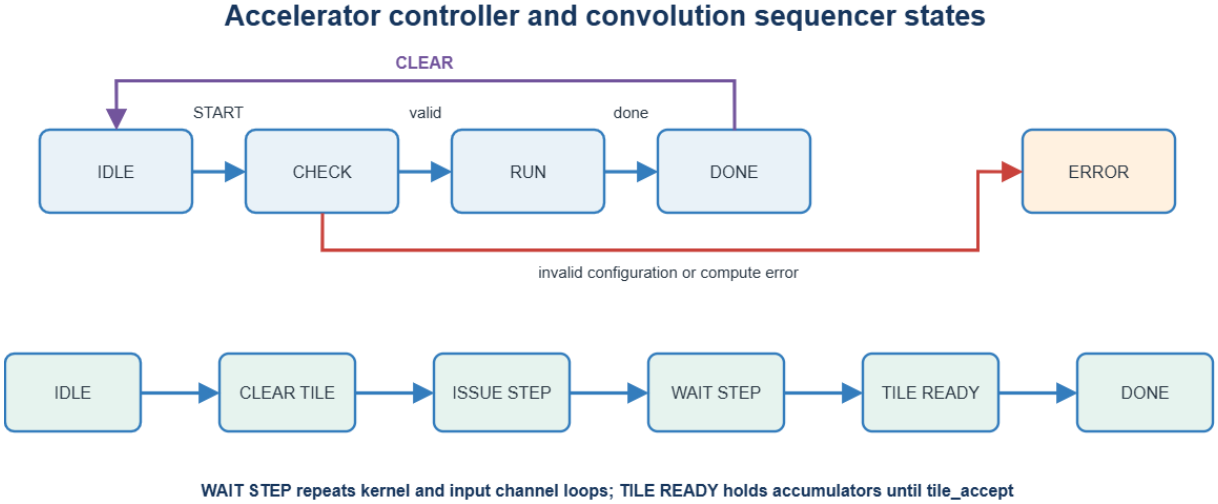


Figure 3.8. Accelerator Controller and Sequencer State Transitions

Table 3.7. Host-visible accelerator control and status

Offset	Register or signal	Function
0x00	CTRL	START bit 0 CLEAR bit 1
0x04	STATUS	BUSY DONE ERROR
0x08-0x2C	Configuration	H W Cin Cout kernel stride padding ReLU and output dimensions
0x30	INPUT_DATA	Four packed INT8 activation bytes
0x34	WEIGHT_DATA	Four packed INT8 weight bytes
0x38	OUTPUT_DATA	Sequential signed INT32 output
0x3C-0x44	COUNTS	Stored input weight and output counts
0x48	ERROR_CODE	Validation or execution diagnostic

3.5. RISC-V SoC and Memory Subsystem

The system combines a team-developed 32-bit, non-pipelined RV32I processor with separate instruction and data memories and memory-mapped peripherals. The processor implements the base integer instruction classes required by the bare-metal C firmware and exposes synchronous instruction and data buses, byte write strobes, access size, wait, fault, trap, and debug signals to the SoC top level.

3.5.1. RV32I Processor Architecture

Program Counter

The 32-bit program counter addresses IMEM and normally advances by four. Branch, JAL, and JALR targets replace PC+4 when their decoded conditions are met. A bus wait freezes the PC so an outstanding synchronous read or MMIO transaction is not repeated with a different address.

Instruction Decode and Control

The decoder recognizes LUI, AUIPC, JAL, JALR, conditional branches, loads, stores, immediate ALU operations, register ALU operations, and FENCE. Opcode, funct3, and funct7 select ALU behavior, branch comparison, load sign extension, store strobes, and register write-back. Unsupported encodings raise an illegal-instruction trap.

Register File

Thirty-two 32-bit integer registers supply two source operands and one destination write port. Register x0 always reads as zero and discards writes. The destination is written only when the instruction completes without a wait or fault.

Immediate Generator

I-, S-, B-, U-, and J-type immediates are assembled from the instruction fields and sign-extended to 32 bits. The generated value supplies ALU immediates, effective-address offsets, branch displacements, and jump targets.

Arithmetic Logic Unit

The ALU performs addition, subtraction, logical operations, shifts, and signed or unsigned comparisons required by RV32I. It also calculates load/store addresses and AUIPC results. Multiplication used by the C runtime is supplied by the compiled support routine rather than an ISA M-extension instruction.

Branch and Jump Logic

Branch comparison implements equality, inequality, signed less-than or greater-or-equal, and unsigned comparisons. JAL writes PC+4 and applies the J immediate; JALR adds the I immediate to rs1 and clears bit zero of the target.

Load and Store Interface

Data memory and MMIO peripherals remain outside the CPU core. The ALU result supplies the byte address, the second register operand supplies store data, and the load/store interface issues read or write enable, access size, and four byte strobes. Returned byte and halfword values are selected from the 32-bit bus word and sign- or zero-extended according to funct3. Misaligned accesses and decoded access faults set the corresponding trap cause.

Stall Wait and Trap Handling

bus_wait holds architectural state while synchronous DMEM or MMIO completes. A load commits its destination only after valid read data arrives. Illegal instructions, misalignment, or access faults record a cause and assert the debug trap output; firmware and simulation use the trap state to diagnose invalid transactions.

Verification. CPU unit tests and full-firmware simulation observe PC progression, decoded instruction classes, register writes, branch targets, load/store formatting, wait-state stability, and trap causes. The full CNN simulation completed with zero errors.

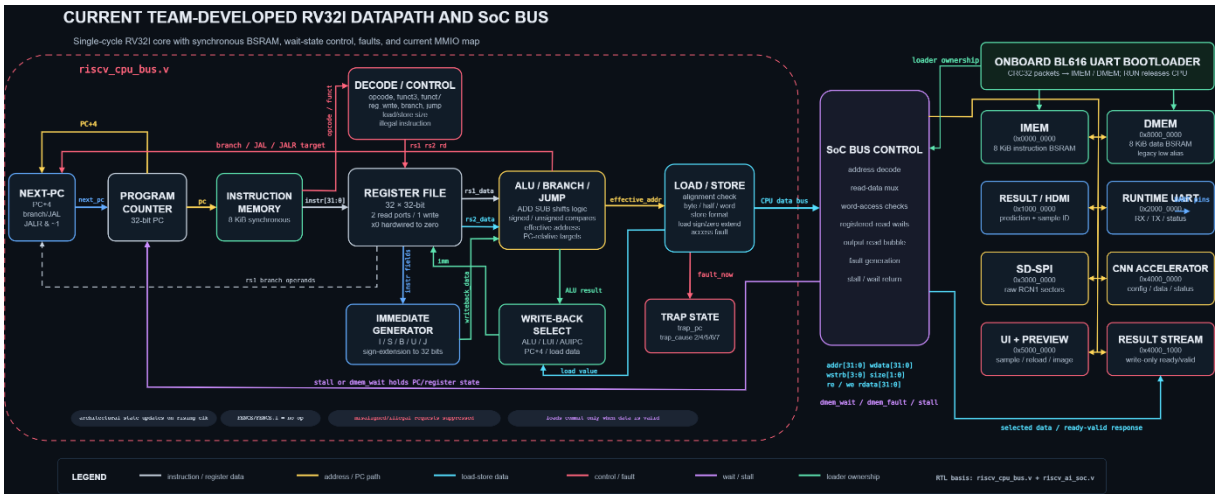


Figure 3.9. Detailed datapath and bus interface of the implemented RV32I processor.

3.5.2. Instruction and Data Memory Organization

IMEM and DMEM are separate 2,048 × 32-bit physical memories, each 8 KiB. IMEM uses a loader write port and synchronous CPU instruction read port. DMEM accepts loader writes before RUN and CPU byte-strobed writes during runtime. The power-of-two depth and synchronous reads allow Gowin BSRAM inference.

Ownership is mode-based. In Loader mode, the CPU remains in reset and only CRC-accepted loader packets may write IMEM or DMEM. In Runtime mode, bootloader writes are disabled, the CPU fetches instructions and owns the data path, and the serial pins are multiplexed to runtime UART. The GUI pads unused IMEM words with RV32I NOP 0x00000013 and supplies the required initialized DMEM extent.

Verification. Loader simulation checks word alignment, byte-to-word assembly, address bounds, IMEM/DMEM selection, and the transition from reset to instruction fetch after RUN. Runtime tests check byte strobes and synchronous-load wait behavior.

3.5.3. SoC Bus and MMIO Integration

riscv_ai_soc decodes the CPU data address and selects DMEM or one MMIO block. The implemented regions are 0x1000_0000 for result handoff, 0x2000_0000 for runtime UART, 0x3000_0000 for SD-SPI, 0x4000_0000 through 0x4000_0048 for the accelerator, 0x4000_1000 for the write-only result stream, and 0x5000_0000 for UI control and preview words.

Read data, wait, and fault signals return through one mux. The accelerator permits aligned 32-bit accesses only. The image-preview window packs 800 bits into 25 words. The result handoff uses a commit pulse so the display domain observes a coherent sample ID, prediction, confidence, and image-valid state.

Verification. Bus tests exercise every region, unmapped addresses, access sizes, waits, faults, result commits, and preview-word order. Figure 3.10 distinguishes memory ownership from runtime address decoding.

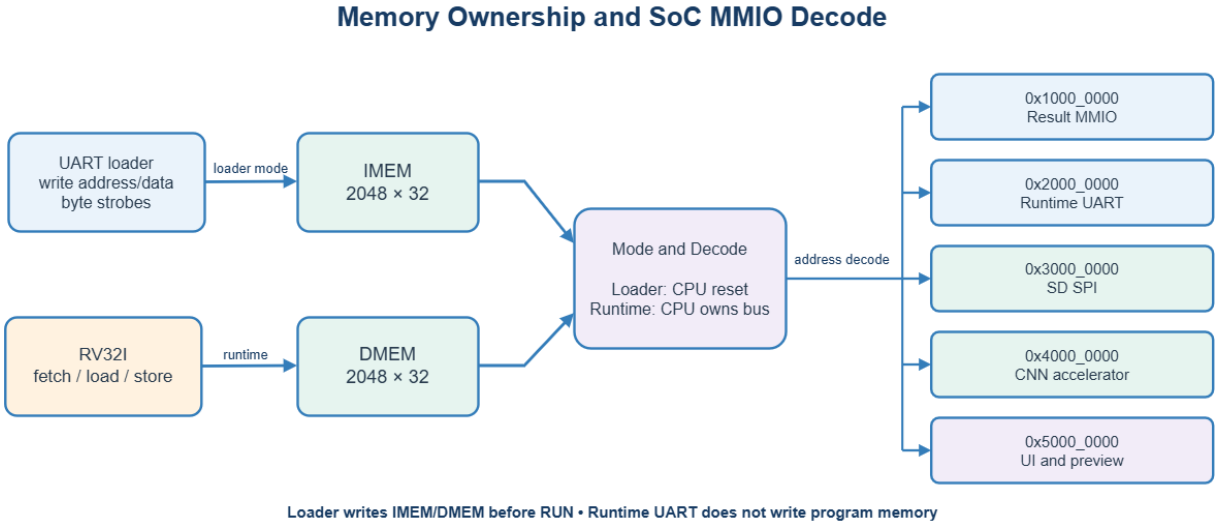


Figure 3.10. MMIO interconnection of the implemented RISC-V CNN SoC.

3.6. Peripheral and I/O Interface Design

The I/O subsystem contains the UART loader and runtime UART, SD-SPI, result and preview interfaces, status indicators, and HDMI. The onboard BL616 bridge is used for serial communication; no external CP2102 is required.

3.6.1. UART Bootloader

UART Receiver

uart_rx synchronizes the asynchronous input, detects the falling start edge, confirms the start bit near its midpoint, samples eight LSB-first data bits at the configured baud interval, validates the stop bit, and pulses rx_valid for one received byte. Its states are IDLE, START_BIT, DATA_BITS, STOP_BIT, and CLEANUP.

UART Transmitter

uart_tx accepts a byte only while tx_ready is high. It serializes one low start bit, eight LSB-first data bits, and one high stop bit. Its busy interval prevents the bootloader or runtime MMIO block from overwriting an active transmission.

UART Bootloader and Packet Protocol

uart_bootloader searches for 0x55 and 0xAA, then receives command, four little-endian byte-address bytes, two payload-length bytes, payload, and four CRC32 bytes. The supported commands are 0x01 WRITE_IMEM, 0x02 WRITE_DMEM, 0x03 RUN, and 0x04 PING. Payload length is at most 256 bytes. The loader replies ACK 0x79, PONG 0xA5, or NACK 0x1F.

The receive FSM has explicit states for both magic bytes, command, each address and length byte, payload, each CRC byte, and transmitter arm/wait states. It checks command legality, alignment, bounds, length, complete-word writes, and CRC before committing data or releasing the CPU.

Runtime UART MMIO

After RUN, uart_mmio exposes RX_DATA, TX_DATA, STATUS, and CONTROL at 0x2000_0000 through 0x2000_000C. Firmware reads received bytes, writes diagnostic and protocol replies, and polls availability and transmitter readiness.

Loader Runtime Multiplexer

The top-level TX mux selects the bootloader while boot_active is asserted and the runtime transmitter after CPU release. RX can feed both receivers, but only the owner may act on a protocol. A reload request resets the CPU and returns TX ownership to the loader.

Verification. Simulation must show start, eight data bits, stop, rx_valid, magic synchronization, bounds and CRC decisions, IMEM/DMEM writes, PING/PONG, ACK/NACK, RUN, and ownership switching. Board evidence requires one GUI capture with the BL616 COM port, IMEM and DMEM at 100 percent, CRC success, and RUN acknowledged.

Table 3.8. UART loader packet format and response bytes

Field	Size	Meaning
Magic	2 bytes	0x55 0xAA frame synchronization
Command	1 byte	01 IMEM 02 DMEM 03 RUN 04 PING
Byte address	4 bytes	Little-endian destination or RUN sample value
Length	2 bytes	Payload length maximum 256
Payload	0-256 bytes	Firmware data

Field	Size	Meaning
CRC32	4 bytes	IEEE zlib CRC over command address length and payload
Response	1 byte	ACK 0x79 PONG 0xA5 NACK 0x1F

3.6.2. SD-SPI and Raw RCN1 Storage

SD SPI Hardware and MMIO

sd_spi_mmio is a byte-oriented SPI master. CONTROL at offset 0x00 selects chip-select and starts a byte transfer, STATUS at 0x04 reports busy or complete state, DATA at 0x08 carries the transmit and received byte, and DIVIDER at 0x0C sets the SPI clock rate. The RTL controls electrical transfers; firmware implements the card command protocol.

Initialization and CMD17 Read

Firmware clocks the card with chip-select inactive, enters SPI mode, checks the operating conditions, completes initialization, and selects a faster divider when permitted. CMD17 reads one 512-byte sector: firmware sends the command and address, waits for the response and data token, receives exactly 512 data bytes plus CRC bytes, and releases chip-select.

CMD24 Write and Read Back

CMD24 sends one sector with a data token and waits for the card's data-response and busy release. The Dataset Manager then reads the same sector with CMD17 and compares all 512 bytes. A failed command, timeout, response, read-back, image CRC, or record CRC prevents the record-valid flag from being committed.

RCN1 Model and Dataset Organization

Sector zero contains the RCN1 v3 header, section table, configuration metadata, and header CRC32. Sector-aligned model sections store Conv1, Conv2, FC1, and FC2 parameters plus quantization metadata. The dataset begins at sector 408 and contains 100 fixed-size records. Figure 3.4 shows the logical layout; it is a raw application image, not a FAT file system.

Dataset Update Transaction

The host issues begin, chunk, and commit messages to Dataset Manager. Firmware first invalidates the destination record, accepts the 784-byte image in chunks of at most

128 bytes, constructs affected sectors, writes and reads them back, verifies both CRC levels, and sets valid only after success. AI Runtime is reloaded afterward so the new slot can be selected and displayed.

External Image Transfer and Runtime Restoration

When the user selects Replace Image on microSD, the GUI first probes Dataset Manager with CMD_STATUS. If AI Runtime owns the UART, the probe is expected to fail. The application then waits for the user to hold the external pin-72 button for approximately 0.5 s and polls the loader for up to 20 s until PING receives PONG. Dataset Manager is loaded only after the loader is detected.

CMD_SLOT_BEGIN carries the selected slot number, sample ID, class label, Conv2 calibration maximum, and ten confidence bytes. The GUI sends the 784-byte INT8 image as seven 112-byte CMD_SLOT_WRITE packets, which remain below the firmware limit of 128 payload bytes. CMD_SLOT_COMMIT carries the image CRC32. Dataset Manager writes the affected sectors, reads them back, compares the data, verifies image and record CRC values, and marks the record valid only after all checks pass.

After a successful commit, the GUI requests a return to the loader, waits 0.7 s, and retries PING while the mode changes. It then reloads AI Runtime and places the selected slot number in the RUN address field. The new image is displayed immediately, while inference remains an explicit action initiated by S1. This separation prevents a file-selection action from being mistaken for a completed board inference.

Any invalid slot or label, missing digit, serial timeout, microSD error, read-back mismatch, or CRC failure aborts the operation and leaves the record invalid. The application reports the error in both the progress state and the UART/System Log. A previously valid record is never reported as the newly committed image unless the complete transaction succeeds.

Verification. The SD testbench and full-system simulation check initialization, CMD17, CMD24, data-token boundaries, 512-byte transfer counts, timeout paths, read-back, and CRC failure handling. Board claims require the corresponding dated GUI or logic-analyzer capture.

3.6.3. HDMI and User Interaction

Result MMIO and Clock Domain Handoff

Firmware writes prediction, sample ID, confidence, and control fields in result_mmio, then emits a commit pulse. The UI preview window supplies 25 words, or 800 bits, containing 784 monochrome pixels plus padding. The display path consumes a stable committed snapshot rather than sampling changing CPU writes.

Pixel Timing Generator

hdmi_digit_renderer generates CEA-861 720 × 480p timing at a 27.000 MHz pixel clock. Horizontal timing is 720 active, 16 front porch, 62 sync, and 60 back porch pixels. Vertical timing is 480 active, 9 front porch, 6 sync, and 30 back porch lines. Sync pulses are active low.

Image Digit and Status Renderer

The renderer is zero-framebuffer. Pixel counters, built-in digit glyphs, committed status fields, and the 28 × 28 preview determine RGB values on demand. The display presents the selected sample, prediction, confidence when valid, image preview, and operating status without storing a complete video frame.

TMDS Encoding and Serialization

Three tmds_encoder instances convert 8-bit RGB channels into 10-bit symbols while controlling transition count and DC balance. During blanking, the blue channel carries HSYNC and VSYNC control symbols. Four Gowin OSER10 primitives serialize the clock and three data channels at 135 MHz, and ELVDS output buffers drive the differential HDMI pairs.

Verification. RTL simulation checks counter limits, sync polarity, data-enable windows, representative RGB regions, control symbols, and 10-bit serialization order. Physical evidence requires a photograph of the real 720 × 480p monitor output with sample ID, image, prediction, confidence or status, and the board in the same setup.

Table 3.9. Principal board-level interfaces

Interface	Pin or address	Role
Onboard UART RX	FPGA pin 70	BL616 to loader or runtime receiver
Onboard UART TX	FPGA pin 69	Loader or runtime reply to BL616
Reload button	FPGA pin 72	External active-high 0.5 s loader return
microSD	SPI pins	Raw RCN1 v3 sector access
HDMI	4 differential pairs	Clock and three TMDS data channels
S1 S2 LEDs	Board GPIO	Run select and status

3.7. Firmware Design and Hardware–Software Integration

The 8 KiB IMEM limit requires two deployable firmware images. AI Runtime performs inference and display. Dataset Manager performs verified slot updates. Shared drivers implement UART, SD-SPI, RCN1 parsing, accelerator control, runtime messages, and low-level startup.

3.7.1. Bare-Metal Software Structure

start.S initializes the stack and C environment. main.c selects the application role. uart.c, sd_spi.c, accelerator.c, model_image.c, cnn.c, and runtime_protocol.c separate peripheral access, model parsing, layer execution, and host messages. The linker places code in IMEM and initialized data, BSS, workspace, and stack in DMEM.

AI Runtime reports initialization progress, validates storage metadata, waits for S1 or a host run request, and publishes explicit errors instead of continuing with incomplete tensors. Dataset Manager uses transactional record updates so power loss or a failed transfer does not expose a partially updated slot as valid.

3.7.2. Loading and Integer Inference

For each accelerated layer, firmware clears the peripheral, writes dimensions and ReLU control, packs and streams inputs and weights, validates loaded counts, writes START, polls STATUS with a timeout, checks ERROR_CODE, reads OUTPUT_COUNT results, and requantizes them for the next software stage. Figure 3.11 shows the full path.

After Conv1, firmware performs Pool1 and requantization. It repeats the accelerator sequence for Conv2, then performs Pool2, FC1 with ReLU, FC2, and Argmax. FC weights are read from microSD as needed to avoid retaining the entire dense model in the 8 KiB DMEM.

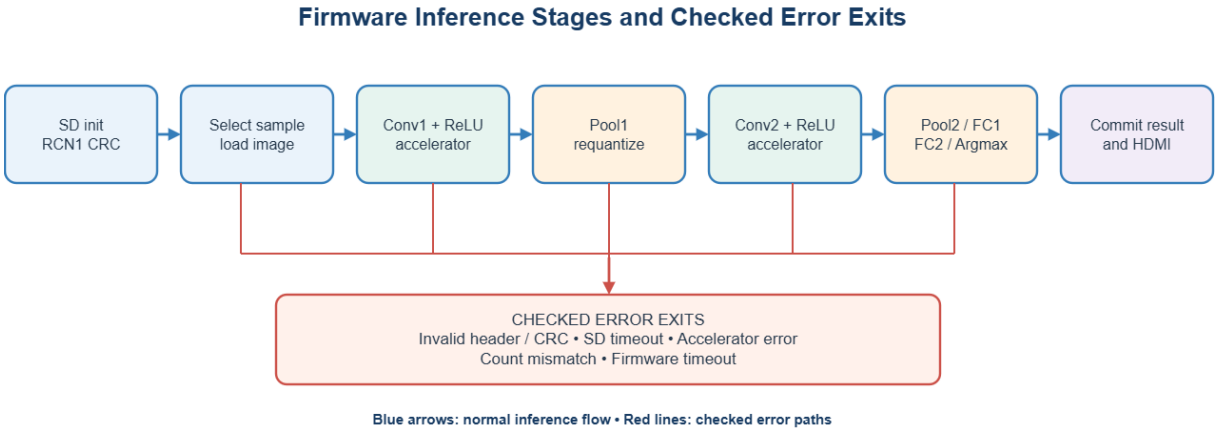


Figure 3.9. Firmware inference stages and error exits.

3.7.3. Driver Sequence and Error Handling

Every driver has bounded polling. SD timeouts, invalid response tokens, header or section CRC failures, invalid image records, accelerator ERROR, unexpected output counts, and runtime message errors return a diagnostic status. Firmware does not publish a new prediction until all required stages complete.

The host log, runtime UART, LEDs, and HDMI status fields provide different levels of observability. A simulation pass demonstrates digital behavior in the modeled environment; it does not by itself prove the same transaction on the board.

3.7.4. One Complete User Transaction

The user selects a sample with S2 or a host command. Firmware reads and validates the record, updates the preview, and waits for an inference trigger. S1 release starts the two accelerator calls and software layers. After Argmax, firmware writes the prediction and confidence, commits the result, and returns to the ready state. Table 3.10 identifies the owner and observable result at each step.

The sample selector computes next = (current + step) mod 100 with step selected from 3, 7, 9, or 11. Each step is coprime with 100, so repeated addition visits all 100 indices once before returning to the start. At the cycle boundary, a 7-bit LFSR changes

the offset and chooses a new step. The design therefore avoids a 100-entry permutation memory while preventing early repetition.

Stored Dataset Mode

In the normal board-only workflow, S2 advances through the 100 verified records without changing microSD contents. The preview and sample ID update first, and S1 launches inference for the visible record. This mode is appropriate for repeatable demonstrations and the version-matched 100-image accuracy run.

External Image Mode

In the external-image workflow, the user selects a file, chooses a target slot, enters a sample ID and ground-truth class, and reviews the original and processed previews. The application updates the chosen RCN1 record through Dataset Manager, restores AI Runtime, and starts it with the replaced slot selected. The user then presses S1 to obtain the FPGA prediction on HDMI.

The desktop host prediction is shown before transfer so that an obviously incorrect crop or threshold can be detected early. It is a diagnostic result from the host implementation, not the reported hardware result. A complete experiment links the selected file, processed 28 x 28 tensor, slot, label, host prediction, image CRC32, FPGA prediction, and HDMI capture.

Figures 3.12 and 3.13 make the external-image workflow visible at the operator level. Figure 3.12 identifies the controls before a file is selected. Figure 3.13 then shows the selected source image beside the 28 x 28 representation that will be stored on microSD.

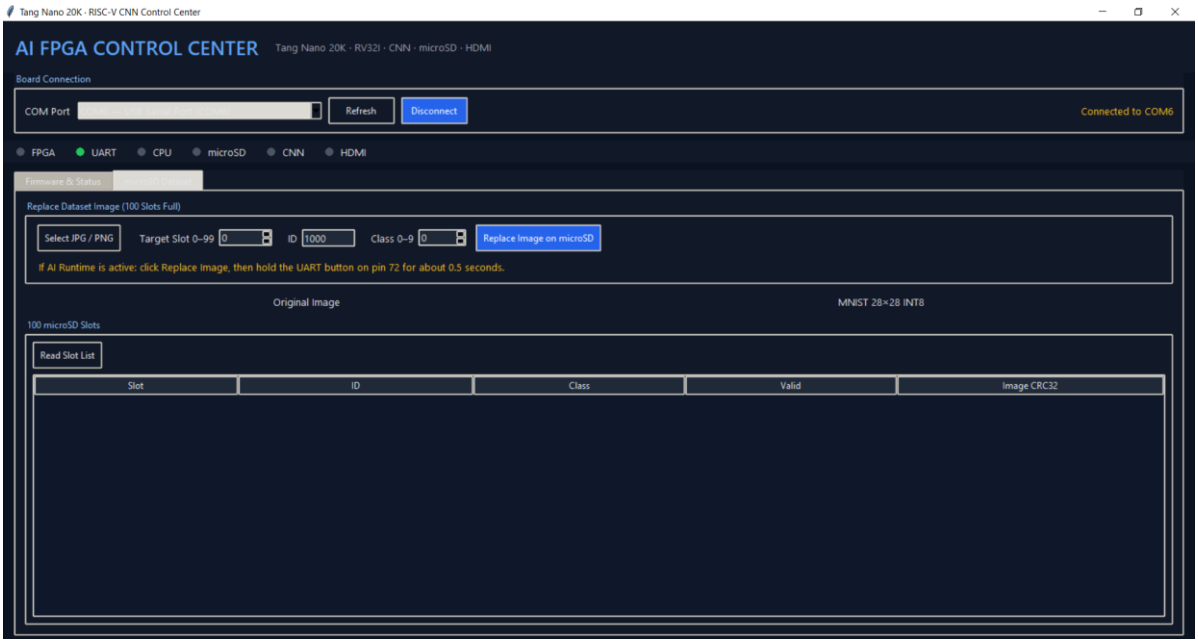


Figure 3.12. External-image dataset tab before file selection, showing the target slot, sample ID, class, and replacement controls.

This first capture is an orientation image. It shows where the user selects a JPG or PNG file and enters the destination metadata, but it does not yet demonstrate preprocessing, microSD transfer, CRC verification, or inference.

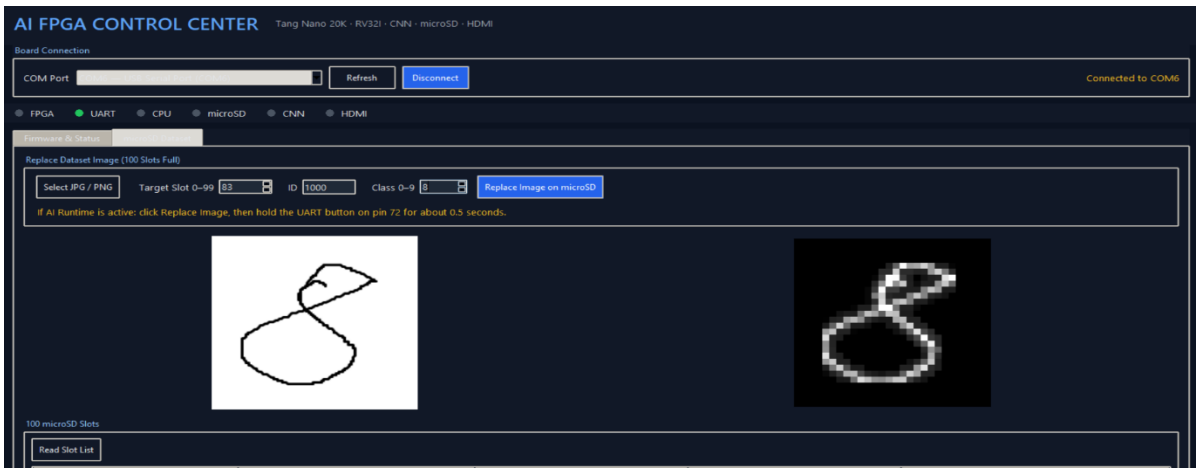


Figure 3.13. External handwritten digit 8 prepared for target slot 83 with assigned ID 1000 and class 8; the original image is shown on the left and the processed 28 x 28 input on the right.

Figure 3.13 demonstrates the conversion boundary clearly: the left pane preserves the user-supplied drawing, while the right pane shows the normalized low-resolution image that the deployed CNN will receive. The capture records the target slot and assigned class, so it can be paired with the later HDMI result.

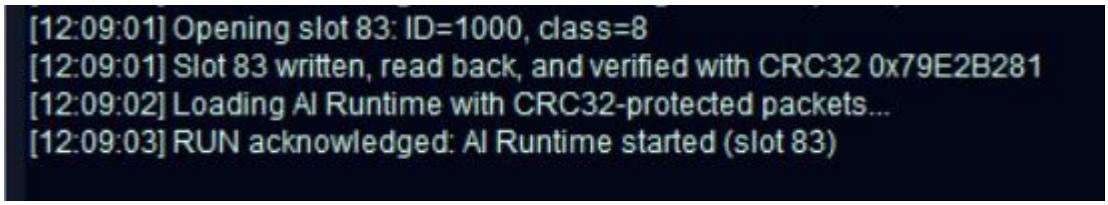


Figure 3.14. UART log confirming custom-image replacement, CRC32 verification, and AI Runtime restart from slot 83.

The UART log confirms that the custom image was written to slot 83 with ID 1000 and class 8. The record was subsequently read back and verified using CRC32 value 0x79E2B281. After reloading the AI Runtime, the processor acknowledged the RUN command and started inference from slot 83.

Table 3.10. End-to-end transaction from selection to display

Stage	Owner	Observable result
Select sample	S2 and UI logic	New ID and preview
Read image model	RV32I and SD-SPI	Valid header sections and record
Conv1 Conv2	CNN accelerator	DONE count and INT32 outputs
Pooling dense Argmax	RV32I firmware	Ten logits and selected class
Commit	Result MMIO	Atomic prediction confidence sample update
Display	HDMI pipeline	720 × 480p dashboard
Select external file	Desktop GUI	Original and 28 x 28 INT8 previews plus host reference result
Prepare replacement record	Desktop GUI	Slot ID label calibration and confidence metadata
Return to loader	Pin-72 reload and UART loader	PING PONG confirms loader ownership
Write and verify slot	Dataset Manager and SD-SPI	784 image bytes read back and CRC32 accepted

Stage	Owner	Observable result
Restore selected slot	GUI and UART loader	AI Runtime RUN acknowledged with the replacement slot
Run external image	S1 FPGA and HDMI	FPGA prediction and confidence appear for the selected ID

3.8. FPGA Implementation

Implementation uses Gowin EDA for synthesis, place-and-route, timing analysis, bitstream generation, and external-Flash programming. ModelSim verifies modules and the integrated firmware path. Python scripts export the model and SD image, and the control center coordinates deployment and dataset maintenance.

3.8.1. Source Organization and Tools

Connection

AI_FPGA_Control.py scans serial ports, lets the user select the onboard BL616 COM interface, opens or closes the connection, sends PING, and reports the active mode.

Firmware Loader

The loader selects AI Runtime or Dataset Manager, transfers IMEM and DMEM with progress bars and CRC acknowledgments, sends RUN, and records each reply. Firmware can be replaced after a pin-72 reload without programming the .fs image again.

Board Status and UART Log

Status indicators and the console distinguish FPGA configuration, UART connection, loader or CPU mode, SD status, CNN activity, and HDMI lock when the corresponding information is available. The log records commands, progress, acknowledgments, errors, and runtime messages.

Image Processor and Dataset Manager

The image workflow previews both the original file and the processed 28 × 28 tensor, assigns slot, sample ID, and label, computes CRC values, runs Dataset Manager, verifies the commit, then restores AI Runtime. The GUI also writes a complete RCN1 v3 image to a physical microSD card when requested.

Desktop Operation and Concurrency

Lengthy firmware and microSD operations execute in a worker thread. Progress, log, lamp, mode, slot-list, completion, and error events return to the Tk interface through a queue, so serial work does not block window updates. A busy flag rejects a second operation while the first transaction is active, preventing overlapping firmware loads or slot writes on the same COM port.

Input checks are performed before the first write: a processed image and its metadata must exist, the slot must be within 0-99, and the class must be within 0-9. Runtime responses also validate the command echo, payload length, status code, and response CRC32. These checks make the GUI part of the deployment control path rather than a passive serial terminal.

FPGA Programmer

The hardware action programs and verifies the final .fs file in external Flash. This operation changes the fixed FPGA layer and is separate from routine firmware loading or dataset updates.

Table 3.11. Project sources and build roles

Source or tool	Role
src/*.v	SoC CPU loader SD accelerator UI and HDMI RTL
firmware_c	AI Runtime Dataset Manager and peripheral drivers
tools/ai_fpga_gui.py	COM control, firmware loading, external-image preprocessing, host reference metadata, verified slot replacement, runtime restoration, progress, and logs
tools/export_sd_image.py	RCN1 v3 model and dataset export
tb and transcript	Module and full-firmware simulation evidence
impl/pnr	Final .fs resource and timing reports

3.8.2. Constraints, Build, and Deployment

The constraint file assigns the 27 MHz input, onboard BL616 UART RX pin 70 and TX pin 69, external reload pin 72, buttons, LEDs, microSD signals, and HDMI TMDS pairs. Timing constraints cover the 27 MHz system clock, 5.4 MHz SoC clock, 27 MHz pixel clock, and HDMI serialization clocks.

The reproducible build order is firmware compile and RV32I check, IMEM and DMEM image generation, RCN1 v3 export, RTL simulation, Gowin synthesis and place-and-route, timing review, .fs programming, UART loading, and end-to-end board capture. Chapter 4 uses the most recent inspected report rather than older screenshots.

CHAPTER 4: EXPERIMENTAL RESULTS AND EVALUATION

This chapter evaluates the final implementation using four evidence classes: offline model results, full-path simulation, Gowin implementation reports, and physical-board captures. The supplied GUI and HDMI images are interpreted only for what is visible in each capture. A separate procedure is defined for the new external-image mode so that preprocessing, microSD replacement, runtime restoration, and FPGA inference can be traced as one experiment.

4.1. Experimental Setup

The evaluation baseline is the final source tree, the exported RCN1 v3 image, the recorded full-firmware ModelSim transcript, and the latest Gowin place-and-route and timing reports dated 17 October 2026. Board captures must use matching .fs, firmware, and SD-image versions.

4.1.1. Hardware Configuration

The target is the Tang Nano 20K with GW2AR-LV18QN88C8/I7. The board uses its 27 MHz oscillator, onboard BL616 USB-to-UART bridge on FPGA RX pin 70 and TX pin 69, microSD socket, S1/S2 buttons, LEDs, and HDMI connector. A separate active-high pushbutton connects pin 72 to 3V3 for firmware reload. No external CP2102 is part of the final setup.

The GUI transfers AI Runtime or Dataset Manager at 115200-8-N-1, programs the .fs image to external Flash when hardware changes, writes the raw RCN1 image to microSD, prepares replacement images, and records acknowledgments and runtime messages.

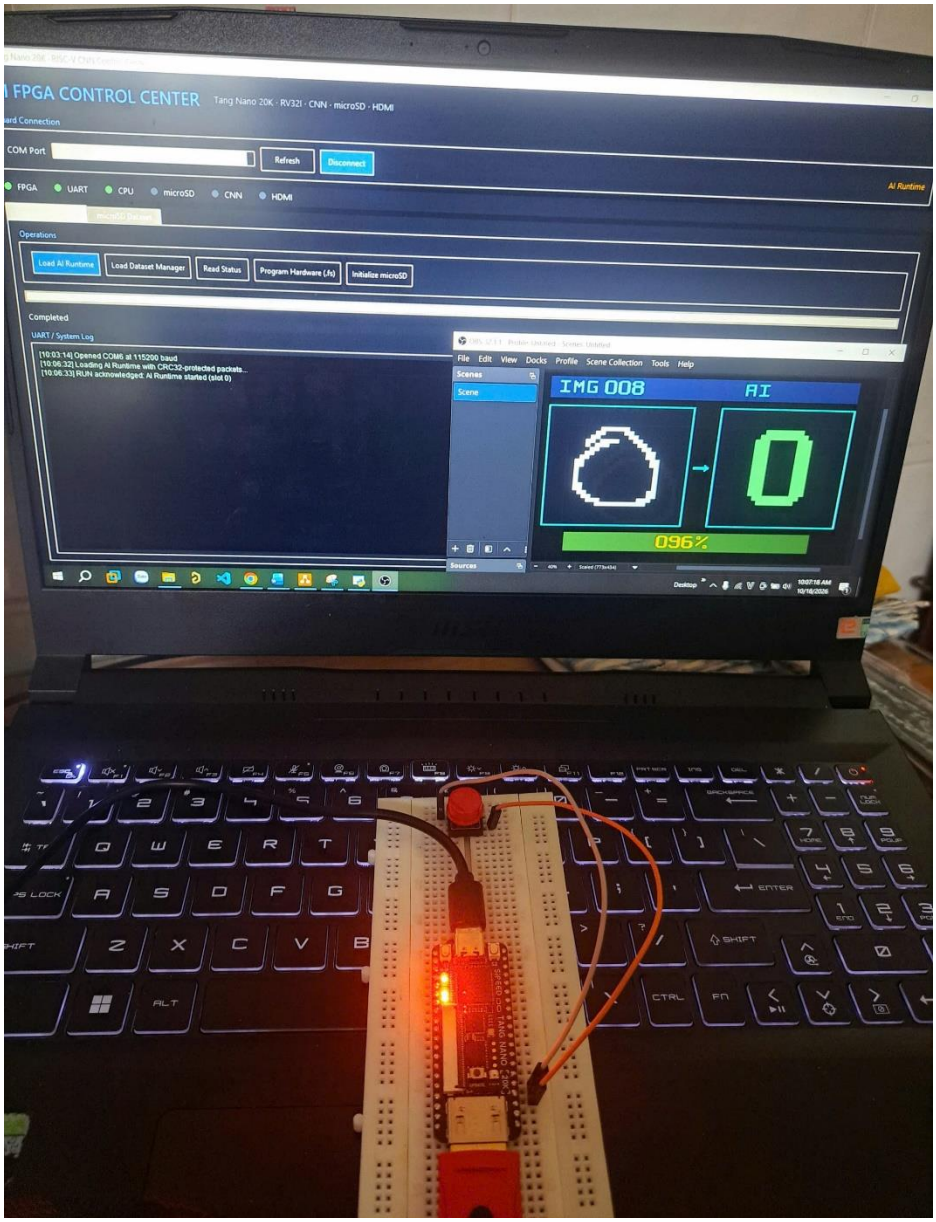


Figure 4.1. Experimental setup of the Tang Nano 20K RISC-V CNN system with the PC control interface and HDMI output preview.

4.1.2. Evidence and Evaluation Scope

Model evidence includes MNIST Float32 accuracy, simulated INT8 accuracy, and the separate custom 100-image integer reference. Simulation evidence includes the stored full CNN transcript. Implementation evidence includes the resource and timing

reports. Physical-board evidence requires photographs, GUI logs, or logic-analyzer traces obtained from the stated version set.

The current artifacts do not contain a version-matched 100-sample board log or defensible end-to-end latency and software-only baseline. These values are therefore listed as required measurements rather than inferred from offline accuracy or RTL throughput.

Version and Traceability Record

Each board record should identify the .fs build, AI Runtime and Dataset Manager binaries, RCN1 image version or CRC, selected COM port, slot, sample ID, assigned label, and test time. These identifiers are required because the hardware, firmware, and dataset can be updated independently. A display photograph without the matching log verifies the visible result but cannot identify the exact deployed build.

External Image Acceptance Criteria

The external-image path passes only when one test record shows the selected source file, the processed 28 x 28 preview, host reference prediction, destination slot and ID, loader recovery, Dataset Manager load, all seven image chunks, read-back and CRC success, AI Runtime reload with the same slot, S1 inference, and the matching HDMI result. A failure in preprocessing, serial transfer, SD verification, or runtime restoration must be reported as a failed test rather than as a classification result.

4.2. FPGA Implementation Results

The latest inspected Gowin report supersedes the older screenshots and values embedded in the previous report. Tables 4.1 and 4.2 transcribe the current report and identify the remaining timing limitation.

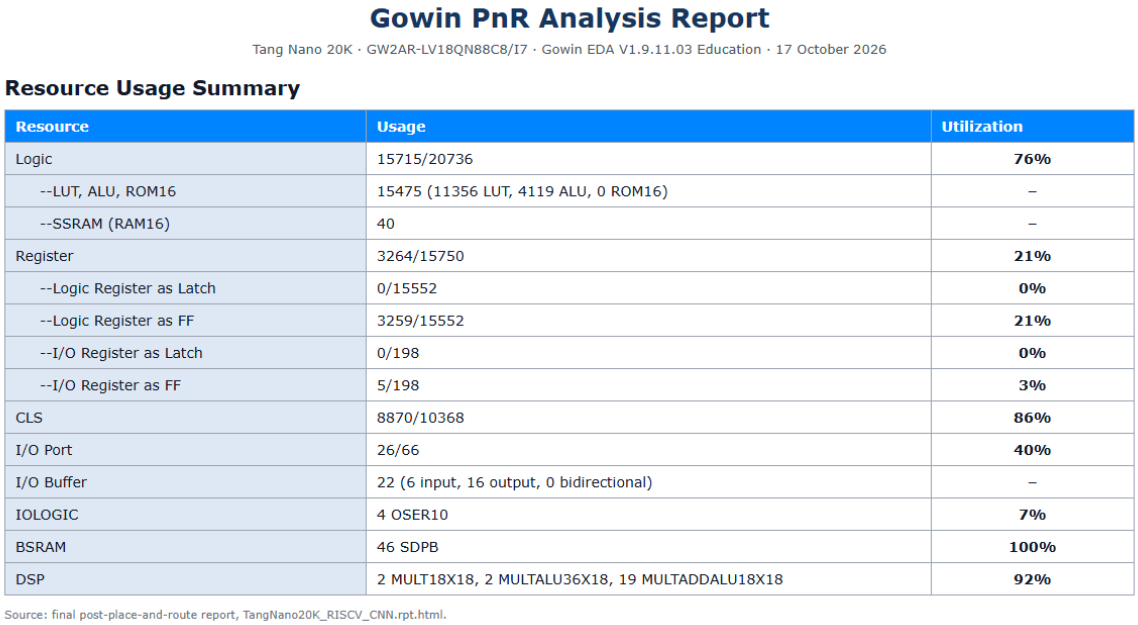
4.2.1. FPGA Resource Utilization

The design uses 15,715 of 20,736 logic resources, 3,264 of 15,750 registers, 8,870 of 10,368 CLS, 26 of 66 I/O ports, all 46 BSRAM blocks, and 22 of 24 DSP blocks. Four OSER10 primitives implement HDMI clock and data serialization.

BSRAM at 100 percent and DSP at 92 percent are the tightest capacity constraints. A larger model, wider PE array, or larger on-chip memories would require resource reduction, external streaming, or a larger FPGA.

Table 4.1. FPGA resource utilization reported in the supplied Gowin screenshots.

Resource	Usage	Utilization
Logic	15,715 / 20,736	76%
Registers	3,264 / 15,750	21%
CLS	8,870 / 10,368	86%
I/O ports	26 / 66	40%
BSRAM	46 / 46	100%
DSP	22 / 24	92%
OSER10	4	HDMI serialization



Gowin PnR Analysis Report

Tang Nano 20K · GW2AR-LV18QN88C8/I7 · Gowin EDA V1.9.11.03 Education · 17 October 2026

Resource Usage Summary

Resource	Usage	Utilization
Logic	15715/20736	**76%**
--LUT, ALU, ROM16	15475 (11356 LUT, 4119 ALU, 0 ROM16)	–
--SSRAM (RAM16)	40	–
Register	3264/15750	**21%**
--Logic Register as Latch	0/15552	**0%**
--Logic Register as FF	3259/15552	**21%**
--I/O Register as Latch	0/198	**0%**
--I/O Register as FF	5/198	**3%**
CLS	8870/10368	**86%**
I/O Port	26/66	**40%**
I/O Buffer	22 (6 input, 16 output, 0 bidirectional)	–
IOLOGIC	4 OSER10	**7%**
BSRAM	46 SDPB	**100%**
DSP	2 MULT18X18, 2 MULTALU36X18, 19 MULTADDALU18X18	**92%**

Source: final post-place-and-route report, TangNano20K_RISCV_CNN.rpt.html.

Figure 4.2. Gowin resource screenshots showing the values transcribed in Table 4.1.

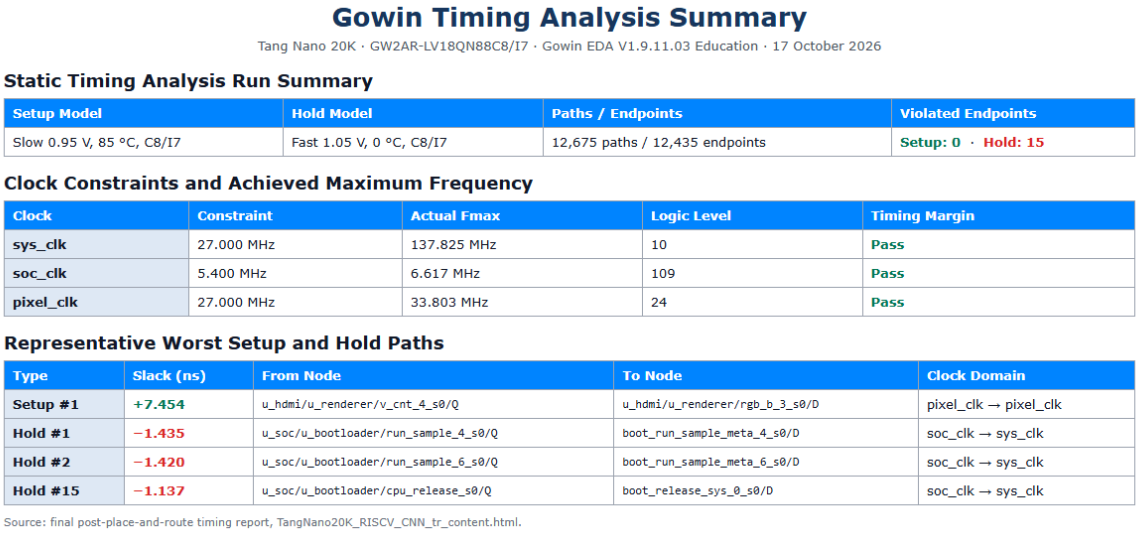
4.2.2. Timing Analysis

The report lists zero setup-violated endpoints. The reported Fmax values are 137.825 MHz for sys_clk against 27 MHz, 6.617 MHz for soc_clk against 5.4 MHz, and 33.803 MHz for pixel_clk against 27 MHz. The SoC clock therefore has the smallest setup margin.

The same report lists fifteen hold-violated endpoints. Several involve multi-clock control or handoff paths. Complete timing closure is not claimed. Constraint intent, clock-domain crossings, pulse synchronizers, and hold paths must be reviewed and the design reimplemented until the report shows zero setup and zero hold violations.

Table 4.2. Frequency and timing values shown in Gowin

Clock or endpoint	Constraint	Reported result	Status
sys_clk	27.000 MHz	Fmax 137.825 MHz	Setup passes
soc_clk	5.400 MHz	Fmax 6.617 MHz	Setup passes limited margin
pixel_clk	27.000 MHz	Fmax 33.803 MHz	Setup passes
Endpoint summary	-	0 setup violations 15 hold violations	Hold closure unresolved



Gowin Timing Analysis Summary

Tang Nano 20K · GW2AR-LV18QN88C8/I7 · Gowin EDA V1.9.11.03 Education · 17 October 2026

Static Timing Analysis Run Summary

Setup Model	Hold Model	Paths / Endpoints	Violated Endpoints
Slow 0.95 V, 85 °C, C8/I7	Fast 1.05 V, 0 °C, C8/I7	12,675 paths / 12,435 endpoints	Setup: 0 · Hold: 15

Clock Constraints and Achieved Maximum Frequency

Clock	Constraint	Actual Fmax	Logic Level	Timing Margin
sys_clk	27.000 MHz	137.825 MHz	10	Pass
soc_clk	5.400 MHz	6.617 MHz	109	Pass
pixel_clk	27.000 MHz	33.803 MHz	24	Pass

Representative Worst Setup and Hold Paths

Type	Slack (ns)	From Node	To Node	Clock Domain
Setup #1	+7.454	u_hdmi/u_renderer/v_cnt_4_s0/Q	u_hdmi/u_renderer/rgb_b_3_s0/D	pixel_clk → pixel_clk
Hold #1	−1.435	u_soc/u_bootloader/run_sample_4_s0/Q	boot_run_sample_meta_4_s0/D	soc_clk → sys_clk
Hold #2	−1.420	u_soc/u_bootloader/run_sample_6_s0/Q	boot_run_sample_meta_6_s0/D	soc_clk → sys_clk
Hold #15	−1.137	u_soc/u_bootloader/cpu_release_s0/Q	boot_release_sys_0_s0/D	soc_clk → sys_clk

Source: final post-place-and-route timing report, TangNano20K_RISCV_CNN_tr_content.html.

Figure 4.3. Setup and Hold Path Slack Values

4.3. Hardware Deployment Results

The source implements external-Flash programming, BL616 UART loading, pin-72 reload, verified SD updates, GUI status, and HDMI output. The following subsections state the exact evidence required to elevate each implemented path to board-verified status.

4.3.1. FPGA Configuration and Firmware Loading

Figure 4.4 records COM6 opened at 115200 baud and two separate AI Runtime load sequences. The first sequence ends with RUN acknowledged at 10:06:33, and the second ends with another RUN acknowledgment at 11:26:33. The repeated successful loads show that the application can transfer and start the runtime more than once in the same GUI session.

The visible log is summary evidence. It does not display the raw PING/PONG exchange, individual IMEM and DMEM percentages, or per-packet ACK and CRC values, and a still image cannot prove that the second load followed a continuous 0.5 s pin-72 hold without a power cycle. A complete reload record should therefore retain the raw serial trace or a continuous screen recording in addition to Figure 4.4.

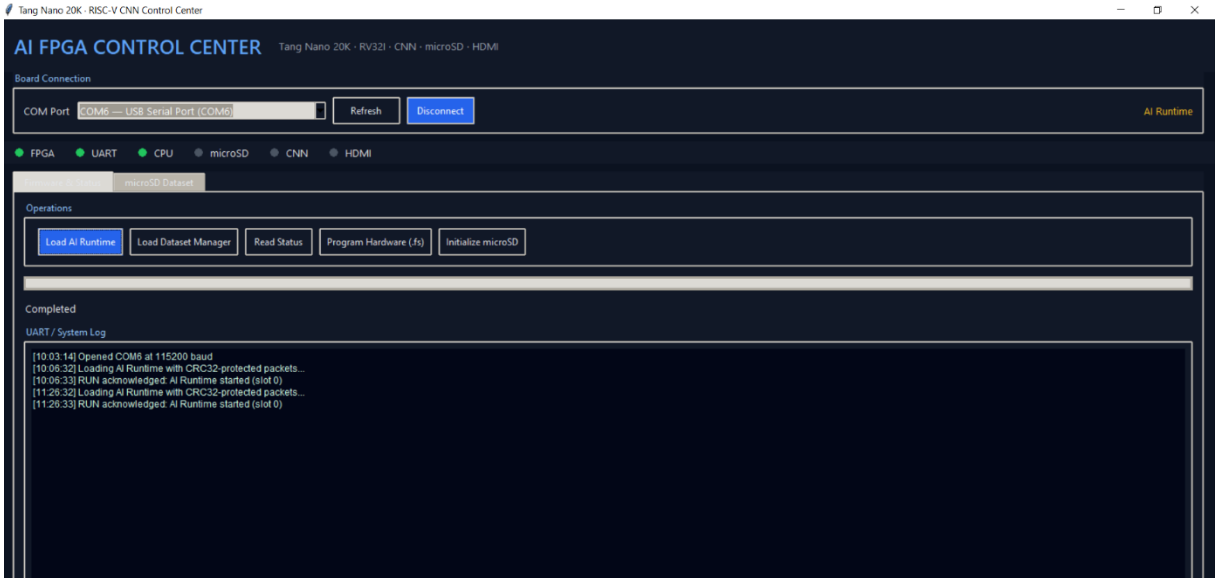


Figure 4.4. Successful UART firmware loading before and after the 0.5 s pin-72 reload request.

4.3.2. Peripheral and User Interface Operation

The implemented peripheral path covers SD initialization, CMD17 sector reads, CMD24 sector writes, sector read-back, byte comparison, image CRC, record CRC, and final valid commit. The GUI exposes the original file, processed 28 x 28 preview, target slot, sample ID, label, progress, slot list, and completion log. HDMI displays the selected ID, input preview, prediction, and confidence or status after AI Runtime executes the record.

External Image Board Procedure

The test starts with AI Runtime active and a known free or replaceable slot. The operator selects a JPG, PNG, JPEG, or BMP file and checks that the 28 x 28 preview still represents the intended digit. The operator then records the target slot, a unique sample ID, and the ground-truth class before selecting Replace Image on microSD.

If the runtime owns the UART, the GUI asks for the pin-72 reload and waits for the loader. Dataset Manager is loaded, the slot is invalidated, seven image chunks are

written, and commit is accepted only after microSD read-back and CRC verification. The GUI then restores AI Runtime with the replacement slot selected. S1 starts inference, and the resulting HDMI ID and prediction must match the same slot recorded in the GUI log.

Negative and Boundary Cases

The validation set should include a light-background photograph that requires inversion, an off-center digit, a thin stroke, and a difficult digit with low confidence. It should also test a blank file, an invalid class, an out-of-range slot, an interrupted UART connection, and a forced CRC or SD write failure. The expected result for these negative cases is a clear error with no valid partial record.

4.4. CNN Inference Results

The training metrics report 99.5 percent Float32 accuracy and 99.5 percent simulated INT8 accuracy on the MNIST test set. The exporter and integer reference report 98 correct classifications from the separate custom 100-image set. These datasets and execution environments are different and are not merged into one accuracy number.

The recorded ModelSim run loads firmware through the CRC32 packet loader, initializes the modeled SD card, executes the CNN using the accelerator and software stages, and ends with “RV32I C FULL CNN FROM SD PASSED prediction=0.” The run reports zero errors and one warning that the preload file contained one extra data word. This is full-path simulation evidence.

4.4.1. Single-Sample Inference Demonstration

Figure 4.5 shows the displayed record IMG 001, a digit 0 input preview, a predicted digit 0, and 100 percent confidence. This is direct visual evidence that the HDMI renderer can present one correct inference result with a coherent sample ID, input image, predicted class, and confidence value.

The cropped display image does not include the GUI log, assigned label, build identifiers, or SD-image CRC. It therefore verifies the visible single-sample result but does not by itself establish whether IMG 001 came from the original dataset or from the external-image update mode. The full test record should pair this display with the corresponding UART and GUI entries.

External Image Inference Evaluation

For a user-supplied image, the source file and processed preview are part of the experimental input. The assigned label is the ground truth supplied by the operator, the host prediction is a preprocessing reference, and the FPGA prediction is the result under evaluation. Reporting these three values separately reveals whether a disagreement originates in labeling, image conversion, or the deployed hardware and firmware path.

A successful external-image demonstration must show more than a plausible HDMI digit. The GUI must confirm the destination slot and CRC-verified commit, AI Runtime must restart with that slot, and the displayed sample ID must match the replacement record. This closes the chain from desktop file to microSD bytes and then to FPGA inference.

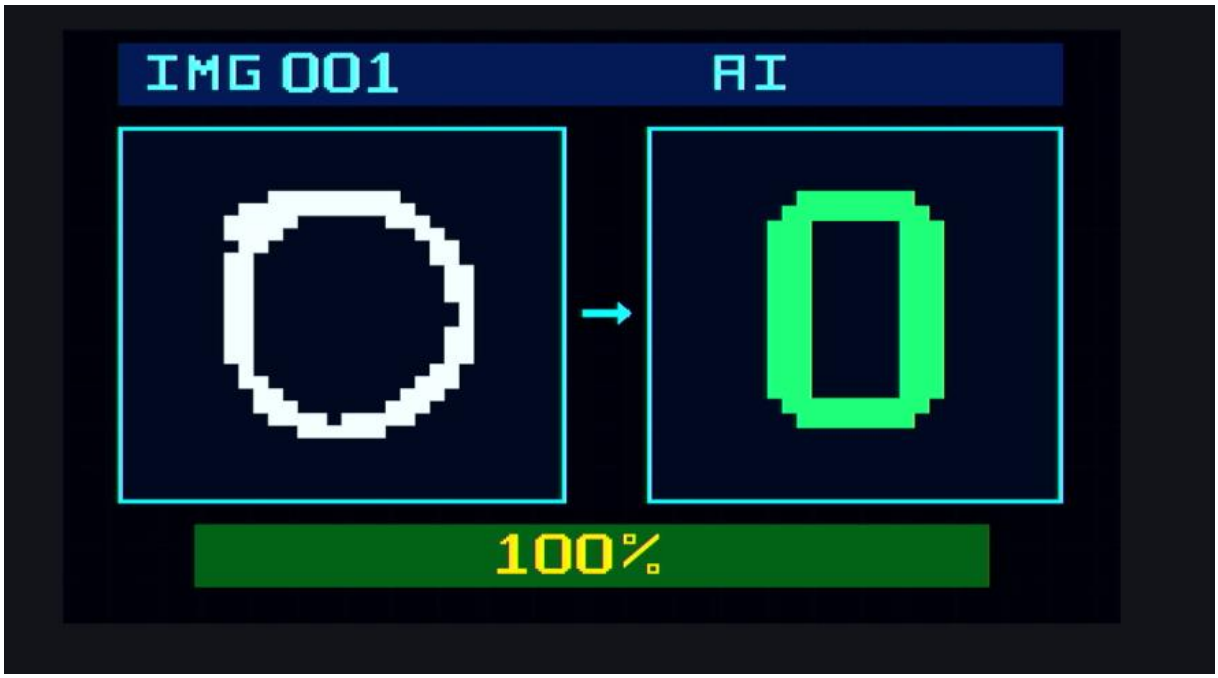


Figure 4.5. HDMI output for IMG 001 showing a correct prediction of digit 0 with 100% confidence.

4.4.2. Multiple-Sample Recognition Results

Figure 4.6 provides three board display cases. IMG 065 is labeled 6 and is predicted as 6 with 49 percent confidence. IMG 074 is labeled 7 and is predicted as 7 with 97 percent confidence. IMG 094 is labeled 9 but is predicted as 3 with 60 percent confidence. The set therefore includes two correct results with different confidence levels and one visible misclassification rather than only favorable examples.

These three displays confirm that the result path carries different sample IDs, classes, and confidence values, and that an incorrect prediction is still reported rather than hidden. They do not replace the 100-record board run. The selector sequence and each result must still be captured under one build set before a physical-board accuracy percentage is calculated.

The same sampling rule applies to external images. Replacing a slot changes the content associated with that position but does not change the selector's 100-position cycle. The test log must preserve both the slot index and the user-assigned sample ID so an overwritten record is not confused with the original image that occupied the slot.

Figure 4.7 completes the visible part of the example begun in Figure 3.13. The externally supplied handwritten digit is classified as 8 by the FPGA and is displayed with 85 percent confidence on HDMI.



Figure 4.7. HDMI result for the externally supplied handwritten digit 8, reported as class 8 with 85% confidence.

The visible mapping is consistent with zero-based target slot 83 being presented as display position IMG 084. However, the HDMI capture does not display the user-entered ID 1000. The final evidence set should therefore retain the slot-list row or the matching runtime log so that the relationship among target slot 83, assigned ID 1000, and IMG 084 is explicit.



Figure 4.6. HDMI inference results for multiple samples: correct predictions for IMG 065 and IMG 074, and a misclassification of IMG 094 as digit 3.

Table 4.3. FPGA display results and ground-truth labels

Dataset and execution	Result	Interpretation
MNIST test set Float32	99.5%	Training notebook metric
MNIST test set simulated INT8	99.5%	Exported-model simulation metric
Custom 100-image integer reference	98 / 100	Offline host integer pipeline
Physical board 100-image run	Not yet recorded	Requires version-matched board log

4.4.3. Classification Accuracy Evaluation

Table 4.3 separates the available accuracy results. MNIST Float32 and simulated INT8 metrics evaluate the trained model. The 98/100 custom result evaluates the exported integer reference on another set. Board-wide accuracy remains unmeasured until all 100 records are run under one version-matched configuration.

The external-image feature should initially be evaluated as a traceability and functional test, not as a new accuracy dataset. One or a few manually selected files can verify conversion and transfer, but they cannot support a statistically meaningful

accuracy claim. If external-image accuracy is required, the source images, labels, preprocessing settings, and failure exclusions must be fixed before the batch is run.

Table 4.4. Real-image evidence plan for the external-image workflow

Status	Evidence image	Required visible content
Included	GUI ready state	Figure 3.12 shows the external-image tab, target slot, ID, class, and replacement control.
Included	Source and processed image	Figure 3.13 shows the handwritten digit 8, target slot 83, assigned ID 1000, class 8, and the processed 28 x 28 input.
Still required	Transfer and CRC log	Capture PING/PONG, all seven image-write packets, read-back, accepted image and record CRC values, AI Runtime reload, and RUN ACK.
Still required	Full hardware context	Photograph the Tang Nano 20K, microSD card, pin-72 reload button, HDMI monitor, and matching GUI log in the same test session.
Included with traceability note	HDMI result	Figure 4.7 shows IMG 084, prediction 8, and 85 percent confidence. Retain the matching GUI or UART log to explain the slot and ID mapping.
Recommended	Rejected update	Capture one invalid-slot, interrupted-transfer, or CRC-failure case and show that no partial record becomes valid.

Table 4.4 distinguishes orientation images from proof of a completed board transaction. The two remaining priority captures are the CRC-verified UART log and one full-system photograph that ties the GUI, board, reload control, microSD card, and HDMI output to the same run.

4.5. Results Discussion and Limitations

The available evidence establishes a complete source implementation, a passing UART-loaded full-CNN simulation from the SD model, and a successful FPGA implementation report for the target device. It does not establish final hold closure, board-wide accuracy, end-to-end latency, or speedup over a software-only RV32I implementation.

4.5.1. Interpretation of the Evidence

The accelerator targets the two convolution layers that account for most learned-layer MACs, but firmware and SD transfers still contribute to latency. Peak 16-MAC-per-cycle array throughput is an upper bound and must not be substituted for measured application speed. Boundary masks, feeder scheduling, MMIO loading, SD reads, pooling, requantization, and dense layers all affect end-to-end time.

The latest P&R data also exposes limited expansion capacity. All BSRAM blocks and 22 of 24 DSPs are occupied, while the 5.4 MHz SoC path has modest setup headroom and fifteen hold violations remain. These constraints should guide future optimization priorities.

Effect of the External Image Path

The new mode extends the demonstration beyond the prebuilt 100-image set without modifying the FPGA bitstream or retraining the network. Its main cost is operational latency: the system must leave AI Runtime, load Dataset Manager, write and verify microSD sectors, return to the loader, reload AI Runtime, and then wait for S1. This latency belongs to dataset maintenance and should be reported separately from CNN inference latency.

The fixed-slot design improves reproducibility because the exact INT8 bytes used by the FPGA remain stored on microSD and are identified by an image CRC32. It also creates a limitation: every external submission overwrites an existing slot, and an incorrect operator label becomes part of the record metadata. The experiment log should retain the original filename and preserve a copy of the processed 784-byte tensor or its CRC.

4.5.2. Current Limits and Next Measurements

The final board test should power the board from Flash, connect the GUI, load IMEM and DMEM, run firmware, validate microSD, select a sample, execute Conv1, Pool1 and requantization, Conv2, Pool2, FC1, FC2 and Argmax, commit the result, and photograph HDMI. Timestamped logs should measure SD reads, each MMIO transfer, each accelerator call, software post-processing, and total inference.

The same test should repeat all 100 sample IDs and store ID, label, prediction, confidence or status, error code, and timing under one bitstream, firmware, and SD-

image version. A software-only RV32I path on the same integer tensors is required as the speedup denominator.

Measurements for External Image Operation

The GUI should timestamp preprocessing, loader detection, Dataset Manager loading, slot write, read-back verification, AI Runtime restoration, S1 press, and result publication. Report preprocessing time, dataset-update time, runtime-reload time, CNN inference time, and total user-visible time separately. The current screenshots do not provide these measurements, so no latency value is inferred from them.

A compact regression set should cover digits 0-9 with varied background brightness, stroke width, scale, and centering. For every item, store the source filename, assigned label, host prediction, slot, sample ID, image CRC32, FPGA prediction, confidence, status, and outcome. This dataset can then distinguish preprocessing failures from model errors and transport or storage failures.

4.6. RISC-V Instruction-Group Validation

Before integration with the complete AI SoC, the processor core was exercised in a dedicated standalone validation build. That test configuration used a compact 256-word (1 KiB) instruction memory and an idle-timed UART loader, whereas the integrated AI Runtime uses the 8 KiB packet-loader configuration described in Section 3.5.2. The two configurations share the RV32I execution core, but their memory capacity and program-start procedure are reported separately to avoid treating the validation harness as the final SoC specification.

The standalone test program covered arithmetic and logic, shifts and comparisons, data memory, conditional branches, upper immediates, and jumps. All six instruction groups completed successfully on the Tang Nano 20K. Loading the program through UART allowed the instruction image to be changed between runs without rebuilding the FPGA configuration.

4.6.1. Standalone Hardware Setup and UART Loading

Figure 4.8 shows the Tang Nano 20K validation setup connected to the host computer. For the recorded run, the host-side flow generated program.bin and run_program_test.py opened COM8 at 115,200 baud, transmitted the image, and retained the serial connection to collect the execution report. The image contained 133

32-bit instructions, or 532 bytes, which fit within the 1 KiB instruction memory used by this standalone build.

The validation loader assembled incoming bytes into little-endian 32-bit instruction words and wrote them sequentially from instruction address zero. The CPU remained in reset during reception and started automatically after the complete transfer and the loader's 100 ms idle interval. Therefore, no manual start-button action was required for this test sequence.

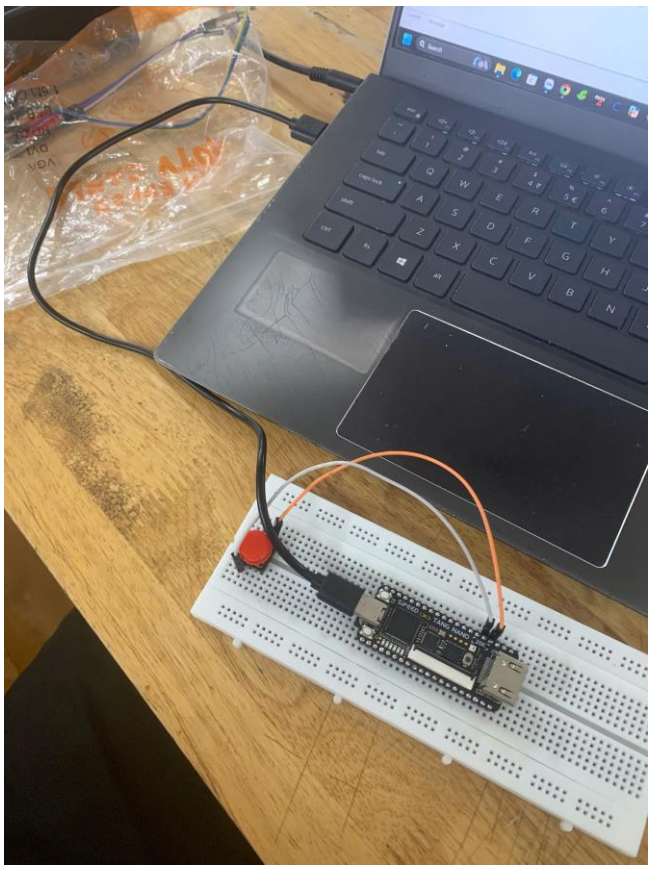


Figure 4.8. Tang Nano 20K setup used for standalone UART-based RV32I instruction testing.

4.6.2. Instruction Coverage and Results

The assembly program compared each operation result with its expected value and reported the completed group through UART. A passing group emitted its test number and a pass marker; a detected mismatch emitted a failure marker and stopped the sequence. Table 4.5 lists the implemented coverage for the recorded run.

Table 4.5. RV32I instruction-group validation coverage and results

Test	Instruction coverage	Result
1	ADD, SUB, ADDI, XOR/XORI, OR/ORI, AND/ANDI	PASS
2	SLL/SLLI, SRL/SRLI, SRA/SRAI, SLT/SLTI, SLTU/SLTIU	PASS
3	SW and LW	PASS
4	BEQ, BNE, BLT, BGE, BLTU and BGEU	PASS
5	LUI and AUIPC	PASS
6	JAL, JALR and link-address checks	PASS

The captured terminal output in Figure 4.9 reports all six groups as passed and confirms that the program reached its completion marker. These results validate the listed instruction behaviors for the tested operands and execution paths; they do not constitute exhaustive verification of every RV32I encoding or corner case.

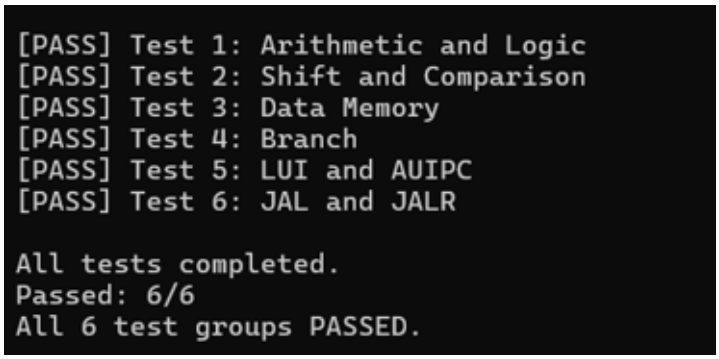


Figure 4.9. UART test output confirming that all six RV32I instruction groups passed.

CHAPTER 5: CONCLUSION AND FUTURE WORK

This chapter summarizes the implemented system at the evidence level supported by the final source, simulation, and Gowin reports, then identifies the work required for timing closure and quantitative board evaluation.

5.1. Project Conclusion

The project implements a RISC-V-controlled CNN AI SoC for the Tang Nano 20K. The fixed FPGA layer integrates the RV32I processor, separately loadable 8 KiB IMEM and DMEM, onboard BL616 UART loader and runtime UART, MMIO interconnect, SD-SPI controller, modular 4 × 4 INT8 convolution accelerator, result and preview handoff, buttons, LEDs, and a 720 × 480p HDMI pipeline.

The hardware/software partition is explicit. Conv1 and Conv2 with ReLU execute in RTL, while the RV32I firmware performs storage management, pooling, requantization, FC1, FC2, Argmax, and result publication. The RCN1 v3 image stores model parameters, quantization metadata, and 100 updateable records. Dataset Manager uses invalidation, sector write, read-back, CRC verification, and final commit to avoid exposing a partial update.

The host application unifies FPGA programming, BL616 connection, IMEM/DMEM loading, SD initialization, preprocessing, dataset update, progress, and logs. Firmware can be replaced after a 0.5 s pin-72 reload without rebuilding the FPGA hardware, and an image in classes 0-9 can be replaced without retraining the model.

Available results include 99.5 percent Float32 and simulated-INT8 MNIST accuracy, 98/100 on the separate custom integer reference, and a passing full-CNN ModelSim run from UART-loaded firmware and the SD model. The latest P&R report uses 76 percent of logic, 100 percent of BSRAM, and 92 percent of DSP. Setup constraints pass, but fifteen hold violations remain; complete timing closure and board-wide accuracy are not claimed.

5.2. System Limitations

- All 46 BSRAM blocks and 22 of 24 DSP blocks are used, leaving little capacity for larger memories, a wider array, or a larger model.
- The accelerator executes convolution and ReLU only. Pooling, requantization, dense layers, and Argmax remain processor workloads.

- The 5.4 MHz SoC path has a reported Fmax of 6.617 MHz, so setup headroom is modest.
- Fifteen hold-violated endpoints remain in the latest timing report and require CDC, constraint, and implementation review.
- A version-matched 100-sample physical-board log, end-to-end latency, accelerator-only latency, and software-only baseline have not yet been recorded.
- The raw RCN1 image is application-specific and does not provide FAT file-system interoperability.

5.3. Future Development

Timing and CDC closure is the first priority. Multi-clock controls and pulse crossings should be audited, synchronized or handshaken as required, constrained according to their real relationships, and reimplemented until both setup- and hold-violation counts are zero. The final timing capture must come from the same build as the delivered .fs file.

Performance work should add cycle counters around SD reads, MMIO transfers, each convolution, software post-processing, and total inference. A software-only RV32I path should run the same quantized tensors to provide a valid speedup denominator. DMA, double buffering, better operand reuse, and a more deeply pipelined feeder can then be evaluated against measured bottlenecks.

Model scalability may be improved through per-channel quantization, parameterized PE arrays, selective dense-layer acceleration, and external-memory streaming. Because BSRAM and DSP use are already high, every extension requires a resource and timing budget or migration to a larger FPGA.

Dataset maintenance can be extended with stronger version identifiers, transactional metadata for the complete SD image, automatic model-package updates, and richer GUI diagnostics. Supporting classes beyond 0-9 requires retraining and a changed final output layer; it is not only a dataset update.

REFERENCES

[1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, version 20260120, Jan. 2026. [Online]. Available: https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html.

[2] RISC-V International, "Ratified specifications," 2026. [Online]. Available: https://riscv.org/specifications/ratified/.

[3] RISC-V International, "RISC-V Instruction Set Architecture (ISA) Manuals," 2026. [Online]. Available: https://docs.riscv.org/reference/isa/.

[4] RISC-V International, *RISC-V Architecture Profiles: Introduction*. [Online]. Available: https://docs.riscv.org/reference/rva20-rvi20-rva22/intro.html.

[5] RISC-V International, "RISC-V Ratified Specifications Library," 2026. [Online]. Available: https://docs.riscv.org/.

[6] RISC-V International, "Why develop a new ISA?," *History and Acknowledgments*. [Online]. Available: https://docs.riscv.org/reference/isa/v20191213/_attachments/riscv-unprivileged.pdf.

[7] University of California, Berkeley, "Introduction," *CS 61C Course Notes*.

[8] University of California, Berkeley, "Instruction timing," *CS 61C Course Notes*.

[9] RISC-V International, "RV32I base integer instruction set, version 2.1," in *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, release 20240411, 2024. [Online]. Available: https://docs.riscv.org/reference/isa/v20240411/unpriv/rv32.html. [Accessed: Oct. 9, 2026].

[10] University of California, Berkeley, "Supporting branches," *CS 61C Course Notes*.

[11] University of California, Berkeley, "State elements on the datapath," *CS 61C Course Notes*.

[12] University of California, Berkeley, "Control logic design," *CS 61C Course Notes*.

[13] University of California, Berkeley, "Supporting immediates," *CS 61C Course Notes*.

[14] University of California, Berkeley, "R-type datapath, ALU," *CS 61C Course Notes*.

[15] University of California, Berkeley, “Supporting loads and stores,” *CS 61C Course Notes*.

[16] Cornell University, “Pipelining & performance,” *CS 3410 Course Notes*, Fall 2025. [Online]. Available: https://www.cs.cornell.edu/courses/cs3410/2025fa/notes/pipelining.html. [Accessed: Oct. 9, 2026].

[17] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.

[18] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[19] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, May 2015, doi: 10.1038/nature14539.

[20] PyTorch, “Conv2d,” *PyTorch Documentation*. [Online]. Available: https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html. [Accessed: Sep. 28, 2026].

[21] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning,” *arXiv*, arXiv:1603.07285, 2016, revised 2018.

[22] V. Nair and G. E. Hinton, “Rectified linear units improve restricted Boltzmann machines,” in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 807–814.

[23] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[24] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017, doi: 10.1109/JPROC.2017.2761740.

[25] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017, doi: 10.1109/JSSC.2016.2616357.

[26] B. Jacob *et al*., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286.

[27] Sipeed, *Sipeed Tang Nano 20K Datasheet*, version 1.3.

[28] PyTorch, “Training a classifier,” *PyTorch Tutorials*. [Online]. Available: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html. [Accessed: Oct. 11, 2026].

[29] GOWIN Semiconductor Corp., *GW2AR Series of FPGA Products Datasheet*, DS226, version 2.7E, Jul. 2026. [Online]. Available: https://www.gowinsemi.com/en/document/main/database/1846/.

[30] A. Paszke *et al*., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[31] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST database of handwritten digits.” [Online]. Available: https://yann.lecun.org/exdb/mnist/. [Accessed: Oct. 11, 2026].